

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống hỏi đáp tài liệu kỹ thuật tiếng
Việt với tinh chỉnh embedding và chưng cất mô
hình xếp hạng lại**

TRẦN QUANG HUY

huy.tq225201@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lê Thanh Hương – Khoa Khoa học máy tính
TS. Đỗ Tiến Dũng – Khoa Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 07/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Xây dựng hệ thống hỏi đáp tài liệu kỹ thuật tiếng
Việt với tinh chỉnh embedding và chưng cất mô
hình xếp hạng lại**

TRẦN QUANG HUY
huy.tq225201@sis.hust.edu.vn

Chương trình đào tạo: Kỹ thuật máy tính

Giảng viên hướng dẫn: PGS. TS. Lê Thanh Hương – Khoa Khoa học máy tính
TS. Đỗ Tiến Dũng – Khoa Kỹ thuật máy tính

Trường: Công nghệ thông tin và Truyền thông

Chữ ký GVHD

Chữ ký GVHD

HÀ NỘI, 07/2026

LỜI CẢM ƠN

Trước hết, em xin gửi lời cảm ơn sâu sắc đến PGS. TS. Lê Thanh Hương, giảng viên hướng dẫn chính của em, người đã trực tiếp định hướng, kiên nhẫn góp ý và đồng hành cùng em qua từng giai đoạn của đề án. Những chỉ dẫn chuyên môn, sự tận tình và những góp ý quý báu của cô đã giúp em nhìn nhận vấn đề rõ ràng hơn, từng bước hoàn thiện đề tài và có thêm nhiều kinh nghiệm trong quá trình nghiên cứu, xây dựng hệ thống.

Em cũng xin chân thành cảm ơn TS. Đỗ Tiến Dũng, giảng viên đồng hướng dẫn, đã dành thời gian hỗ trợ trong quá trình em thực hiện đề án.

Em xin cảm ơn các thầy cô Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội, đã truyền đạt cho em không chỉ kiến thức chuyên môn mà còn cả tư duy học tập, nghiên cứu và tinh thần làm nghề trong suốt những năm đại học. Những kiến thức và trải nghiệm được tích lũy tại trường là nền tảng quan trọng giúp em có thể thực hiện và hoàn thành đề án này.

Con xin cảm ơn bố mẹ và gia đình, những người luôn là chỗ dựa vững vàng và lặng lẽ phía sau mọi cố gắng của con. Em cũng xin cảm ơn những người anh em, bạn bè đã cùng em học tập, chia sẻ, động viên và vượt qua nhiều giai đoạn khó khăn trong suốt hành trình đại học.

Cuối cùng, em muốn cảm ơn chính mình vì đã không ngừng nỗ lực, kiên trì và kiên định để đi đến cuối hành trình. Tất cả sẽ là một trong những kỷ niệm đẹp nhất của em trong những năm tháng tuổi đôi mươi.

Em xin chân thành cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hệ thống hỏi–đáp dựa trên truy hồi (Retrieval-Augmented Generation, RAG) cho tài liệu kỹ thuật tiếng Việt đối mặt với hai thách thức: chất lượng truy hồi trên văn bản chuyên ngành nhiều thuật ngữ và bảng biểu, cùng chi phí và độ trễ của tầng xếp hạng lại. Các giải pháp hiện tại thường dùng cross-encoder lớn như BGE-reranker-v2-m3 (568M tham số) để xếp hạng lại và một mô hình ngôn ngữ lớn qua API để định tuyến câu hỏi — cho độ chính xác cao nhưng nặng, chậm trên CPU và phát sinh chi phí, trong khi embedding nền tảng chưa được tối ưu cho miền dữ liệu. Đồ án tối ưu toàn bộ pipeline thành các thành phần nhẹ, chạy được trên CPU với chi phí thấp mà vẫn giữ chất lượng gần với mô hình lớn.

Embedding Vietnamese_Embedding_v2 được tinh chỉnh bằng hàm mất mát MultipleNegativesRankingLoss trên các cặp câu hỏi–positive của miền, kết hợp Matryoshka Representation Learning để một mô hình duy nhất cho biểu diễn dùng được ở ba số chiều 256/512/1024; cấu hình 512 chiều được chọn để triển khai, giảm một nửa dung lượng vector index. Reranker được chưng cất tri thức từ teacher BGE-reranker-v2-m3 sang student mmarco-mMiniLMv2 117M và một bản cắt tỉa từ vựng 35.6M bằng hàm mất mát ADR-MSE dựa trên thứ hạng, rồi lượng tử hoá INT8 để chạy trên CPU. Bộ định tuyến ý định dùng SetFit trên nền Sentence-BERT tiếng Việt, thay thế hoàn toàn lời gọi API.

Kết quả (trung bình trên ba hạt giống ngẫu nhiên), embedding tinh chỉnh cải thiện $\text{Acc}@1$ từ 65.19% lên $73.21 \pm 0.60\%$ trên tập in-domain và đạt $\text{MRR}@10 = 0.7982 \pm 0.0017$ ở 512 chiều trên tập tài liệu chưa từng xuất hiện khi huấn luyện. Reranker 35.6M đạt $\text{MRR}@10 = 0.8844$, vượt ViRanker 568M (0.8593) dù nhỏ hơn khoảng 16 lần; student 117M đạt 0.8862, bằng đúng teacher 568M. Sau lượng tử hoá INT8, reranker chỉ còn 34.8 MB và nhanh hơn teacher khoảng 10 lần trên CPU. Đánh giá đầu cuối trên 390 câu trắc nghiệm cho thấy pipeline tối ưu giữ nguyên độ chính xác của hệ thống ban đầu (97.4%) trong khi phần truy hồi–xếp hạng nhanh hơn khoảng 20 lần, chứng minh tính khả thi của một hệ thống RAG tiếng Việt hiệu quả với chi phí thấp.

Sinh viên thực hiện

Huy

Trần Quang Huy

ABSTRACT

Retrieval-Augmented Generation (RAG) systems for Vietnamese technical documents face two challenges: retrieval quality over specialized text dense with terminology and tables, and the cost and latency of the reranking stage. Existing solutions typically rely on a large cross-encoder such as BGE-reranker-v2-m3 (568M parameters) for reranking and a large language model accessed through an API for query routing — accurate, but heavy, slow on CPU, and costly, while the underlying embedding model is not optimized for the target domain. This thesis optimizes the entire pipeline into lightweight components that run on CPU at low cost while preserving quality close to that of large models.

The embedding model `Vietnamese_Embedding_v2` is fine-tuned with the `MultipleNegativesRankingLoss` on in-domain query–positive pairs, combined with `Matryoshka Representation Learning` so that a single model yields representations usable at 256, 512, and 1024 dimensions; the 512-dimensional configuration is chosen for deployment, halving the size of the vector index. The reranker is distilled from the BGE-reranker-v2-m3 teacher into an `mmarco-mMiniLMv2` 117M student and a vocabulary-pruned 35.6M variant using the rank-based ADR-MSE loss, then quantized to INT8 for CPU deployment. The intent router is trained with `SetFit` on a Vietnamese Sentence-BERT backbone, eliminating API calls entirely.

As a result (averaged over three random seeds), fine-tuning improves Acc@1 from 65.19% to $73.21 \pm 0.60\%$ on the in-domain set and reaches an MRR@10 of 0.7982 ± 0.0017 at 512 dimensions on documents never seen during training. The 35.6M reranker attains an MRR@10 of 0.8844, surpassing the 568M ViRanker (0.8593) despite being about 16 times smaller; the 117M student reaches 0.8862, matching the 568M teacher exactly. After INT8 quantization, the reranker occupies only 34.8 MB and runs roughly 10 times faster than the teacher on CPU. End-to-end evaluation on 390 multiple-choice questions shows that the optimized pipeline preserves the accuracy of the original system (97.4%) while making the retrieval–reranking stage about 20 times faster, demonstrating the feasibility of an efficient, low-cost Vietnamese RAG system.

MỤC LỤC

DANH MỤC TỪ VIẾT TẮT	ix
DANH MỤC THUẬT NGỮ.....	xi
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện có và hạn chế.....	1
1.3 Mục tiêu và định hướng giải pháp	2
1.4 Đóng góp đồ án	2
1.5 Bố cục đồ án	3
CHƯƠNG 2. TỔNG QUAN VÀ CÔNG TRÌNH LIÊN QUAN.....	5
2.1 Tổng quan về hệ thống RAG	5
2.1.1 Hạn chế của mô hình ngôn ngữ lớn thuần túy	5
2.1.2 Retrieval-Augmented Generation	5
2.1.3 Đặc thù của RAG cho tài liệu kỹ thuật tiếng Việt	6
2.2 Truy hồi thông tin.....	6
2.2.1 BM25 và truy hồi thưa (sparse retrieval)	6
2.2.2 Truy hồi dày (dense retrieval) và bi-encoder	7
2.2.3 Truy hồi lai và Reciprocal Rank Fusion.....	8
2.3 Xếp hạng lại bằng cross-encoder.....	9
2.3.1 Bi-encoder và cross-encoder	9
2.3.2 BGE-reranker-v2-m3.....	10
2.3.3 Chung cất tri thức cho reranker	10
2.3.4 Các hàm mất mát cho chung cất reranker	11
2.3.5 Nén reranker và cắt tỉa từ vựng.....	12

2.4 Mô hình nhúng văn bản và tinh chỉnh embedding	13
2.4.1 Từ Transformer đến Sentence-BERT	13
2.4.2 Mô hình embedding cho truy hồi	14
2.4.3 MultipleNegativesRankingLoss.....	14
2.4.4 False negatives và GISTEmbedLoss	14
2.4.5 Matryoshka Representation Learning.....	15
2.5 Mô hình ngôn ngữ lớn và thiết kế prompt trong RAG	16
2.5.1 Vai trò của LLM trong sinh câu trả lời.....	16
2.5.2 Kỹ thuật prompt Chain-of-Thought.....	16
2.6 Định tuyến truy vấn và phân loại ý định.....	16
2.6.1 Định tuyến truy vấn trong pipeline RAG	16
2.6.2 PhoBERT, SBERT và SetFit.....	17
2.6.3 Mất cân bằng lớp trong phân loại ý định	17
2.7 Các độ đo đánh giá	18
2.7.1 Độ đo cho truy hồi và xếp hạng lại	18
2.7.2 Độ đo cho phân loại ý định và pipeline	19
2.8 Khoảng trống nghiên cứu	19
CHƯƠNG 3. MÔ HÌNH ĐỀ XUẤT	21
3.1 Tổng quan kiến trúc hệ thống đề xuất	21
3.2 Vai trò của hệ thống v1 trong đồ án	23
3.3 Quy trình tiền xử lý tài liệu.....	23
3.3.1 Trích xuất văn bản từ tài liệu PDF	23
3.3.2 Chiến lược chia đoạn nhận biết bảng.....	25
3.3.3 Xây dựng chỉ mục	26
3.4 Kiến trúc hai pipeline RAG	27
3.4.1 Pipeline Baseline Router.....	27

3.4.2 Pipeline Router-Summary	28
3.4.3 So sánh kiến trúc hai pipeline.....	29
3.5 Xác định các nút thắt	29
3.5.1 Nút thắt độ trễ: BGE-reranker-v2-m3	30
3.5.2 Nút thắt chi phí: bộ định tuyến ý định GPT	30
3.5.3 Nút thắt chất lượng: embedding chưa tối ưu theo miền.....	30
3.6 Tổng quan định hướng v2.....	31
3.7 Tinh chỉnh embedding bằng MRL và resample positive theo epoch.....	31
3.7.1 Xác định positive cho embedding	31
3.7.2 Gộp positive và resample theo epoch	32
3.7.3 Thiết kế hàm mất mát cho tầng embedding.....	32
3.7.4 Cấu hình huấn luyện.....	33
3.7.5 Chiến lược triển khai với MRL.....	33
3.8 Reranker nhẹ bằng chưng cất tri thức.....	34
3.8.1 Động lực và thiết kế tổng quan.....	34
3.8.2 Điều chỉnh dữ liệu cho reranker.....	35
3.8.3 Giai đoạn A: Học tương phản.....	36
3.8.4 Giai đoạn B: Chưng cất tri thức	37
3.8.5 Cắt tỉa từ vựng cho mmarco	39
3.9 Tinh chỉnh bộ phân loại ý định Sentence-BERT.....	40
3.9.1 Kiến trúc và dữ liệu.....	40
3.9.2 Xử lý mất cân bằng lớp.....	42
3.9.3 Logic định tuyến trong pipeline v2	43
CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM	44
4.1 Thiết lập thực nghiệm	44
4.1.1 Môi trường và phần cứng.....	44

4.1.2 Tập dữ liệu đánh giá.....	44
4.1.3 Độ đo đánh giá	45
4.2 Đánh giá embedding	45
4.2.1 Thiết lập đánh giá	45
4.2.2 Kết quả trên hai tập kiểm thử	46
4.2.3 Phân tích số chiều biểu diễn MRL	47
4.2.4 Thử nghiệm mở rộng coverage bằng query tổng hợp	48
4.3 Đánh giá reranker	49
4.3.1 Kết quả chính trên tập kiểm thử reranker.....	49
4.3.2 Nghiên cứu loại bỏ thành phần	50
4.3.3 Vai trò của giai đoạn A và hiện tượng quên thăm khốc	52
4.3.4 Khảo sát trọng số α giữa chung cất và neo contrastive.....	53
4.3.5 Cắt tỉa từ vựng cho mmarco	54
4.3.6 Phân tích sự phụ thuộc của reranker vào độ mạnh của bộ truy hồi .	55
4.3.7 Khảo sát các kỹ thuật nén	56
4.3.8 Đánh giá triển khai trên CPU	58
4.4 Đánh giá bộ phân loại ý định	59
4.5 Đánh giá đầu cuối trên câu hỏi trắc nghiệm.....	61
4.5.1 Triển khai reranker ONNX INT8 trong pipeline đầu cuối	63
4.5.2 Phân rã nguồn lỗi đầu cuối.....	64
4.5.3 Kiểm chứng đối chiếu trên mẫu độc lập	65
4.6 Phân tích token ngoài từ vựng sau cắt tỉa	67
4.7 Khuyến nghị cấu hình triển khai.....	68
4.8 Phân tích lỗi tổng thể	68
CHƯƠNG 5. KẾT LUẬN.....	71
5.1 Tổng kết	71

5.2 Đóng góp chính	71
5.3 Hạn chế	72
5.4 Hướng phát triển	73
TÀI LIỆU THAM KHẢO	76
PHỤ LỤC A. MÔI TRƯỜNG VÀ CẤU HÌNH HUẤN LUYỆN	77
A.1 Môi trường phần cứng và phần mềm	77
A.2 Siêu tham số huấn luyện embedding	77
A.3 Siêu tham số huấn luyện reranker	78
A.4 Siêu tham số huấn luyện bộ phân loại ý định	78
PHỤ LỤC B. VÍ DỤ DỮ LIỆU HUẤN LUYỆN	79
B.1 Ví dụ bộ ba cho embedding.....	79
B.2 Ví dụ hai lớp ý định	79
PHỤ LỤC C. CÁC PROMPT SỬ DỤNG VỚI MÔ HÌNH NGÔN NGỮ .	81
C.1 Prompt định tuyến ý định.....	81
C.2 Prompt sinh câu trả lời theo ý định	81
C.3 Prompt giám khảo xác định positive	82
C.4 Prompt sinh query-positive bổ sung	83

DANH MỤC HÌNH VẼ

Hình 2.1	Kiến trúc tổng quan của hệ thống RAG	6
Hình 3.1	Kiến trúc tổng thể của hệ thống đề xuất	22
Hình 3.2	Luồng xử lý xây dựng chỉ mục	26
Hình 3.3	Luồng xử lý của pipeline Baseline	27
Hình 3.4	Luồng xử lý của pipeline Router-Summary	28
Hình 3.5	Quy trình xác định positive cho embedding	31
Hình 3.6	Kiến trúc huấn luyện reranker hai giai đoạn	34
Hình 3.7	Kiến trúc huấn luyện bộ phân loại ý định theo SetFit	41

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh BM25, truy hồi dày (dense) và truy hồi lai (hybrid)	8
Bảng 2.2	So sánh bi-encoder và cross-encoder	9
Bảng 2.3	Tóm tắt các nhóm hàm mất mát cho chung cất reranker . . .	12
Bảng 3.1	Vai trò của các tập dữ liệu trong quy trình phát triển	23
Bảng 3.2	So sánh kiến trúc Baseline Router và Router-Summary . . .	29
Bảng 3.3	Ánh xạ nút thất của v1 sang giải pháp v2	31
Bảng 3.4	Cấu hình huấn luyện embedding	33
Bảng 3.5	Các chế độ triển khai từ một checkpoint MRL	33
Bảng 3.6	Thống kê negative khai thác theo nguồn	36
Bảng 3.7	Tổng quan tập dữ liệu huấn luyện reranker	36
Bảng 3.8	Siêu tham số của giai đoạn A	37
Bảng 3.9	Ví dụ minh họa cách tính ADR-MSE với năm ứng viên . . .	39
Bảng 3.10	Siêu tham số của giai đoạn B	39
Bảng 3.11	Siêu tham số tinh chỉnh Sentence-BERT (SetFit) cho bộ phân loại ý định	42
Bảng 4.1	Các tập dữ liệu dùng trong đánh giá	44
Bảng 4.2	Các tập dữ liệu đánh giá embedding	45
Bảng 4.3	Kết quả trên tập D1 (158 câu)	46
Bảng 4.4	Kết quả trên tập D2 (390 câu)	46
Bảng 4.5	So sánh 512d và 1024d trên D1 và D2	47
Bảng 4.6	So sánh baseline và with_gen qua 3 seed (Acc@1, %) . . .	48
Bảng 4.7	Kết quả reranker trên bộ 390 câu hỏi	50
Bảng 4.8	So sánh cấu hình huấn luyện trên MiniLM, H384 và pruned H384	50
Bảng 4.9	So sánh các hàm mất mát chung cất trên MiniLM 33M (giai đoạn A no-mMARCO, giai đoạn B $\alpha = 0.7$)	51
Bảng 4.10	So sánh các hàm mất mát chung cất trên H384 (giai đoạn A, giai đoạn B $\alpha = 0.7$)	52
Bảng 4.11	Vai trò của giai đoạn A trên H384 (ADR-MSE, $\alpha = 0.7$) . .	52
Bảng 4.12	Khảo sát α trên pruned H384 (ADR-MSE, không giai đoạn A)	53
Bảng 4.13	Nén Mmarco bằng cắt tia từ vụng	54
Bảng 4.14	Chất lượng pruned reranker sau tinh chỉnh	55
Bảng 4.15	So sánh các reranker trên bộ truy hồi mạnh	55
Bảng 4.16	Cắt tia tầng: L12 so với L6 trên pruned H384	56

Bảng 4.17	Lượng tử hoá INT8 trên pruned 35.6M	57
Bảng 4.18	So sánh ba kỹ thuật nén	57
Bảng 4.19	Độ trễ trên CPU (16 luồng, rerank top-20)	58
Bảng 4.20	So sánh cấu hình ý định classifier theo F1-macro	59
Bảng 4.21	Đánh giá nội tại bộ phân loại ý định v2 (SetFit)	60
Bảng 4.22	So sánh bộ định tuyến v1 (GPT) và v2 (Sentence-BERT/SetFit)	60
Bảng 4.23	Đánh giá đầu cuối trên 390 câu trắc nghiệm: pipeline v1 so với v2	62
Bảng 4.24	Độ trễ trung bình theo thành phần (ms/câu)	62
Bảng 4.25	Dung lượng lưu trữ của embedding cache và index trong pipeline đầu cuối	63
Bảng 4.26	Ma trận định tuyến $\times$ đáp án cuối trên 390 câu trắc nghiệm .	65
Bảng 4.27	So sánh v1 và v2 trên mẫu đối chiếu độc lập (71 câu)	66
Bảng 4.28	Ma trận định tuyến $\times$ đáp án cuối trên mẫu đối chiếu (71 câu)	66
Bảng 4.29	Độ trễ trung bình theo thành phần trên mẫu đối chiếu (ms/câu)	66
Bảng 4.30	Các cấu hình pipeline khuyến nghị	68
Bảng A.1	Các thư viện chính sử dụng trong đồ án	77
Bảng A.2	Siêu tham số huấn luyện embedding	77
Bảng A.3	Siêu tham số huấn luyện reranker	78
Bảng A.4	Siêu tham số huấn luyện bộ phân loại ý định	78
Bảng B.1	Ví dụ một bộ ba huấn luyện embedding	79
Bảng B.2	Ví dụ câu hỏi hai lớp ý định	79

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
Acc	Accuracy	Độ chính xác
ADR-MSE	Approx Discounted Rank MSE	Hàm mất mát chung cắt dựa trên thứ hạng (Schlatt và cộng sự, 2025)
API	Application Programming Interface	Giao diện lập trình ứng dụng
BCE	Binary Cross-Entropy	Entropy chéo nhị phân
BERT	Bidirectional Encoder Representations from Transformers	Mô hình biểu diễn ngôn ngữ hai chiều dựa trên Transformer
BGE	BAAI General Embedding	Họ mô hình embedding/reranker của BAAI
BM25	Best Matching 25	Thuật toán xếp hạng văn bản theo từ khóa
CoT	Chain-of-Thought	Chuỗi suy luận từng bước
CPU	Central Processing Unit	Bộ xử lý trung tâm
F1	F1-score	Trung bình điều hòa giữa precision và recall
FP32	32-bit Floating Point	Kiểu số thực dấu phẩy động 32 bit, độ chính xác mặc định của trọng số mô hình
GIST	Guided In-sample Selection of Training Negatives	Chọn lọc negative trong batch có hướng dẫn
GPT	Generative Pre-trained Transformer	Mô hình Transformer sinh ngôn ngữ
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
Hit	Hit rate	Tỉ lệ truy hồi có chứa đáp án đúng trong top- k
HTML	HyperText Markup Language	Ngôn ngữ đánh dấu siêu văn bản
INT8	8-bit Integer	Kiểu số nguyên 8 bit, dùng trong lượng tử hoá mô hình
KD	Knowledge Distillation	Chung cất tri thức
KL	Kullback–Leibler Divergence	Độ lệch Kullback–Leibler giữa hai phân phối
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MCQ	Multiple Choice Question	Câu hỏi trắc nghiệm
mMARCO	multilingual MS MARCO	Bộ dữ liệu MS MARCO đa ngôn ngữ

Viết tắt	Tên tiếng Anh	Tên tiếng Việt
MNRL	Multiple Negatives Ranking Loss	Hàm mất mát xếp hạng đa negative
MRL	Matryoshka Representation Learning	Học biểu diễn lồng nhau (Matryoshka)
MRR	Mean Reciprocal Rank	Nghịch đảo thứ hạng trung bình
NDCG	Normalized Discounted Cumulative Gain	Độ lợi tích lũy chiết khấu chuẩn hóa
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
ONNX	Open Neural Network Exchange	Định dạng trao đổi mô hình mạng nơ-ron, kèm runtime suy luận tối ưu
PDF	Portable Document Format	Định dạng tài liệu di động
Recall@k	Recall at k	Tỉ lệ đáp án đúng xuất hiện trong top- k
RAG	Retrieval-Augmented Generation	Sinh có tăng cường truy hồi
RRF	Reciprocal Rank Fusion	Hợp nhất theo thứ hạng nghịch đảo
SBERT	Sentence-BERT	Mô hình BERT tạo embedding ở mức câu
UNK	Unknown token	Token không có trong từ vựng tokenizer
XLNet	XLNet	Mô hình RoBERTa đa ngôn ngữ

DANH MỤC THUẬT NGỮ

Thuật ngữ	Giải thích
Ablation	Thực nghiệm loại bỏ hoặc thay đổi từng thành phần để đo đóng góp của thành phần đó
Baseline	Mô hình hoặc cấu hình cơ sở dùng làm mốc so sánh
BGE-M3	Mô hình embedding/truy hỏi đa ngôn ngữ thuộc họ BGE, dùng làm backbone hoặc guide model trong đề án
BGE-reranker-v2-m3	Mô hình reranker dạng cross-encoder thuộc họ BGE, dùng làm teacher cho chung cất reranker
Bi-encoder	Kiến trúc mã hóa query và tài liệu độc lập thành hai vector riêng
Candidate pool	Tập ứng viên (chunk) được truy hỏi để đưa vào bước xếp hạng lại
Checkpoint	Bản lưu trọng số mô hình tại một thời điểm huấn luyện
Chunk / Chunking	Đoạn văn bản chia nhỏ từ tài liệu / quá trình chia tài liệu thành các đoạn
Chroma	Cơ sở dữ liệu vector dùng để lưu và truy vấn embedding của các đoạn văn bản
Corpus	Tập văn bản hoặc tập đoạn được dùng để truy hỏi, huấn luyện hoặc đánh giá
Cross-encoder	Kiến trúc đưa đồng thời query và tài liệu qua mô hình để chấm điểm liên quan
Dense retrieval	Truy hỏi dựa trên vector ngữ nghĩa
Embedding	Vector biểu diễn ngữ nghĩa của văn bản
Fine-tune	Tinh chỉnh mô hình tiền huấn luyện trên dữ liệu của miền cụ thể
Gold chunk	Đoạn văn bản chứa thông tin trả lời đúng cho query
Guide model	Mô hình dẫn hướng dùng để lọc false negative trong GIST
Hard negative	Negative khó: gần positive về ngữ nghĩa nhưng không trả lời đúng query
In-batch negative	Negative lấy từ các mẫu khác trong cùng một batch huấn luyện
Intent classifier	Bộ phân loại ý định câu hỏi, dùng để chọn nhánh xử lý tra cứu hoặc tính toán
Latency	Độ trễ xử lý mỗi truy vấn
Logit	Điểm số thô đầu ra của mô hình trước khi chuẩn hóa thành xác suất hoặc phân phối
Lượng tử hoá	Nén mô hình bằng cách biểu diễn trọng số ở độ chính xác số học thấp hơn (ví dụ INT8 thay cho FP32)

Thuật ngữ	Giải thích
Markdown	Định dạng văn bản đánh dấu nhẹ dùng để lưu cấu trúc tài liệu sau OCR/chuyển đổi
ONNX Runtime	Bộ thực thi suy luận tối ưu cho mô hình định dạng ONNX, hỗ trợ tính toán INT8 hiệu quả trên CPU
PhoBERT	Mô hình BERT được tiền huấn luyện cho tiếng Việt, dùng trong backbone phân loại ý định
Pipeline	Chuỗi các bước xử lý nối tiếp của hệ thống
p95	Phân vị thứ 95 của độ trễ: 95% số truy vấn có độ trễ thấp hơn giá trị này
Prompt	Chỉ dẫn đầu vào gửi cho mô hình ngôn ngữ hoặc mô hình giám khảo
Pruned model	Mô hình đã được cắt tỉa một phần tham số hoặc từ vựng để giảm dung lượng
Query	Câu truy vấn của người dùng
Reranker	Mô hình xếp hạng lại các ứng viên đã được truy hồi
Router	Thành phần định tuyến câu hỏi sang nhánh xử lý phù hợp trong pipeline
SetFit	Phương pháp fine-tune sentence transformer bằng học tương phản rồi huấn luyện classifier nhẹ
Seed (hạt giống ngẫu nhiên)	Giá trị khởi tạo bộ sinh số ngẫu nhiên; đổi seed làm đổi thứ tự trộn batch nên kết quả huấn luyện dao động
Sparse retrieval	Truy hồi dựa trên trùng khớp từ khóa (ví dụ BM25)
Teacher / Student	Mô hình thầy (lớn) và trò (nhỏ) trong chung cất tri thức
Tokenizer	Bộ tách văn bản thành token trước khi đưa vào mô hình ngôn ngữ
Triplet	Mẫu huấn luyện gồm query, positive và negative dùng cho học tương phản hoặc reranking
Vocabulary pruning	Cắt tỉa từ vựng của tokenizer để giảm kích thước mô hình
Zero-shot	Đánh giá trên dữ liệu hoặc tài liệu chưa từng xuất hiện khi huấn luyện

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Sự bùng nổ của các mô hình ngôn ngữ lớn (LLM) đã mở ra hướng tiếp cận mới cho bài toán hỏi đáp tự động. Tuy nhiên, LLM thuần túy bộc lộ hạn chế rõ rệt khi cần trả lời câu hỏi về kiến thức chuyên biệt, cập nhật theo thời gian hoặc đòi hỏi tham chiếu tài liệu cụ thể — những tình huống thường xuyên gặp trong môi trường doanh nghiệp. Kiến trúc Retrieval-Augmented Generation (RAG) [1] ra đời nhằm giải quyết vấn đề này bằng cách kết hợp truy xuất thông tin từ kho tài liệu nội bộ với khả năng sinh ngôn ngữ của LLM, giúp câu trả lời bám sát nguồn và có thể kiểm chứng.

Đề án này được thực hiện trên tập dữ liệu của cuộc thi Viettel AI Race 2025, gồm 991 câu hỏi trắc nghiệm tiếng Việt trải rộng trên 200 tài liệu kỹ thuật đa lĩnh vực. Tập tài liệu có cấu trúc phức tạp — bảng biểu, ký hiệu kỹ thuật, thuật ngữ chuyên ngành — và đặt ra thách thức không nhỏ cho các bước chia đoạn (chunking), truy hồi và xếp hạng. Đây là bối cảnh thực tiễn phản ánh đúng những khó khăn khi triển khai RAG trong môi trường sản xuất thực tế, không phải trên các tập đánh giá học thuật lý tưởng hóa.

1.2 Các giải pháp hiện có và hạn chế

Các hệ thống RAG hiện nay thường kết hợp truy hồi lai (BM25 + truy hồi dày đặc + RRF), reranker dạng cross-encoder, và sinh câu trả lời bằng LLM. Đây là kiến trúc được chứng minh hiệu quả về mặt học thuật. Tuy nhiên, trong quá trình xây dựng và đánh giá hệ thống trên tập dữ liệu cuộc thi, chúng tôi quan sát thấy ba vấn đề thực tế mà các công trình hiện có chưa giải quyết triệt để:

- Reranker quá nặng cho triển khai thực tế. BGE reranker [2] (568M tham số) cho chất lượng xếp hạng cao nhưng chi phí suy luận lớn. Trên môi trường không có GPU mạnh — chẳng hạn khi triển khai trên CPU hoặc hạ tầng nội bộ hạn chế — độ trễ của mô hình tăng cao và trở thành điểm nghẽn của toàn bộ luồng xử lý. Các công trình hiện tại thường chỉ báo cáo độ chính xác mà bỏ qua chi phí suy luận, trong khi đây là yếu tố quyết định khả năng triển khai thực tế.
- Định tuyến bằng LLM lớn tốn kém không cần thiết. Luồng Router-Summary sử dụng GPT-4o-mini [3] để phân loại ý định — một tác vụ thực chất chỉ cần phân biệt hai lớp (tra_cuu / tinh_toan). Mỗi lần gọi API thêm khoảng 740ms độ trễ và phát sinh chi phí, trong khi độ chính xác phân loại chỉ đạt 94.62% —

một ngưỡng hoàn toàn có thể đạt được, thậm chí vượt qua, bằng mô hình nhỏ hơn nhiều bậc chạy cục bộ.

- Mô hình embedding đã huấn luyện sẵn chưa tối ưu cho miền. Mô hình Vietnamese Embedding v2 [4], dù là embedding mạnh nhất hiện có cho tiếng Việt, chỉ đạt $\text{Acc}@1 = 65.19\%$ trên tập kiểm thử nội bộ khi đánh giá riêng bước truy hồi. Chất lượng truy hồi là giới hạn trên của toàn luồng xử lý — nếu không lấy đúng đoạn văn bản ngay từ đầu, các bước sau không thể cứu vãn.

1.3 Mục tiêu và định hướng giải pháp

Đề án hướng tới mục tiêu xây dựng và tối ưu hệ thống RAG tiếng Việt theo hai giai đoạn phát triển có căn cứ thực nghiệm.

Giai đoạn 1 xây dựng và đánh giá hai luồng xử lý RAG — Baseline Router và Router-Summary — trên tập 991 câu hỏi, nhằm có bức tranh định lượng rõ ràng về độ chính xác, độ trễ và các dạng lỗi thực tế trước khi quyết định tối ưu.

Giai đoạn 2 dựa trên kết quả giai đoạn 1 đề xuất ba hướng tối ưu có chủ đích, mỗi hướng giải quyết đúng một điểm nghẽn đã xác định:

- Chung cất BGE reranker (568M tham số) sang ba reranker nhỏ hơn: MiniLM 33M, mmarco 117M và pruned mmarco 35.6M. Mục tiêu là giảm độ trễ và dung lượng trong khi duy trì chất lượng ranking.
- Huấn luyện router cục bộ dựa trên Sentence-BERT/SetFit để thay thế GPT-4o-mini, nhằm loại bỏ chi phí API và giảm độ trễ định tuyến.
- Tinh chỉnh mô hình embedding bằng MultipleNegativesRankingLoss kết hợp Matryoshka Representation Learning, nhằm cải thiện chất lượng truy hồi trên miền tài liệu kỹ thuật tiếng Việt đồng thời cho phép triển khai ở số chiều rút gọn.

1.4 Đóng góp đề án

Các đóng góp chính của đề án bao gồm:

- **[C1] Quy trình xây dựng dữ liệu huấn luyện dùng chung cho embedding và reranker.** Quy trình dùng mô hình ngôn ngữ làm giám khảo để xác định positive (gold chunk), lọc khả năng trả lời độc lập, rồi khai thác hard negative từ kết quả truy hồi cho tầng xếp hạng. Cùng một nguồn dữ liệu được tái sử dụng cho cả hai tầng: tầng embedding chỉ dùng các cặp câu hỏi–positive, còn tầng reranker áp dụng tiêu chí lọc và ghép cặp chặt hơn để tạo triplet phục vụ học xếp hạng.
- **[C2] Tinh chỉnh embedding theo miền kết hợp Matryoshka Representa-**

tion Learning. Tinh chỉnh mô hình embedding bằng MultipleNegativesRankingLoss trên các cặp câu hỏi–positive của miền, kết hợp MRL để một mô hình duy nhất cho biểu diễn dùng được ở ba số chiều 256/512/1024. Trên tập kiểm thử in-domain, Acc@1 tăng từ 65.19% lên $73.21 \pm 0.60\%$ (trung bình trên ba hạt giống ngẫu nhiên); trên tập tài liệu chưa từng xuất hiện khi huấn luyện, MRR@10 đạt 0.7982 ± 0.0017 ở 512 chiều. Nhờ MRL, cấu hình 512 chiều giữ được phần lớn chất lượng của 1024 chiều trong khi giảm một nửa dung lượng vector index — đánh đổi phù hợp cho hệ thống chạy trên tài nguyên hạn chế.

- **[C3] Nén reranker bằng chưng cất tri thức kết hợp cắt tỉa từ vựng.** Chưng cất teacher BGE-reranker-v2-m3 (568M tham số) sang các student nhẹ qua quy trình hai giai đoạn (thích nghi miền bằng contrastive learning, rồi chưng cất tín hiệu xếp hạng của teacher), sau đó cắt ma trận embedding của bộ tách từ từ 250.002 xuống 36.324 token. Kết hợp hai kỹ thuật, reranker 35.6M tham số (giảm khoảng 70% so với mmarco 117M) đạt $\text{MRR}@10 = 0.8844$ — giữ 99.8% chất lượng của mmarco 117M (0.8862, vốn đã bằng teacher 568M) và vượt ViRanker 568M (0.8593) dù nhỏ hơn khoảng 16 lần. Sau lượng tử hoá INT8, mô hình chỉ còn 34.8 MB và chạy trên CPU nhanh hơn teacher khoảng 10 lần, loại bỏ hoàn toàn yêu cầu GPU khỏi tầng xếp hạng lại.
- **[C4] Hệ thống RAG tối ưu đầu cuối.** Tích hợp embedding tinh chỉnh 512 chiều, reranker chưng cất–lượng tử hoá và router cục bộ dựa trên Sentence-BERT/SetFit (thay GPT router). Trên 390 câu trắc nghiệm, hệ thống tối ưu giữ nguyên độ chính xác của v1 (97.4%) trong khi phân truy hồi–xếp hạng nhanh hơn khoảng 20 lần và không còn phụ thuộc dịch vụ API cho cả định tuyến lẫn xếp hạng; router cục bộ thậm chí phân loại ý định chính xác hơn GPT router (97.69% so với 94.62%).

1.5 Bố cục đồ án

Nội dung còn lại của đồ án được tổ chức như sau:

- Chương 2 trình bày nền tảng lý thuyết của các kỹ thuật được sử dụng, từ BM25, dense retrieval, cross-encoder reranker, đến chưng cất tri thức (knowledge distillation), MRL, Sentence-BERT/SetFit và PhoBERT.
- Chương 3 trình bày mô hình đề xuất: hệ thống RAG ban đầu (v1), quy trình tiền xử lý tài liệu, kiến trúc luồng xử lý, các điểm nghiên quan sát được và thiết kế các thành phần tối ưu cho v2.
- Chương 4 báo cáo kết quả đánh giá thực nghiệm, bao gồm nghiên cứu loại bỏ (ablation), so sánh với các mô hình nền (baseline), đánh giá từng thành phần

và đánh giá đầu cuối v_1 so với v_2 .

- Chương 5 đưa ra kết luận, thảo luận hạn chế và định hướng phát triển.

CHƯƠNG 2. TỔNG QUAN VÀ CÔNG TRÌNH LIÊN QUAN

2.1 Tổng quan về hệ thống RAG

2.1.1 Hạn chế của mô hình ngôn ngữ lớn thuần túy

Các mô hình ngôn ngữ lớn (Large Language Models – LLM) như GPT, Llama hay Qwen đã đạt kết quả tốt trong nhiều tác vụ xử lý ngôn ngữ tự nhiên. Tuy nhiên, khi áp dụng trực tiếp vào bài toán hỏi đáp trên kho tài liệu kỹ thuật nội bộ, LLM thuần túy vẫn gặp một số hạn chế căn bản.

Thứ nhất là hiện tượng hallucination, tức mô hình tạo ra câu trả lời nghe hợp lý nhưng không được hỗ trợ bởi nguồn tài liệu chính xác. Nguyên nhân là LLM sinh văn bản dựa trên xác suất của token tiếp theo, trong khi không có cơ chế mặc định để kiểm chứng câu trả lời với tài liệu bên ngoài. Với tài liệu kỹ thuật, nơi các câu trả lời thường liên quan tới thông số, điều kiện vận hành, đơn vị đo hoặc quy trình cụ thể, hallucination có thể làm giảm đáng kể độ tin cậy của hệ thống.

Thứ hai là giới hạn về tri thức. Tri thức của LLM bị cố định tại thời điểm huấn luyện và không tự động cập nhật theo tài liệu mới. Nếu tổ chức thay đổi quy trình, bổ sung phiên bản tài liệu mới hoặc cập nhật thông số thiết bị, LLM thuần túy không thể biết các thay đổi này nếu không được tinh chỉnh lại hoặc không có cơ chế truy cập tài liệu ngoài.

Thứ ba là khoảng cách miền dữ liệu (domain gap). Dữ liệu huấn luyện của LLM thường có tính tổng quát, trong khi tài liệu kỹ thuật nội bộ có nhiều thuật ngữ chuyên ngành, bảng biểu, ký hiệu, mã tài liệu và cách diễn đạt đặc thù. Vì vậy, LLM có thể hiểu tốt ngôn ngữ tự nhiên nói chung nhưng vẫn trả lời thiếu chính xác khi câu hỏi yêu cầu thông tin rất cụ thể trong một corpus hẹp.

2.1.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation – RAG [1] là hướng tiếp cận kết hợp truy hỏi thông tin với khả năng sinh ngôn ngữ của LLM. Thay vì yêu cầu LLM trả lời hoàn toàn dựa trên tri thức trong tham số mô hình, RAG truy hỏi các đoạn tài liệu liên quan trước, sau đó đưa các đoạn này vào prompt để LLM sinh câu trả lời dựa trên ngữ cảnh được cung cấp.

Một hệ thống RAG thường gồm hai giai đoạn chính:

- **Giai đoạn offline:** tài liệu được tiền xử lý, chia thành các đoạn nhỏ gọi là đoạn, mã hóa thành vector embedding và lưu trong chỉ mục truy hỏi.
- **Giai đoạn online:** hệ thống nhận truy vấn từ người dùng, truy hỏi các đoạn

liên quan, có thể xếp hạng lại các đoạn này, sau đó đưa top đoạn vào prompt cho LLM sinh câu trả lời.

Kiến trúc tổng quát của RAG có thể được mô tả như sau:

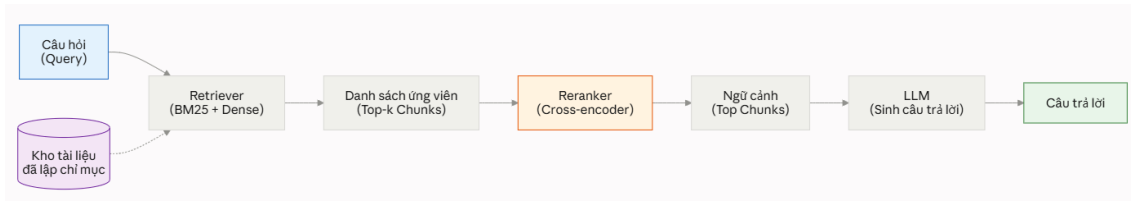


Figure 2.1: Kiến trúc tổng quan của hệ thống RAG

Trong pipeline này, chất lượng câu trả lời cuối phụ thuộc mạnh vào chất lượng của truy hồi và xếp hạng lại. Nếu đoạn chứa thông tin cần thiết không xuất hiện trong tập ứng viên, LLM gần như không có cơ sở để trả lời đúng. Ngược lại, nếu tập ứng viên đã chứa gold đoạn nhưng thứ hạng chưa tốt, reranker có thể cải thiện thứ tự trước khi đưa ngữ cảnh vào LLM. Vì vậy, trong các hệ thống RAG thực tế, truy hồi và xếp hạng lại thường là hai thành phần quan trọng cần tối ưu trước khi tối ưu prompt hoặc LLM.

2.1.3 Đặc thù của RAG cho tài liệu kỹ thuật tiếng Việt

Bài toán hỏi đáp tài liệu kỹ thuật tiếng Việt có một số đặc thù so với RAG trên văn bản tổng quát. Thứ nhất, nhiều câu hỏi yêu cầu truy hồi chính xác một giá trị, một điều kiện hoặc một thông số nằm trong bảng biểu. Thứ hai, tài liệu có thể chứa nhiều ký hiệu, mã thiết bị, tên module hoặc định danh tài liệu, khiến khớp từ khóa chính xác vẫn giữ vai trò quan trọng. Thứ ba, cùng một chủ đề kỹ thuật có thể xuất hiện ở nhiều đoạn khác nhau, làm tăng nguy cơ false negatives khi huấn luyện truy hồi model. Thứ tư, hệ thống cần cân bằng giữa chất lượng trả lời, độ trễ và chi phí triển khai, đặc biệt khi chạy trên CPU hoặc môi trường tài nguyên hạn chế.

Các đặc thù này dẫn đến lựa chọn kiến trúc trong đề án: sử dụng truy hồi lai (hybrid) để kết hợp truy hồi thưa (sparse) và truy hồi dày (dense), dùng cross-encoder reranker để cải thiện thứ hạng, sau đó tối ưu các thành phần nặng bằng tinh chỉnh, chưng cất và nén mô hình.

2.2 Truy hồi thông tin

2.2.1 BM25 và truy hồi thưa (sparse retrieval)

BM25 (Best Matching 25) là một mô hình truy hồi dựa trên thống kê từ khóa, được sử dụng rộng rãi như một baseline mạnh trong truy hồi thông tin [5]. BM25 thuộc nhóm truy hồi thưa (sparse) vì tài liệu và truy vấn được biểu diễn thông qua các term rời rạc thay vì vector dày đặc.

Với truy vấn q gồm các term q_i và tài liệu d , điểm BM25 được tính như sau:

$$\text{BM25}(d, q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \frac{f(q_i, d)(k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \quad (2.1)$$

Trong đó, $f(q_i, d)$ là tần suất xuất hiện của term q_i trong tài liệu d , $|d|$ là độ dài tài liệu, avgdl là độ dài trung bình của các tài liệu trong corpus, k_1 điều khiển mức bão hòa của term frequency và b điều khiển mức chuẩn hóa theo độ dài tài liệu.

Thành phần IDF được dùng để tăng trọng số cho các term hiếm và giảm trọng số cho các term phổ biến:

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}, \quad (2.2)$$

trong đó N là tổng số tài liệu và $n(q_i)$ là số tài liệu chứa term q_i .

Ưu điểm của BM25 là đơn giản, nhanh, không cần GPU và hoạt động tốt với các truy vấn có chứa từ khóa chính xác, ví dụ mã tài liệu, tên thiết bị, ký hiệu kỹ thuật hoặc giá trị cụ thể. Tuy nhiên, BM25 không nắm bắt đầy đủ quan hệ ngữ nghĩa. Nếu truy vấn và tài liệu diễn đạt cùng một ý bằng các từ khác nhau, BM25 có thể xếp hạng thấp tài liệu liên quan.

2.2.2 Truy hồi dày (dense retrieval) và bi-encoder

Truy hồi dày (dense) sử dụng các mô hình neural embedding để mã hóa truy vấn và document thành các vector dày đặc trong cùng một không gian ngữ nghĩa. Kiến trúc phổ biến nhất là bi-encoder, trong đó truy vấn và document được mã hóa độc lập:

$$\mathbf{e}_q = E_q(q), \quad \mathbf{e}_d = E_d(d). \quad (2.3)$$

Độ liên quan giữa truy vấn và document thường được tính bằng cosine similarity:

$$s(q, d) = \cos(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \quad (2.4)$$

Trong nhiều hệ thống, E_q và E_d dùng chung trọng số. Khi các vector được chuẩn hóa L2, cosine similarity tương đương với tích vô hướng giữa hai vector. Ưu điểm lớn của bi-encoder là document embeddings có thể được tính trước ở giai đoạn offline. Khi suy luận, hệ thống chỉ cần mã hóa truy vấn một lần rồi tìm các vector

gần nhất trong vector database hoặc approximate nearest neighbor index.

So với BM25, truy hồi dày (dense) có khả năng bắt quan hệ ngữ nghĩa tốt hơn. Ví dụ, truy vấn “thiết bị chịu được nhiệt độ bao nhiêu” có thể gần với đoạn “nhiệt độ vận hành tối đa là 85°C” dù hai câu không trùng khớp hoàn toàn về mặt từ vựng. Hạn chế của truy hồi dày là chất lượng phụ thuộc mạnh vào mô hình embedding. Nếu model chưa được tối ưu cho miền tài liệu kỹ thuật, các đoạn liên quan có thể không được xếp đủ cao trong top-k.

2.2.3 Truy hồi lai và Reciprocal Rank Fusion

Truy hồi thưa (sparse) và truy hồi dày (dense) có tính bổ sung. BM25 mạnh với khớp chính xác, trong khi truy hồi dày mạnh với khớp ngữ nghĩa. Truy hồi lai (hybrid) kết hợp cả hai hướng để tăng khả năng đưa gold đoạn vào tập ứng viên.

Một khó khăn khi kết hợp là điểm số của hai retriever nằm trên các thang đo khác nhau. Điểm BM25 không bị chặn trên, còn cosine similarity thường nằm trong khoảng $[-1, 1]$. Việc cộng trực tiếp hai loại điểm có thể khiến kết quả bị chi phối bởi retriever có scale lớn hơn.

Reciprocal Rank Fusion – RRF [6] giải quyết vấn đề này bằng cách kết hợp dựa trên thứ hạng thay vì điểm số tuyệt đối:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}. \quad (2.5)$$

Trong đó, R là tập các retriever, $\text{rank}_r(d)$ là thứ hạng của tài liệu d theo retriever r , và k là hằng số làm mượt, thường được đặt bằng 60. Một tài liệu xuất hiện ở thứ hạng cao trong nhiều retriever sẽ nhận điểm RRF cao hơn. Cách này giúp truy hồi lai ổn định hơn khi các retriever có scale điểm khác nhau.

Table 2.1: So sánh BM25, truy hồi dày (dense) và truy hồi lai (hybrid)

Đặc điểm	BM25	Truy hồi dày	Truy hồi lai
Biểu diễn	Từ khóa sparse	Vector embedding	Kết hợp nhiều retriever
Điểm mạnh	Khớp chính xác, mã tài liệu, thuật ngữ kỹ thuật	Ngữ nghĩa, diễn đạt tương đương	Tăng recall tổng thể
Điểm yếu	Kém khi khác cách diễn đạt	Phụ thuộc mô hình embedding	Cần cơ chế fusion
Vai trò trong RAG	First-stage truy hồi	First-stage truy hồi	Candidate generation

2.3 Xếp hạng lại bằng cross-encoder

2.3.1 Bi-encoder và cross-encoder

Trong truy hồi pipeline, bi-encoder phù hợp cho truy hồi nhanh trên corpus lớn, nhưng do truy vấn và document được mã hóa độc lập, mô hình có thể bỏ sót các tương tác chi tiết giữa token của truy vấn và token của document. Cross-encoder khắc phục hạn chế này bằng cách nhận đồng thời truy vấn và document làm đầu vào.

Với một cặp (q, d) , input của cross-encoder thường có dạng:

$$[\text{CLS}] \ q \ [\text{SEP}] \ d \ [\text{SEP}]. \quad (2.6)$$

Transformer encoder xử lý chuỗi ghép này và cho phép cross-attention đầy đủ giữa truy vấn và document. Điểm liên quan được sinh từ hidden state của token $[\text{CLS}]$ hoặc một pooling layer:

$$s(q, d) = \text{Linear}(\mathbf{h}_{[\text{CLS}]}) . \quad (2.7)$$

Nhờ cross-attention, cross-encoder thường cho chất lượng ranking cao hơn bi-encoder. Ví dụ, token “tối đa” trong truy vấn có thể attend trực tiếp tới cụm “85°C” trong document, giúp mô hình đánh giá liên quan chính xác hơn. Hạn chế là cross-encoder không thể tính trước document embeddings. Với mỗi truy vấn mới, mô hình phải chạy lại trên từng cặp truy vấn–ứng viên. Do đó, độ trễ của cross-encoder tăng gần tuyến tính theo số ứng viên cần xếp hạng lại.

Trong pipeline RAG thực tế, kiến trúc thường dùng là hai tầng: bi-encoder hoặc truy hồi lai tạo tập ứng viên nhanh, sau đó cross-encoder xếp hạng lại một số lượng nhỏ ứng viên như top-20 hoặc top-50.

Table 2.2: So sánh bi-encoder và cross-encoder

Đặc điểm	Bi-encoder	Cross-encoder
Cách mã hóa	Query và document độc lập	Query và document cùng một input
Tính trước document	Có	Không
Tốc độ trên corpus lớn	Nhanh	Chậm nếu tập ứng viên lớn
Khả năng tương tác token	Hạn chế	Đầy đủ qua cross-attention
Vai trò phổ biến	First-stage truy hồi	Second-stage xếp hạng lại

2.3.2 BGE-reranker-v2-m3

Trong đồ án, teacher reranker là BGE-reranker-v2-m3 [2]. Đây là mô hình cross-encoder thuộc họ BGE reranker, nhận cặp truy vấn–ứng viên và trả về một liên quan score dạng logit. Logit này là điểm số thô, không phải xác suất liên quan đã chuẩn hóa trong khoảng $[0, 1]$.

Với một truy vấn q và danh sách n ứng viên đoạn $\{c_1, \dots, c_n\}$, teacher tạo vector điểm:

$$\mathbf{s}^T = [s_1^T, s_2^T, \dots, s_n^T], \quad s_i^T = f_T(q, c_i). \quad (2.8)$$

Các ứng viên được sắp xếp theo s_i^T giảm dần để tạo thứ hạng teacher. Trong quá trình chưng cất, student cần học lại tín hiệu xếp hạng này bằng một mô hình nhỏ hơn. Việc phân biệt rõ BGE-reranker-v2-m3 với BGE-M3 embedding là cần thiết: BGE-M3 thường được dùng để chỉ mô hình embedding/truy hỏi đa ngôn ngữ, còn BGE-reranker-v2-m3 là cross-encoder reranker.

2.3.3 Chưng cất tri thức cho reranker

Knowledge Distillation – KD [7] là kỹ thuật chuyển tri thức từ teacher model lớn sang student model nhỏ hơn. Trong bài toán xếp hạng lại, teacher thường là một cross-encoder mạnh, còn student là cross-encoder nhẹ hơn. Thay vì chỉ học hard label relevant/irrelevant, student học từ điểm số hoặc thứ hạng do teacher tạo ra.

Cơ chế KD tổng quát có thể được viết dưới dạng:

$$\mathcal{L}_{KD} = D(g(\mathbf{s}^T), g(\mathbf{s}^S)), \quad (2.9)$$

trong đó $\mathbf{s}^T$ và $\mathbf{s}^S$ lần lượt là vector logits của teacher và student trên cùng ứng viên list, $g(\cdot)$ là phép chuẩn hóa hoặc biến đổi tín hiệu, còn $D(\cdot, \cdot)$ là hàm đo khoảng cách giữa hai tín hiệu.

Tùy theo cách sử dụng teacher signal, các loss cho reranker chưng cất có thể chia thành ba nhóm chính:

- **Listwise**: học phân phối điểm trên toàn bộ ứng viên list.
- **Pairwise**: học quan hệ ưu tiên giữa từng cặp ứng viên.
- **Rank-based**: học trực tiếp từ thứ hạng, giảm phụ thuộc vào scale logit của teacher.

Lựa chọn negative cho dữ liệu chung cất. Chất lượng của student phụ thuộc nhiều vào tập ứng viên mà teacher được yêu cầu chấm điểm. Nếu negative quá dễ — chẳng hạn lấy ngẫu nhiên từ corpus — teacher sẽ chấm điểm gần như bằng không cho tất cả, và student không học được ranh giới phân biệt nào có ý nghĩa. Ngược lại, mined hard negatives là các đoạn thật nằm gần gold trong kết quả truy hỏi nhưng không trả lời đúng truy vấn; chúng phản ánh đúng những trường hợp mà bộ truy hỏi thực sự nhầm lẫn, nên là ứng viên có giá trị nhất để teacher phân biệt và student học theo.

Cách khai thác phổ biến là dùng chính bộ truy hỏi để lấy top- k ứng viên cho mỗi truy vấn, loại bỏ các đoạn thuộc tập gold, và giữ lại phần còn lại làm negative. Rủi ro chính của cách này là false negatives: một đoạn không được gán nhãn gold vẫn có thể trả lời được truy vấn, và khi bị dùng làm negative, nó dạy mô hình những tín hiệu sai. Trong bối cảnh chung cất, rủi ro này được giảm nhẹ một cách tự nhiên: thay vì gán nhãn cứng “đúng/sai”, teacher cross-encoder chấm cho mỗi ứng viên một điểm liên tục, nên một false negative sẽ nhận điểm cao và student học được rằng đoạn đó vẫn liên quan thay vì bị ép đẩy ra xa. Đây là một ưu điểm của chung cất so với huấn luyện contrastive trên nhãn nhị phân.

2.3.4 Các hàm mất mát cho chung cất reranker

Khi chung cất reranker, lựa chọn hàm mất mát quyết định cách student tiếp nhận tín hiệu của teacher. Các phương pháp phổ biến khác nhau ở chỗ chúng khai thác loại tín hiệu nào từ teacher: điểm thô (logit), chênh lệch điểm giữa các cặp, hay thứ hạng của các ứng viên.

- **Listwise KL Divergence** chuyển logit của teacher và student thành hai phân phối mềm trên toàn bộ danh sách ứng viên (bằng softmax với nhiệt độ τ) rồi buộc phân phối của student tiến gần phân phối của teacher. Cách này học ưu tiên tương đối trên cả danh sách nhưng nhạy với nhiễu và nhiệt độ.
- **Margin-MSE** [8] cho student khớp chênh lệch điểm giữa các cặp ứng viên thay vì điểm tuyệt đối, phù hợp khi thứ tự tương đối quan trọng hơn giá trị điểm; tuy nhiên dễ bị chi phối bởi các cặp dễ có margin lớn.
- **RankNet** [9] mô hình hóa xác suất một ứng viên được ưu tiên hơn ứng viên khác qua hàm sigmoid của hiệu điểm, tạo tín hiệu theo từng cặp phong phú nhưng vẫn phụ thuộc vào thang điểm của teacher.
- **ADR-MSE**, kế thừa từ Rank-DistiLLM [10], chỉ dùng thứ hạng của ứng viên (ánh xạ về một khoảng cố định) làm mục tiêu, nên không buộc student khớp thang điểm tuyệt đối của teacher — giảm được ảnh hưởng của chênh lệch

thang điểm giữa hai mô hình.

Công thức chi tiết của từng hàm mất mát và lựa chọn sử dụng trong đồ án được trình bày ở mục 3.8.4. Bảng 2.3 tóm tắt khác biệt cốt lõi giữa bốn phương pháp.

Table 2.3: Tóm tắt các nhóm hàm mất mát cho chung cất reranker

Hàm mất mát	Tín hiệu teacher	Đặc điểm chính
Listwise KL	Phân phối softmax trên danh sách ứng viên	Học toàn bộ danh sách, nhạy với câu hỏi nhiều và nhiệt độ
Margin-MSE	Chênh lệch logit giữa hai ứng viên	Giữ thông tin khoảng cách tương đối, có thể chịu ảnh hưởng bởi margin lớn
RankNet	Xác suất ưu tiên theo từng cặp	Tín hiệu theo cặp phong phú, số cặp tăng theo $O(n^2)$
ADR-MSE	Vị trí xếp hạng của teacher	Ít phụ thuộc vào thang điểm tuyệt đối của teacher

2.3.5 Nén reranker và cất tia từ vựng

Reranker cross-encoder thường có chất lượng cao nhưng chi phí suy luận lớn. Để triển khai thực tế, đặc biệt trên CPU, cần giảm kích thước và độ trễ của reranker. Hai hướng phổ biến là chung cất tri thức và cất tia.

Chung cất tri thức giảm chi phí bằng cách huấn luyện student nhỏ hơn bắt chước teacher lớn. Student có thể có ít layer hơn, hidden size nhỏ hơn hoặc kiến trúc nhẹ hơn. Nếu dữ liệu chung cất đúng miền, student có thể học được hành vi ranking quan trọng của teacher mà không cần giữ toàn bộ capacity.

Cất tia từ vựng (vocabulary pruning) là một dạng nén khác, tập trung vào embedding matrix của mô hình ngôn ngữ. Với các model đa ngôn ngữ, tokenizer thường chứa số lượng token rất lớn để hỗ trợ nhiều ngôn ngữ. Khi triển khai trên một miền hẹp, chẳng hạn tài liệu kỹ thuật tiếng Việt, chỉ một phần nhỏ từ vựng được sử dụng thường xuyên. Cất bỏ các token không cần thiết giúp giảm số tham số ở embedding matrix và giảm dung lượng lưu trữ. Tuy nhiên, cất tia từ vựng thường không làm giảm đáng kể độ trễ của Transformer encoder nếu số layer và hidden size được giữ nguyên.

Lượng tử hoá (quantization). Hướng nén thứ ba tác động vào độ chính xác số học của trọng số thay vì vào kiến trúc hay từ vựng. Thay vì lưu trọng số ở dạng dấu phẩy động 32 bit (FP32), lượng tử hoá biểu diễn chúng bằng số nguyên 8 bit (INT8), giảm dung lượng khoảng bốn lần. Có hai biến thể chính. Lượng tử hoá động (dynamic quantization) chỉ nén trọng số, còn activation vẫn được tính ở FP32 và phải chuyển đổi qua lại ở mỗi tầng; cách này không cần dữ liệu hiệu chuẩn nên

rất dễ triển khai, nhưng mức tăng tốc thực tế thấp hơn giới hạn lý thuyết $4\times$ vì chi phí chuyển đổi. Lượng tử hoá tĩnh (static quantization) nén cả activation, đạt tăng tốc cao hơn nhưng đòi hỏi một tập dữ liệu hiệu chuẩn đại diện để ước lượng dải giá trị của activation.

Lượng tử hoá thường được triển khai cùng một runtime suy luận tối ưu như ONNX Runtime, vốn thực hiện thêm các tối ưu ở mức đồ thị tính toán (hợp nhất toán tử, loại bỏ nút thừa) và cung cấp các nhân tính toán INT8 hiệu quả trên CPU. Ngoài lợi ích về dung lượng và thông lượng, một hiệu ứng ít được nhắc tới nhưng quan trọng khi triển khai là độ ổn định của độ trễ: trọng số INT8 nhỏ hơn bốn lần nên dễ nằm gọn trong bộ nhớ đệm của CPU, giảm số lần truy cập bộ nhớ chính và do đó thu hẹp đuôi trễ (p95, p99) — chỉ số thường quyết định trải nghiệm người dùng nhiều hơn giá trị trung vị.

Ba hướng nén trên tác động vào ba thành phần khác nhau nên có thể kết hợp: chùng cất giảm số tầng và kích thước ẩn, cắt tia từ vệt giảm ma trận embedding, lượng tử hoá giảm độ chính xác số học của toàn bộ trọng số còn lại. Trong đồ án, cả ba được áp dụng nối tiếp: student học ranking từ teacher, sau đó giảm từ vệt để giảm dung lượng mô hình mà vẫn giữ năng lực xử lý tiếng Việt trong miền đích, và cuối cùng lượng tử hoá INT8 để triển khai trên CPU.

2.4 Mô hình nhúng văn bản và tinh chỉnh embedding

2.4.1 Từ Transformer đến Sentence-BERT

Phần lớn các mô hình embedding và reranker hiện đại đều dựa trên kiến trúc Transformer [11], với cơ chế self-attention cho phép mỗi token tham chiếu đến toàn bộ các token khác trong chuỗi để nắm bắt quan hệ ngữ cảnh. BERT [12] kế thừa Transformer encoder hai chiều và được tiền huấn luyện trên khối dữ liệu lớn, tạo ra biểu diễn ngữ cảnh mạnh cho nhiều tác vụ xử lý ngôn ngữ. PhoBERT [13] là biến thể của BERT cho tiếng Việt, huấn luyện trên ngữ liệu tiếng Việt và yêu cầu đầu vào đã tách từ — là điểm cần lưu ý khi xử lý dữ liệu trong đồ án.

Tuy nhiên, BERT gốc không tạo ra biểu diễn câu phù hợp để so khớp trực tiếp bằng độ tương đồng. Sentence-BERT [14] khắc phục hạn chế này bằng cách tinh chỉnh BERT để sinh ra vector biểu diễn cho cả câu, cho phép đo mức tương đồng giữa hai câu bằng cosine similarity một cách hiệu quả. Đây chính là nền tảng cho hai thành phần của đồ án: mô hình embedding dùng để truy hồi (mục 2.4.2) và bộ phân loại ý định dựa trên embedding câu (mục 2.6.2).

2.4.2 Mô hình embedding cho truy hồi

Trong RAG, mô hình embedding có nhiệm vụ ánh xạ truy vấn và đoạn vào cùng một không gian vector sao cho các cặp liên quan nằm gần nhau. Với corpus tiếng Việt, mô hình embedding cần xử lý tốt cả ngôn ngữ tự nhiên, thuật ngữ kỹ thuật, ký hiệu, bảng biểu và cách diễn đạt đặc thù.

Các mô hình như BGE-M3 [15] và các biến thể đã tinh chỉnh cho tiếng Việt hỗ trợ truy hồi dày (dense) đa ngôn ngữ. Trong đồ án, AITeamVN/Vietnamese_Embedding_v2 được sử dụng làm base mô hình embedding. Mô hình này dựa trên BGE-M3 và tạo vector 1024 chiều. Các chương sau trình bày cách tinh chỉnh mô hình này để phù hợp hơn với miền tài liệu kỹ thuật, đồng thời hỗ trợ triển khai embedding 512 chiều thông qua Matryoshka Representation Learning.

2.4.3 MultipleNegativesRankingLoss

MultipleNegativesRankingLoss – MNRL [16] là một loss phổ biến cho contrastive learning trong sentence embedding. Với batch gồm N cặp truy vấn–positive $\{(q_i, p_i)\}_{i=1}^N$, positive của các truy vấn khác được xem như in-batch negatives đối với truy vấn q_i .

Loss được tính như sau:

$$\mathcal{L}_{\text{MNRL}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)}. \quad (2.10)$$

Trong đó, τ là temperature. Nếu dùng scale γ , hai cách viết tương đương qua quan hệ $\gamma = 1/\tau$. MNRL có ưu điểm là tận dụng được nhiều negatives trong một batch mà không cần gán nhãn negative tường minh. Tuy nhiên, giả định “mọi positive khác trong batch đều là negative” có thể không đúng trong corpus kỹ thuật, nơi nhiều đoạn khác nhau cùng liên quan tới một chủ đề.

2.4.4 False negatives và GISTEmbedLoss

False negative xảy ra khi một mẫu bị xem là negative trong quá trình huấn luyện nhưng thực tế vẫn liên quan tới truy vấn. Trong MNRL, false negatives có thể xuất hiện vì positive của truy vấn khác trong cùng batch được dùng làm negative. Với tài liệu kỹ thuật, hiện tượng này khá phổ biến do nhiều đoạn cùng mô tả một thiết bị, một điều kiện vận hành hoặc các phần khác nhau của cùng một bảng thông số.

GISTEmbedLoss – Guided In-sample Selection of Training Negatives [17] giảm ảnh hưởng của false negatives bằng cách sử dụng một guide model mạnh để lọc các in-batch negatives đáng ngờ. Với truy vấn q_i , positive p_i và một ứng viên p_j trong

batch, guide model tính độ tương đồng $\text{sim}_g(q_i, p_j)$. Candidate p_j được giữ lại trong mẫu số của loss nếu nó không bị guide đánh giá là tương đồng với truy vấn ngang bằng hoặc hơn positive tương ứng.

Một cách biểu diễn mask là:

$$m_{ij} = \begin{cases} 1, & \text{nếu } j = i, \\ 1, & \text{nếu } \text{sim}_g(q_i, p_j) < \text{sim}_g(q_i, p_i) - \delta, \\ 0, & \text{ngược lại.} \end{cases} \quad (2.11)$$

Trong đó, $m_{ij} = 1$ nghĩa là ứng viên được giữ lại trong loss, còn $m_{ij} = 0$ nghĩa là ứng viên bị loại khỏi tập negative. Khi $\delta = 0$, các ứng viên có độ tương đồng với truy vấn lớn hơn hoặc bằng positive sẽ bị loại khỏi negative set.

Loss có dạng:

$$\mathcal{L}_{\text{GIST}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j:m_{ij}=1} \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)}. \quad (2.12)$$

So với MNRL, điểm khác biệt chính là mẫu số chỉ chứa positive và các negatives không bị guide loại bỏ. Nhờ đó, mô hình giảm nguy cơ bị phạt khi xếp hạng cao một đoạn thực chất vẫn liên quan tới truy vấn.

2.4.5 Matryoshka Representation Learning

Matryoshka Representation Learning – MRL [18] huấn luyện mô hình embedding sao cho các prefix của vector embedding cũng là các biểu diễn hữu ích. Ý tưởng này lấy cảm hứng từ búp bê Matryoshka: vector đầy đủ chứa bên trong các vector nhỏ hơn nhưng vẫn có khả năng sử dụng độc lập.

Giả sử embedding đầy đủ có chiều D , MRL chọn một tập kích thước:

$$\mathcal{M} = \{m_1, m_2, \dots, m_K\}, \quad m_K = D. \quad (2.13)$$

Với embedding $\mathbf{e}$, prefix m chiều được ký hiệu:

$$f_m(\mathbf{e}) = \mathbf{e}[:m]. \quad (2.14)$$

Loss MRL được tính bằng tổng có trọng số của loss truy hồi trên nhiều kích thước:

$$\mathcal{L}_{\text{MRL}} = \sum_{m \in \mathcal{M}} w_m \mathcal{L}(f_m(\mathbf{e}_q), f_m(\mathbf{e}_p)). \quad (2.15)$$

Nếu $\mathcal{L}$ là MNRL hoặc GISTEmbedLoss, mô hình sẽ đồng thời tối ưu khả năng truy hồi ở nhiều kích thước embedding. Lợi ích thực tiễn là cùng một checkpoint có thể dùng cho nhiều cấu hình triển khai. Khi cần giảm dung lượng vector index hoặc tăng tốc tìm kiếm, hệ thống có thể dùng embedding prefix ngắn hơn mà không cần huấn luyện lại model từ đầu.

2.5 Mô hình ngôn ngữ lớn và thiết kế prompt trong RAG

2.5.1 Vai trò của LLM trong sinh câu trả lời

Trong hệ thống RAG, LLM thường đảm nhận vai trò sinh câu trả lời cuối cùng từ ngữ cảnh đã được truy hồi. Prompt đầu vào thường gồm câu hỏi, các đoạn liên quan và yêu cầu mô hình chỉ trả lời dựa trên thông tin được cung cấp. Cấu trúc prompt có ảnh hưởng lớn tới độ chính xác, đặc biệt trong bài toán trắc nghiệm hoặc bài toán yêu cầu trả lời ngắn.

Nếu bộ truy hồi cung cấp ngữ cảnh đúng, LLM có thể tổng hợp thông tin và sinh câu trả lời tự nhiên. Tuy nhiên, nếu ngữ cảnh thiếu hoặc chứa nhiều đoạn nhiễu, LLM có thể chọn sai thông tin. Vì vậy, trong RAG, việc tối ưu giai đoạn sinh câu trả lời cần đi kèm với tối ưu truy hồi và tái xếp hạng.

2.5.2 Kỹ thuật prompt Chain-of-Thought

Chain-of-Thought – CoT prompting [19] là kỹ thuật yêu cầu LLM thực hiện các bước suy luận trung gian trước khi đưa ra đáp án cuối. CoT đặc biệt hữu ích với các câu hỏi cần tính toán, so sánh nhiều dữ kiện hoặc suy luận qua nhiều bước.

Trong bài toán hỏi đáp tài liệu kỹ thuật, có thể phân biệt hai nhóm câu hỏi:

- tra_cuu: câu hỏi yêu cầu lấy thông tin có sẵn trong tài liệu.
- tinh_toan: câu hỏi yêu cầu tính toán hoặc suy luận để tạo ra kết quả mới.

Với câu hỏi tra_cuu, prompt trực tiếp thường đủ và có độ trễ thấp hơn. Với câu hỏi tinh_toan, CoT giúp LLM trích xuất dữ kiện, lập công thức và tính toán theo từng bước. Điều này tạo động lực cho thành phần định tuyến truy vấn: hệ thống cần xác định ý định của câu hỏi trước khi chọn prompt phù hợp.

2.6 Định tuyến truy vấn và phân loại ý định

2.6.1 Định tuyến truy vấn trong pipeline RAG

Định tuyến truy vấn là bước phân tích câu hỏi đầu vào để chọn chiến lược xử lý phù hợp. Trong một pipeline RAG đơn giản, mọi câu hỏi có thể đi qua cùng một

luồng xử lý. Tuy nhiên, trong thực tế, các câu hỏi khác nhau có thể cần prompt, số lượng ngữ cảnh, công cụ hoặc chiến lược suy luận khác nhau.

Với bài toán của đồ án, định tuyến tập trung vào hai ý định chính: tra_cuu, tinh_toan. Câu hỏi tra_cuu cần tìm đúng thông tin có sẵn trong tài liệu. Câu hỏi tinh_toan cần lấy dữ kiện từ tài liệu rồi thực hiện phép tính hoặc suy luận. Nếu route sai, hệ thống có thể dùng prompt không phù hợp: câu hỏi tính toán bị trả lời trực tiếp, hoặc câu hỏi tra cứu đơn giản lại bị xử lý qua pipeline reasoning dài không cần thiết.

2.6.2 PhoBERT, SBERT và SetFit

PhoBERT là mô hình BERT cho tiếng Việt, được huấn luyện trên corpus tiếng Việt và thường được sử dụng cho các tác vụ phân loại văn bản. Với phân loại truyền thống, một đầu phân loại được đặt trên hidden state của token [CLS]:

$$\hat{y} = \text{softmax}(\mathbf{W}\mathbf{h}_{[\text{CLS}]} + \mathbf{b}). \quad (2.16)$$

Cách này hiệu quả khi có đủ dữ liệu huấn luyện cân bằng. Tuy nhiên, với bài toán có ít mẫu ở lớp thiểu số, tinh chỉnh trực tiếp bằng cross-entropy có thể kém ổn định.

SetFit [20] là một phương pháp tinh chỉnh sentence transformer trong điều kiện ít dữ liệu. Thay vì chỉ huấn luyện đầu phân loại, SetFit trước hết tinh chỉnh encoder bằng contrastive learning trên các cặp câu. Sau đó, một classifier nhẹ như Logistic Regression được huấn luyện trên embedding câu.

Với một cặp câu (x_a, x_b) và nhãn tương đồng $y_{ab} \in \{0, 1\}$, CosineSimilarityLoss có thể được viết dưới dạng:

$$\mathcal{L}_{\text{cos}} = (\cos(\mathbf{e}_a, \mathbf{e}_b) - y_{ab})^2. \quad (2.17)$$

Các cặp cùng ý định được gán target 1, còn các cặp khác ý định được gán target 0. Sau tinh chỉnh, embedding của các câu cùng ý định có xu hướng gần nhau hơn, giúp classifier tuyến tính học ranh giới tốt hơn.

2.6.3 Mất cân bằng lớp trong phân loại ý định

Trong phân loại ý định, mất cân bằng lớp là vấn đề quan trọng nếu một lớp xuất hiện ít hơn nhiều so với lớp còn lại. Nếu chỉ tối ưu độ chính xác, mô hình có thể ưu tiên lớp đa số và bỏ qua lớp thiểu số. Với bài toán RAG, lỗi ở lớp thiểu số vẫn có thể gây ảnh hưởng lớn vì câu hỏi tinh_toan thường cần pipeline xử lý khác.

Một số chiến lược thường dùng để giảm ảnh hưởng của mất cân bằng lớp gồm:

- dùng chia phân tầng (stratified split) hoặc stratified cross-validation để giữ tỉ lệ lớp trong các fold;
- dùng balanced sampling hoặc oversampling khi tạo cặp huấn luyện;
- sử dụng macro-F1 thay vì chỉ độ chính xác để đánh giá công bằng hơn giữa các lớp;
- phân tích ma trận nhầm lẫn để xem mô hình nhầm theo hướng nào.

Trong bối cảnh SetFit, việc lấy mẫu cặp cân bằng giúp lớp thiểu số xuất hiện đủ trong cả cặp dương và cặp âm, từ đó cải thiện tín hiệu contrastive cho ý định hiếm.

2.7 Các độ đo đánh giá

2.7.1 Độ đo cho truy hồi và xếp hạng lại

Các hệ thống truy hồi và xếp hạng lại thường được đánh giá bằng các metric dựa trên thứ hạng. Gọi r_q là vị trí của gold đoạn đầu tiên trong danh sách kết quả của truy vấn q .

Hit@k.

Hit@k kiểm tra liệu có ít nhất một gold đoạn nằm trong top- k hay không:

$$\text{Hit@k}(q) = \begin{cases} 1, & r_q \leq k, \\ 0, & \text{ngược lại.} \end{cases} \quad (2.18)$$

Metric này đặc biệt quan trọng với RAG vì nếu gold đoạn không xuất hiện trong top- k ngữ cảnh, LLM khó có thể trả lời đúng.

MRR@k.

Mean Reciprocal Rank đo vị trí của gold đoạn đầu tiên:

$$\text{MRR@k} = \frac{1}{|Q|} \sum_{q \in Q} \begin{cases} \frac{1}{r_q}, & r_q \leq k, \\ 0, & r_q > k. \end{cases} \quad (2.19)$$

MRR nhạy với vị trí top đầu: đưa gold từ rank 5 lên rank 1 làm tăng điểm nhiều hơn đưa gold từ rank 20 lên rank 16. Vì vậy, MRR phù hợp để đánh giá reranker.

NDCG@k.

Normalized Discounted Cumulative Gain [21] đo chất lượng toàn bộ thứ tự xếp hạng:

$$\text{DCG}@k = \sum_{i=1}^k \frac{2^{\text{rel}_i} - 1}{\log_2(i + 1)}, \quad (2.20)$$

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k}, \quad (2.21)$$

trong đó rel_i là mức liên quan tại vị trí i , còn IDCG là DCG lý tưởng. NDCG phù hợp khi có nhiều mức liên quan như gold, partial và irrelevant.

2.7.2 Độ đo cho phân loại ý định và pipeline

Với phân loại ý định, độ chính xác đo tỉ lệ dự đoán đúng trên toàn bộ mẫu. Tuy nhiên, khi dữ liệu mất cân bằng lớp, độ chính xác dễ bị lớp đa số chi phối nên không phản ánh đúng chất lượng. Trong trường hợp này, macro-F1 — trung bình điểm F1 (trung bình điều hòa của precision và recall) trên các lớp — là độ đo phù hợp hơn, vì mỗi lớp đóng góp ngang nhau bất kể số lượng mẫu. Đây là lý do đề án dùng macro-F1 làm độ đo chính cho bộ phân loại ý định, nơi hai lớp tra_cuu và tinh_toan chênh lệch nhiều về số lượng.

Với toàn bộ pipeline RAG, ngoài độ chính xác của câu trả lời cuối, độ trễ và dung lượng triển khai cũng là yếu tố quan trọng: một cấu hình có độ chính xác cao nhưng độ trễ quá lớn có thể không phù hợp với sử dụng thực tế. Vì vậy, đề án đánh giá hệ thống theo hướng đa mục tiêu, cân nhắc đồng thời chất lượng truy hồi, chất lượng xếp hạng lại, độ chính xác cuối, độ trễ và dung lượng triển khai.

2.8 Khoảng trống nghiên cứu

Từ cơ sở lý thuyết và đặc thù của bài toán, đề án xác định một số khoảng trống làm cơ sở cho các chương tiếp theo.

Gap 1 – Reranker nhẹ cho tài liệu kỹ thuật tiếng Việt

Cross-encoder reranker mạnh có thể cải thiện chất lượng ranking nhưng thường có kích thước lớn và độ trễ cao. Trong bối cảnh tiếng Việt, đặc biệt với tài liệu kỹ thuật, vẫn cần khảo sát cách nén reranker bằng chưng cất và cắt tỉa từ vựng để giữ chất lượng gần teacher trong khi giảm chi phí triển khai.

Gap 2 – Tinh chỉnh embedding có kiểm soát false negatives

Các mô hình embedding đa ngôn ngữ hoặc tiếng Việt tổng quát chưa chắc tối ưu cho corpus kỹ thuật hẹp. Khi tinh chỉnh bằng contrastive learning, false negatives dễ xuất hiện vì nhiều đoạn cùng liên quan tới một truy vấn hoặc cùng chủ đề — với tài liệu kỹ thuật, hiện tượng này càng phổ biến do nhiều đoạn cùng mô tả một thiết bị hoặc các phần khác nhau của cùng một quy trình. Do đó, cần lựa chọn hàm

mất mát và cách tổ chức dữ liệu huấn luyện sao cho hạn chế được nhiễu từ false negatives, đồng thời tận dụng được lượng dữ liệu miễn vốn hạn chế.

Gap 3 – Tối ưu đồng thời chất lượng, độ trễ và dung lượng

Nhiều hệ thống RAG chỉ báo cáo độ chính xác hoặc truy hồi metrics mà ít phân tích đồng thời độ trễ, chi phí API, dung lượng index và kích thước model. Với hệ thống triển khai thực tế, các yếu tố này cần được đánh giá cùng nhau. Đề án tiếp cận bài toán theo hướng tối ưu nhiều mục tiêu thay vì chỉ tối ưu một metric riêng lẻ.

Gap 4 – Thay thế LLM router bằng classifier cục bộ

LLM-based định tuyến có thể đạt chất lượng tốt nhưng tạo thêm độ trễ và chi phí API cho mỗi truy vấn. Khi bài toán định tuyến chỉ gồm một số lớp ý định rõ ràng, một classifier cục bộ dựa trên SBERT/SetFit có thể là lựa chọn hiệu quả hơn. Vấn đề cần giải quyết là làm sao giữ độ chính xác cao trong điều kiện dữ liệu ít và mất cân bằng lớp.

Các khoảng trống trên dẫn tới các hướng tối ưu được trình bày ở các chương sau: xây dựng dữ liệu huấn luyện từ pipeline v1; tinh chỉnh embedding bằng MNRL kết hợp MRL để hỗ trợ nhiều số chiều triển khai; chưng cất, cắt tỉa từ vựng và lượng tử hoá reranker để chạy được trên CPU; đồng thời thay thế GPT router bằng bộ phân loại ý định chạy cục bộ.

CHƯƠNG 3. MÔ HÌNH ĐỀ XUẤT

Chương này trình bày mô hình đề xuất của đề án cho bài toán hỏi đáp trên kho tài liệu kỹ thuật tiếng Việt. Bài toán nhận đầu vào là một câu hỏi và kho tài liệu đã được lập chỉ mục, yêu cầu truy hồi đúng đoạn văn bản chứa thông tin liên quan rồi sinh câu trả lời bám sát nguồn, trong điều kiện ràng buộc về độ trễ, chi phí và khả năng triển khai trên hạ tầng nội bộ. Như đã nêu ở Chương 1, ba hạn chế chính của một pipeline RAG thông thường trên bài toán này là reranker quá nặng, định tuyến bằng LLM lớn tốn kém, và embedding pretrained chưa tối ưu cho miền.

Thay vì đề xuất giải pháp dựa trên giả định, đề án theo cách tiếp cận hai giai đoạn có căn cứ thực nghiệm. Trước hết, một hệ thống nền tảng (gọi là v1) được xây dựng và đo lường để xác định chính xác các nút thắt. Sau đó, mỗi thành phần cải tiến của hệ thống v2 được thiết kế để giải quyết một nút thắt đã được định lượng.

Để người đọc nắm được bức tranh tổng thể trước khi đi vào chi tiết, Mục 3.1 giới thiệu kiến trúc tổng quan của hệ thống đề xuất. Các mục tiếp theo trình bày chi tiết: Mục 3.2–3.5 phân tích hệ thống v1, quy trình tiền xử lý, kiến trúc pipeline và các nút thắt; Mục 3.6–3.9 trình bày các thành phần của hệ thống v2, gồm tinh chỉnh embedding, nén reranker và bộ phân loại ý định.

3.1 Tổng quan kiến trúc hệ thống đề xuất

Hệ thống đề xuất hoạt động theo hai giai đoạn tách biệt: giai đoạn chuẩn bị và huấn luyện (offline) thực hiện một lần, và giai đoạn phục vụ truy vấn (online) chạy cho mỗi câu hỏi của người dùng. Hình 3.1 minh họa kiến trúc tổng thể cùng vị trí của ba thành phần cải tiến do đề án đề xuất.

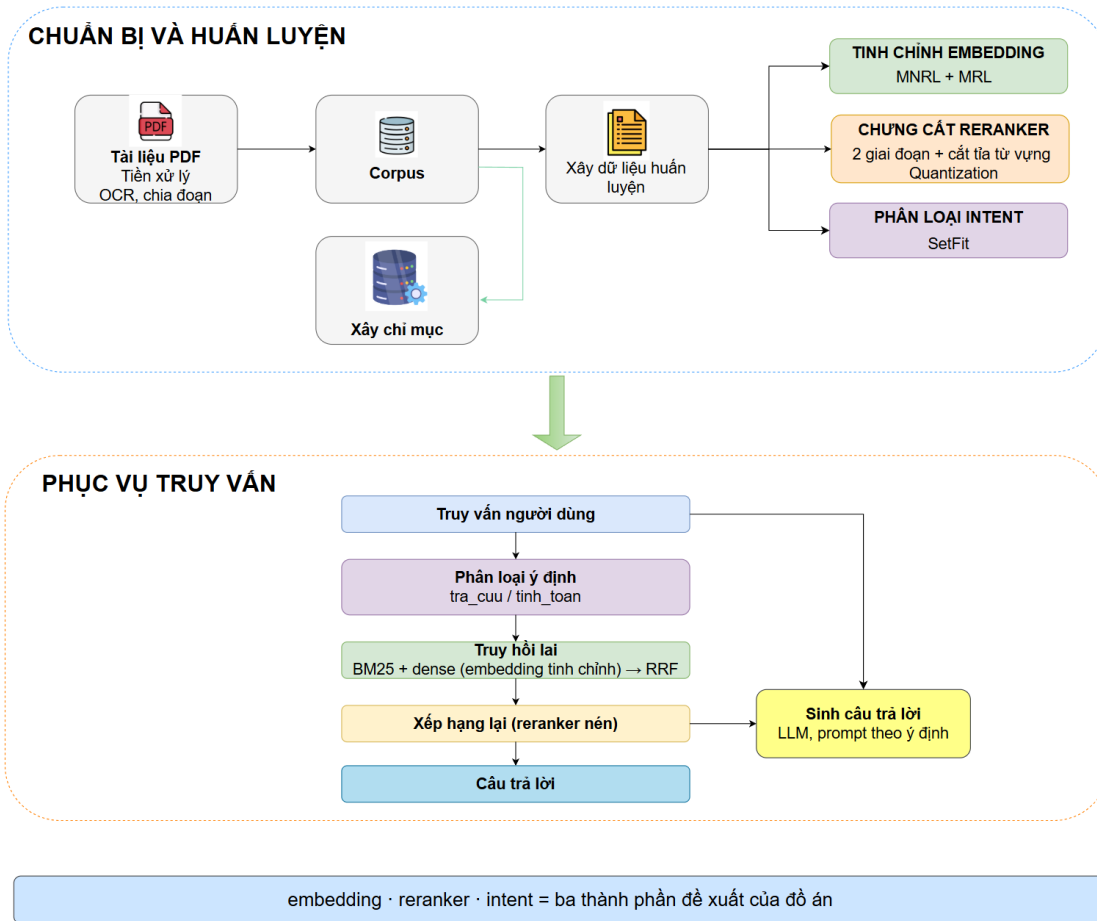


Figure 3.1: Kiến trúc tổng thể của hệ thống đề xuất

Ở giai đoạn offline, tài liệu PDF được tiền xử lý (trích xuất văn bản, chia đoạn) thành kho đoạn văn bản. Từ kho này, một quy trình chung xây dựng dữ liệu huấn luyện được dùng cho cả ba thành phần. Trên nền dữ liệu đó, đồ án (1) tinh chỉnh mô hình embedding bằng MNRL kết hợp resample positive theo epoch và MRL, (2) chưng cất reranker từ teacher BGE-reranker-v2-m3 qua hai giai đoạn kèm cắt tỉa từ vựng, và (3) huấn luyện bộ phân loại ý định bằng SetFit. Kho đoạn cũng được lập chỉ mục thành hai dạng: chỉ mục BM25 và chỉ mục vector dùng chính embedding đã tinh chỉnh.

Ở giai đoạn online, mỗi truy vấn lần lượt đi qua bốn bước. Trước hết, bộ phân loại ý định xác định câu hỏi thuộc loại tra cứu (tra_cuu) hay tính toán (tinh_toan) để chọn cách sinh câu trả lời phù hợp. Tiếp theo, hệ thống truy hồi lại kết hợp BM25 và truy hồi dày dựa trên embedding tinh chỉnh, hợp nhất kết quả bằng RRF. Tập ứng viên sau đó được reranker nén xếp hạng lại để chọn ngữ cảnh tốt nhất. Cuối cùng, mô hình ngôn ngữ sinh câu trả lời bám sát ngữ cảnh, với prompt được chọn theo ý định đã phân loại.

Như vậy, ba thành phần đề xuất không hoạt động độc lập mà phối hợp trong một

hệ thống đầu cuối: embedding tinh chỉnh cải thiện chất lượng truy hồi, reranker nén giảm chi phí ở bước xếp hạng lại, và bộ phân loại ý định cục bộ thay thế bộ định tuyến dựa trên mô hình ngôn ngữ lớn. Các mục tiếp theo của chương trình bày chi tiết từng thành phần, bắt đầu từ việc phân tích hệ thống v1 để làm rõ động lực thiết kế.

3.2 Vai trò của hệ thống v1 trong đồ án

Hệ thống RAG ban đầu, gọi là v1 trong đồ án, được xây dựng như một prototype để kiểm chứng khả năng áp dụng RAG cho bài toán hỏi đáp tài liệu kỹ thuật tiếng Việt. Mục tiêu của v1 không phải tạo ra hệ thống cuối cùng, mà là một bước phát triển trung gian nhằm trả lời ba câu hỏi thực tế: pipeline RAG cơ bản có hoạt động ổn định trên dữ liệu kỹ thuật tiếng Việt hay không; thành phần nào tạo ra bottleneck khi triển khai; và cần tối ưu ở đâu để hình thành đóng góp kỹ thuật rõ ràng.

Trong giai đoạn này, hệ thống được chạy thử trên 991 câu hỏi trắc nghiệm và 200 tài liệu PDF từ tập phát triển. Chính 991 câu hỏi này sau đó được dùng để xây dựng dữ liệu huấn luyện cho v2, gồm nhãn ý định, cặp query–positive đoạn và negative samples cho reranker và embedding. Vì vậy, kết quả trên 991 câu chỉ được xem là tín hiệu phát triển nội bộ, không phải kết quả đánh giá cuối cùng.

Tập kiểm thử độc lập gồm 390 câu hỏi (343 câu gốc cộng 47 câu tính_toan bổ sung sau, mục 4.5) từ 50 tài liệu PDF được giữ riêng cho giai đoạn đánh giá v2 và không được sử dụng để huấn luyện.

Table 3.1: Vai trò của các tập dữ liệu trong quy trình phát triển

Nhóm dữ liệu	Quy mô	Vai trò
Tập phát triển v1	991 câu hỏi, 200 tài liệu PDF	Chạy thử pipeline, quan sát lỗi, đo bottleneck và tạo dữ liệu huấn luyện v2
Dữ liệu huấn luyện v2	Xây dựng từ 991 câu hỏi	Huấn luyện reranker, embedding và bộ phân loại ý định
Tập kiểm thử độc lập	390 câu hỏi, 50 tài liệu PDF	Chỉ dùng để đánh giá v2, không dùng để huấn luyện hoặc lựa chọn cấu hình

3.3 Quy trình tiền xử lý tài liệu

3.3.1 Trích xuất văn bản từ tài liệu PDF

Tập dữ liệu gồm các tài liệu PDF kỹ thuật thuộc nhiều lĩnh vực như phần mềm kiểm thử, thiết bị y tế, ô tô, điện, hệ thống HVAC và hướng dẫn vận hành thiết bị. Đặc điểm chung của các tài liệu này là chứa nhiều bảng thông số, ký hiệu kỹ thuật và cấu trúc phân cấp theo đề mục. Do đó, bước trích xuất văn bản cần bảo toàn được cả nội dung lẫn cấu trúc, thay vì chỉ rút ra văn bản thuần. Quy trình tiền xử

lý gồm ba giai đoạn: loại bỏ phần header lặp lại, chuyển đổi PDF sang Markdown có cấu trúc, và chuẩn hóa định dạng đầu ra.

Loại bỏ header lặp lại đầu trang.

Nhiều tài liệu kỹ thuật có một bảng thông tin (tiêu đề, mã hiệu, phiên bản) lặp lại ở đầu mỗi trang. Nếu giữ nguyên, phần lặp này tạo nhiễu cho cả bước chia đoạn lẫn truy hồi. Hệ thống xác định vùng cần cắt bằng cách phát hiện bảng đầu tiên trên trang một: tỉ lệ cắt được tính theo vị trí cạnh dưới của bảng so với chiều cao trang, cộng thêm một biên nhỏ. Tỉ lệ này sau đó được áp dụng để cắt phần trên của mọi trang. Với tài liệu không phát hiện được bảng ở trang đầu, hệ thống dùng một tỉ lệ cắt mặc định.

Chuyển đổi PDF sang Markdown.

Các tài liệu sau khi cắt header được chuyển đổi sang định dạng Markdown bằng công cụ OCR Marker [22]. Quá trình này giữ lại cấu trúc đề mục (heading) và biểu diễn bảng dưới dạng HTML, cho phép bước chia đoạn ở sau phân biệt được bảng với văn bản thường. Một vấn đề thường gặp là một bảng dài bị tách qua nhiều trang sẽ được OCR nhận diện thành nhiều bảng rời rạc. Để khắc phục, hệ thống tự động ghép các bảng liên kề: khi hai bảng chỉ cách nhau bởi khoảng trắng hoặc dấu phân trang (không có văn bản xen giữa), chúng được gộp lại thành một bảng, trong đó hàng tiêu đề lặp lại của bảng phía dưới được chuyển thành hàng dữ liệu thường. Mỗi tài liệu được lưu thành một file main.md trong thư mục riêng theo định danh tài liệu, kèm một đề mục cấp cao nhất chứa định danh đó.

Chuẩn hóa định dạng Markdown.

Đầu ra của OCR còn một số điểm không nhất quán về định dạng đề mục. Bước chuẩn hóa cuối cùng xử lý các trường hợp này: các tiêu đề được đánh số dạng in đậm (ví dụ ****1.2Tiêu đề****) được chuyển thành đề mục Markdown với cấp tương ứng số thứ tự; các tiêu đề bị OCR đảo vị trí số (ví dụ Tiêu đề1.2) được nhận diện và sửa lại; và các ký hiệu danh sách được thống nhất về một dạng. Việc chuẩn hóa đề mục là cần thiết vì bước chia đoạn sau đó dựa vào cấu trúc đề mục để bảo toàn ngữ cảnh phân cấp.

Mỗi tài liệu được lưu theo định danh Public_XXX; định danh này được giữ trong metadata của từng đoạn để hỗ trợ truy xuất trực tiếp khi câu hỏi nêu rõ tài liệu nguồn, chẳng hạn Public_021. Sau toàn bộ quy trình, một bảng thông số trong tài liệu gốc vẫn giữ được cấu trúc hàng-cột. Việc bảo toàn cấu trúc bảng giúp bước chia đoạn ở mục sau xử lý đúng quan hệ giữa hàng tiêu đề và từng ô dữ liệu.

3.3.2 Chiến lược chia đoạn nhận biết bảng

Chia đoạn (chunking) quyết định đơn vị thông tin được đưa vào truy hồi. Với tài liệu kỹ thuật, đoạn quá lớn (chẳng hạn trên 1.000 từ) làm loãng thông tin và giảm độ chính xác truy hồi; ngược lại, đoạn quá nhỏ (dưới 100 từ) có thể làm mất ngữ cảnh hoặc tách hàng tiêu đề khỏi dữ liệu bảng. Vì văn bản thường và bảng có cấu trúc khác nhau, đồ án sử dụng một chiến lược chia đoạn nhận biết bảng, xử lý riêng hai loại nội dung này.

Chia đoạn văn bản thường.

Văn bản thuần được chia theo đơn vị đoạn văn (paragraph) với ngân sách khoảng 512 từ mỗi đoạn và phần chồng lấn (overlap) khoảng 100 từ giữa hai đoạn liên tiếp. Phần chồng lấn hạn chế mất thông tin tại ranh giới giữa các đoạn. Mỗi đoạn lưu lại đường dẫn phân cấp đề mục (chuỗi heading dạng $H1 > H2 > H3$ dẫn tới đoạn văn) nhằm bảo toàn ngữ cảnh cấu trúc của tài liệu.

Chia đoạn bảng biểu.

Mỗi bảng được chia theo cửa sổ trượt 10 hàng với phần chồng lấn 2 hàng. Hàng tiêu đề được thêm lại vào đầu mỗi đoạn con để giữ ý nghĩa của từng cột. Nội dung HTML của bảng sau đó được chuyển thành văn bản có cấu trúc, ví dụ:

Cột: Thông số | Giá trị | Đơn vị

Dòng 1: Nhiệt độ vận hành tối đa | 85 | độ C

Dòng 2: Áp suất vận hành | 3.0 | bar

Dòng 3: Công suất tiêu thụ | 500 | W

Biểu diễn dạng văn bản này phù hợp với mô hình embedding hơn HTML thô, vì mô hình chủ yếu được huấn luyện trước trên ngôn ngữ tự nhiên.

Metadata của đoạn.

Mỗi đoạn được gán bốn trường metadata chính: định danh tài liệu nguồn, đường dẫn phân cấp đề mục chứa đoạn, loại đoạn (văn bản hoặc bảng), và chỉ số thứ tự của đoạn trong tài liệu. Các trường này hỗ trợ lọc theo tài liệu, xây dựng chỉ mục cấp tài liệu và truy vết nguồn gốc khi phân tích lỗi.

Tổng hợp lại, quy trình chia đoạn gồm bốn bước: tách file Markdown thành các đoạn văn bản và bảng dựa trên thẻ HTML; với đoạn văn bản, gộp các paragraph tới giới hạn kích thước đồng thời giữ phần chồng lấn cho đoạn kế tiếp; với bảng, chia theo cửa sổ hàng và thêm lại hàng tiêu đề; cuối cùng chuyển mỗi đoạn thành văn bản, gán metadata và chỉ số thứ tự.

3.3.3 Xây dựng chỉ mục

Sau chunking, các đoạn được đưa vào ba cấu trúc chỉ mục phục vụ những pipeline khác nhau.

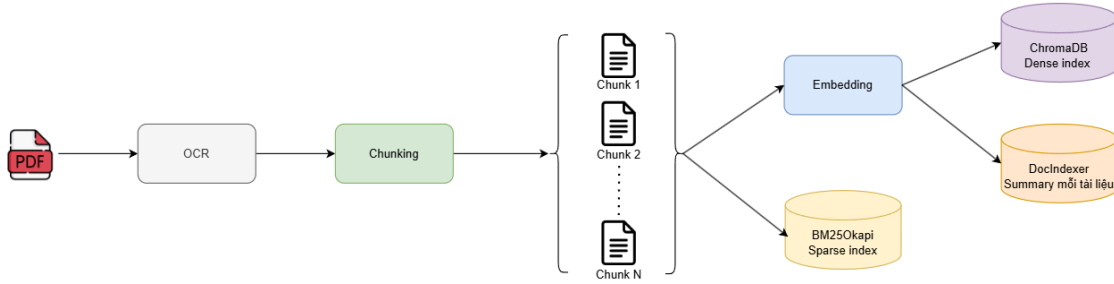


Figure 3.2: Luồng xử lý xây dựng chỉ mục

ChromaDB

ChromaDB lưu dense embeddings của các đoạn. Embedding được tạo bằng AITeamVN/Vietnamese_Embedding_v2 dưới dạng vector 1024 chiều chuẩn hóa L2.

BM25Okapi

BM25Okapi lập chỉ mục toàn bộ đoạn text cho truy hồi thưa (sparse). Thành phần này không cần GPU, có độ trễ dưới khoảng 10ms/query và phù hợp với mã số, ký hiệu hoặc tên thiết bị mà dense model có thể chưa biểu diễn tốt.

DocIndex

DocIndex là chỉ mục bổ sung dành cho Router-Summary, trong đó mỗi tài liệu được biểu diễn bằng một summary vector ở cấp tài liệu. Từ “summary” ở đây không phải là đoạn tóm tắt sinh bởi LLM, mà là biểu diễn vector cô đọng được tính offline từ các đoạn của tài liệu. Mục tiêu là chọn nhanh một vài tài liệu liên quan trước khi truy hồi chi tiết ở cấp đoạn.

Với tài liệu có ít nhất ba text đoạn, DocIndex dùng TextRank trên graph tương đồng giữa các đoạn. Mỗi text đoạn đã có embedding chuẩn hóa L2. Các đoạn được xem như các đỉnh của graph, còn trọng số cạnh là cosine similarity giữa hai embedding:

$$s_{ij} = \mathbf{e}_i^\top \mathbf{e}_j, \quad i \neq j. \quad (3.1)$$

Sau khi bỏ self-loop và chuẩn hóa ma trận similarity theo hàng, thuật toán chạy PageRank với hệ số damping $d = 0.85$:

$$\mathbf{w}^{(t+1)} = \frac{1-d}{n} \mathbf{1} + dS^\top \mathbf{w}^{(t)}. \quad (3.2)$$

Quá trình lặp dừng khi trọng số hội tụ hoặc đạt tối đa 100 vòng. Vector summary phần text của tài liệu sau đó được tính bằng trung bình có trọng số:

$$\mathbf{v}_{text} = \sum_i w_i \mathbf{e}_i. \quad (3.3)$$

Nếu tài liệu có cả bảng, vector cuối cùng được blend giữa text summary và trung bình embedding của bảng theo tỉ lệ 85/15. Với tài liệu chỉ có một hoặc hai text đoạn, TextRank không ổn định vì graph quá nhỏ, nên hệ thống dùng mean text embedding và blend với table embedding theo tỉ lệ 70/30. Với tài liệu chỉ có bảng, hệ thống dùng trung bình có trọng số theo phân cấp: đoạn nằm sâu hơn trong cây heading và xuất hiện sớm hơn trong tài liệu được gán trọng số cao hơn.

Khi truy vấn, DocIndex thu hẹp không gian tìm kiếm từ toàn bộ kho tài liệu xuống top-3 tài liệu trước khi thực hiện truy vấn cấp đoạn.

3.4 Kiến trúc hai pipeline RAG

Trên nền tiền xử lý và chỉ mục chung, đồ án so sánh hai pipeline chính: Baseline Router và Router-Summary. Hai pipeline này dùng chung đoạn, embedding, cache, router GPT-4o-mini và reranker BGE-reranker-v2-m3, nhưng khác nhau ở phạm vi truy xuất và việc sử dụng tóm tắt cấp tài liệu.

3.4.1 Pipeline Baseline Router

Baseline thực hiện truy xuất lai (hybrid) trực tiếp trên toàn corpus rồi đưa ngữ cảnh vào LLM để sinh đáp án.

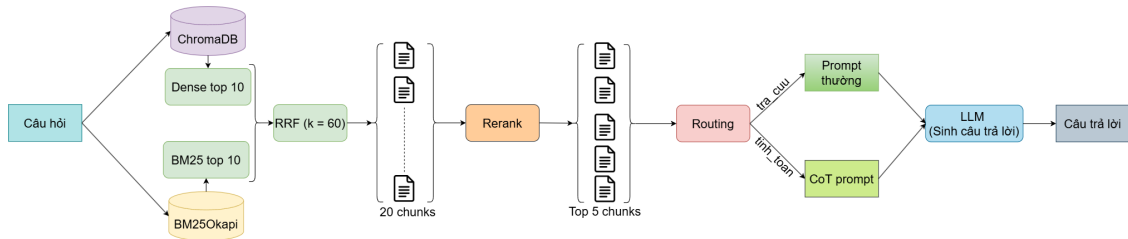


Figure 3.3: Luồng xử lý của pipeline Baseline

Bước 1 – Truy hồi lai

BM25 top-10 và ChromaDB dense top-10 chạy độc lập. Kết quả được hợp nhất bằng RRF với $k = 60$ để tạo ứng viên pool.

nêu; nếu không, hệ thống chuyển sang chọn tài liệu bằng DocIndexer.

Bước 2 – Chọn tài liệu cấp cao

Khi không có Public ID, DocIndexer chọn ra một số tài liệu liên quan nhất từ toàn corpus. Mỗi tài liệu được biểu diễn sẵn bằng một summary vector (mục 3.3.3) nên bước này không cần đọc lại toàn bộ tài liệu lúc truy vấn. DocIndexer kết hợp hai tín hiệu cấp tài liệu — BM25 trên văn bản ghép của tài liệu và độ tương đồng cosine với summary vector — qua Reciprocal Rank Fusion, rồi lấy top-3 tài liệu.

Bước 3 – Truy hồi và xếp hạng lại cấp đoạn

Trong phạm vi các tài liệu đã chọn, hệ thống thực hiện truy hồi lại cấp đoạn (BM25 và dense, hợp nhất bằng RRF, giữ tối đa 5 đoạn mỗi tài liệu), sau đó dùng cross-encoder BGE-reranker-v2-m3 xếp hạng lại và giữ top-5 đoạn làm ngữ cảnh. Việc thu hẹp phạm vi tài liệu trước khi rerank giúp giảm nhiễu, đặc biệt hiệu quả với nhóm `tinh_toan` — nơi ngữ cảnh sai hoặc thiếu một con số có thể làm sai đáp án cuối.

Bước 4 – Sinh câu trả lời theo ý định

Với câu `tra_cuu`, hệ thống dùng prompt trực tiếp để chọn đáp án A/B/C/D. Với câu `tinh_toan`, hệ thống sinh phân tích từng bước trước, rồi mới chọn đáp án cuối.

3.4.3 So sánh kiến trúc hai pipeline

Table 3.2: So sánh kiến trúc Baseline Router và Router-Summary

Đặc điểm	Baseline Router	Router-Summary
Phạm vi truy hồi	Hybrid truy hồi trực tiếp trên toàn corpus	DocIndexer chọn top-3 tài liệu trước khi retrieve cấp đoạn
Phạm vi rerank	Tập ứng viên toàn bộ tài liệu	Tập ứng viên sau khi thu hẹp theo tài liệu
Phân loại ý định	GPT-4o-mini router	GPT-4o-mini router
Số lần gọi LLM/câu	2 lần với <code>tra_cuu</code> ; 3 lần với <code>tinh_toan</code>	2 lần với <code>tra_cuu</code> ; 3 lần với <code>tinh_toan</code>
Prompt theo ý định	Prompt trực tiếp cho <code>tra_cuu</code> ; CoT cho <code>tinh_toan</code>	Prompt trực tiếp cho <code>tra_cuu</code> ; CoT cho <code>tinh_toan</code>
Tập test	200 câu hỏi	200 câu hỏi

3.5 Xác định các nút thắt

Quá trình vận hành và đo đạc hệ thống v1 cho thấy ba nút thắt chính về độ trễ, chi phí và chất lượng truy xuất. Mỗi vấn đề gắn với một thành phần cụ thể và dẫn đến một hướng tối ưu riêng trong v2. Các số liệu định lượng chi tiết của v1 (accuracy, độ trễ) được trình bày trong Chương 4; mục này tập trung phân tích bản chất của từng nút thắt.

3.5.1 Nút thắt độ trễ: BGE-reranker-v2-m3

BAAI/bge-reranker-v2-m3 gồm 568M tham số và là thành phần nặng nhất trong quy trình truy xuất. Do đặc trên v1 cho thấy bước tái xếp hạng bằng Bge-reranker-v2-m3 chiếm phần lớn tổng độ trễ của pipeline; việc thu hẹp phạm vi tài liệu trước khi tái xếp hạng (như trong Router-Summary) giảm được một phần chi phí này, nhưng reranker vẫn là thành phần cần tối ưu nếu muốn giảm thời gian suy luận. Số liệu độ trễ cụ thể được trình bày ở Chương 4.

Giảm kích thước model giúp giảm yêu cầu bộ nhớ và thời gian tải, nhưng một MiniLM chưa qua huấn luyện trên miền có chất lượng xếp hạng thấp. Vì vậy, bài toán đặt ra là chung cất tri thức từ BGE-reranker-v2-m3 sang các reranker nhỏ hơn (từ MiniLM 33M tới mmarco/pruned) mà vẫn duy trì chất lượng xếp hạng. So sánh độ trễ và chất lượng giữa các reranker được trình bày ở Chương 4.

3.5.2 Nút thắt chi phí: bộ định tuyến ý định GPT

Router-Summary dùng GPT-4o-mini cho bước phân loại ý định trước khi sinh đáp án. Lần gọi router bổ sung độ trễ và chi phí API cho mỗi truy vấn, dù tác vụ chỉ phân biệt hai lớp tra_cuu và tinh_toan — một bài toán không đòi hỏi model có năng lực sinh ngôn ngữ lớn. Khi triển khai ở quy mô lớn, chi phí gọi API cho bước định tuyến tích lũy đáng kể, đồng thời tạo phụ thuộc vào dịch vụ ngoài. Một router cục bộ dựa trên Sentence-BERT/SetFit có thể chạy nội bộ với chi phí suy luận biên gần bằng không và loại bỏ phụ thuộc mạng. Phân tích chi phí và độ trễ định lượng được trình bày ở Chương 4.

3.5.3 Nút thắt chất lượng: embedding chưa tối ưu theo miền

Vietnamese_Embedding_v2 là mô hình embedding tiếng Việt mạnh nhưng chưa được tối ưu riêng cho miền tài liệu kỹ thuật của đề án. Khi một phần truy vấn chưa có gold đoạn ở thứ hạng cao nhất, gánh nặng cải thiện thứ hạng dồn sang bộ tái xếp hạng. Nếu gold đoạn rơi khỏi nhóm top-k ứng viên, bộ tái xếp hạng cũng không thể khôi phục. Phân tích định tính cho thấy lỗi truy xuất là một nhóm lỗi đáng kể: các câu hỏi về thông số, bảng số liệu, ký hiệu và thuật ngữ chuyên ngành đòi hỏi embedding nắm được quan hệ đặc thù của miền, thay vì chỉ dựa trên ngữ nghĩa tiếng Việt tổng quát.

Vì mô hình nền vốn đã khá mạnh, hướng tối ưu embedding ở v2 không chỉ nhằm tới độ chính xác mà còn tới hiệu quả biểu diễn: tinh chỉnh bằng MNRL với resample positive theo epoch để thích nghi miền, kết hợp Matryoshka Representation Learning để tạo biểu diễn đa kích thước, cho phép sử dụng số chiều nhỏ hơn mà không làm giảm đáng kể chất lượng. Mức cải thiện cụ thể được đánh giá ở Chương 4.

Ba nút thắt trên xác định ba hướng tối ưu của v2: xây dựng bộ tái xếp hạng nhẹ bằng chung cắt tri thức để giảm độ trễ; huấn luyện router cục bộ dựa trên Sentence-BERT/SetFit để thay thế GPT router; và tinh chỉnh embedding bằng MRL và resample positive theo epoch để thích nghi miền và giảm chi phí biểu diễn.

3.6 Tổng quan định hướng v2

Từ ba nút thắt được xác định ở các mục trên, hệ thống v2 được thiết kế theo nguyên tắc mỗi thay đổi kỹ thuật phải giải quyết một vấn đề đã được đo lường, thay vì tối ưu dựa trên giả định. Ba hướng tối ưu tương ứng được trình bày trong Bảng 3.3.

Table 3.3: Ánh xạ nút thắt của v1 sang giải pháp v2

Nút thắt v1	Giải pháp v2	Mục tiêu thiết kế
Mô hình embedding nền chưa tối ưu theo miền	Tinh chỉnh bằng MRL và resample positive theo epoch	Cải thiện truy xuất, hỗ trợ triển khai 512 chiều và giảm dung lượng lưu trữ
BGE-reranker-v2-m3: 568M tham số, độ trễ lớn khi tái xếp hạng top-20	Chung cất sang mmarco 117M, pruned 35.6M và MiniLM 33M	Giữ MRR@10 gần với mô hình teacher, giảm độ trễ pipeline và giảm dung lượng khi triển khai
GPT router: độ trễ và chi phí API cho mỗi truy vấn	Huấn luyện router cục bộ dựa trên Sentence-BERT/SetFit	Đạt độ chính xác cao, độ trễ thấp và có thể chạy cục bộ

Ba hướng được trình bày theo trình tự của pipeline: trước hết là tinh chỉnh embedding ở tầng truy xuất, tiếp đến là xây dựng bộ tái xếp hạng nhẹ, rồi xây dựng bộ phân loại ý định. Phần chuẩn bị dữ liệu huấn luyện được xây dựng chung, phục vụ embedding trước và được tái sử dụng cho bộ tái xếp hạng. Kết quả đánh giá của cả ba hướng được tổng hợp trong Chương 4.

3.7 Tinh chỉnh embedding bằng MRL và resample positive theo epoch

3.7.1 Xác định positive cho embedding

Dữ liệu positive cho embedding được xây dựng từ 991 câu hỏi gốc qua hai giai đoạn đầu của quy trình dữ liệu dùng chung với reranker (giai đoạn 3–5 dành cho tổng hợp và khai thác hard negative được trình bày riêng ở mục 3.8.2, không dùng cho embedding).

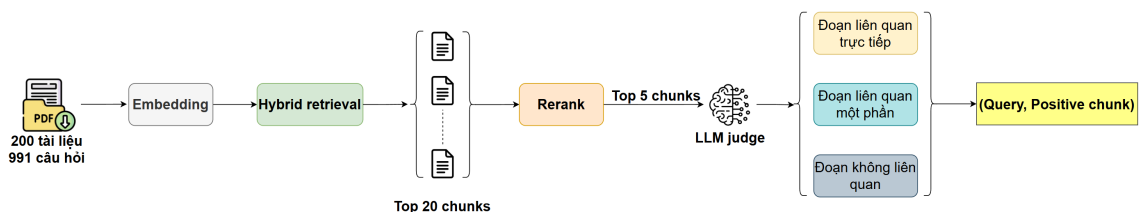


Figure 3.5: Quy trình xác định positive cho embedding

Giai đoạn 1 – Truy hồi và xếp hạng lại. Toàn bộ 991 câu hỏi được đưa qua truy hồi lại (BM25, truy hồi dense và hợp nhất bằng RRF), sau đó được xếp hạng lại bằng BGE-reranker-v2-m3. Mỗi câu hỏi giữ lại 20 ứng viên hàng đầu kèm điểm số và thứ hạng của teacher.

Giai đoạn 2 – Xác định positive bằng mô hình ngôn ngữ. Thay vì mặc định ứng viên xếp hạng cao nhất là positive, một mô hình ngôn ngữ được dùng làm giám khảo để phân loại từng ứng viên thành liên quan trực tiếp, liên quan một phần hoặc không liên quan. Một câu hỏi có thể có nhiều hơn một positive nếu nhiều ứng viên cùng được đánh giá là liên quan trực tiếp. Sau khi loại các câu hỏi không có positive đáng tin cậy, còn lại 792 câu hỏi hợp lệ, chia 80/20 phân tầng theo ý định (634 câu hỏi huấn luyện, 158 câu hỏi kiểm thử in-domain — tập D1).

3.7.2 Gộp positive và resample theo epoch

Nhiều câu hỏi trong 634 câu hỏi huấn luyện có nhiều hơn một positive (nhiều ứng viên cùng được giám khảo đánh giá liên quan trực tiếp). Thay vì chọn cố định một positive đại diện (ví dụ positive có điểm BGE-reranker-v2-m3 cao nhất) và dùng lại positive đó ở mọi epoch, đồ án gộp toàn bộ positive của mỗi câu hỏi thành một danh sách, sau đó ở mỗi epoch chọn ngẫu nhiên đúng một positive trong danh sách này để tạo cặp huấn luyện, với hạt giống ngẫu nhiên đổi theo từng epoch. Nhờ vậy, qua nhiều epoch, một câu hỏi có nhiều positive sẽ lần lượt được ghép với các positive khác nhau thay vì luôn cố định một lựa chọn, giúp mô hình tiếp xúc với nhiều cách diễn đạt hợp lệ của cùng một câu trả lời mà không cần tăng số cặp huấn luyện trong mỗi epoch.

3.7.3 Thiết kế hàm mất mát cho tầng embedding

Mô hình nền là AITeamVN/Vietnamese_Embedding_v2, dựa trên BGE-M3 và tạo vector 1024 chiều. Hàm mất mát là MultipleNegativesRankingLoss (MNRL): với kích thước batch N , mỗi câu hỏi phân biệt positive của nó với $N - 1$ mẫu còn lại trong cùng batch, đóng vai trò in-batch negative:

$$\mathcal{L}_{\text{MNRL}} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_i})/\tau)}{\sum_{j=1}^N \exp(\cos(\mathbf{e}_{q_i}, \mathbf{e}_{p_j})/\tau)} \quad (3.4)$$

Trong thực nghiệm, hệ số scale bằng 20, tương ứng nhiệt độ $\tau = 0.05$. Vì mỗi câu hỏi chỉ có một positive tại mỗi epoch (theo cơ chế resample ở mục 3.7.2) và không có negative tường minh, toàn bộ tín hiệu phân biệt đến từ in-batch negative — positive của các câu hỏi khác trong cùng batch.

Hàm mất mát được bọc trong MatryoshkaLoss để tối ưu đồng thời ba kích thước

$\mathcal{M} = \{256, 512, 1024\}$, mỗi kích thước được tính trên phần đầu (prefix) $f_m(\mathbf{x}) = \mathbf{x}[:m]$ của vector embedding với trọng số bằng nhau giữa ba kích thước:

$$\mathcal{L} = \text{MatryoshkaLoss}(\mathcal{L}_{\text{MNRL}}) \quad (3.5)$$

3.7.4 Cấu hình huấn luyện

Huấn luyện gồm một giai đoạn duy nhất.

Table 3.4: Cấu hình huấn luyện embedding

Hàm mất mát	Dữ liệu	Epoch	Batch	Tốc độ học	Độ dài chuỗi
MNRL + MatryoshkaLoss	634 câu hỏi, positive resample theo epoch	2	16	2×10^{-6}	1024

Các tham số chung gồm bộ tối ưu AdamW, hệ số suy giảm trọng số bằng 0.01, ngưỡng cắt gradient bằng 1.0, huấn luyện ở độ chính xác hỗn hợp (mixed precision), 10 bước khởi động (warmup). Vì mỗi epoch resample lại positive (mục 3.7.2), tập huấn luyện được tái tạo và nạp lại DataLoader ở đầu mỗi epoch thay vì huấn luyện nhiều epoch liên tục trên cùng một tập cặp cố định.

3.7.5 Chiến lược triển khai với MRL

Một checkpoint hỗ trợ ba cấu hình:

Table 3.5: Các chế độ triển khai từ một checkpoint MRL

Số chiều	Cách sử dụng	Mục tiêu
256	Cắt vector còn 256 chiều	Tìm kiếm nhanh, chỉ mục nhỏ hơn 4 lần
512	Cắt vector còn 512 chiều	Cân bằng chất lượng và tài nguyên
1024	Kích thước mặc định	MRR/NDCG cao nhất khi không bị giới hạn dung lượng

Một checkpoint MRL hỗ trợ ba cấu hình triển khai: 256 chiều khi ưu tiên tốc độ và bộ nhớ (chỉ mục nhỏ hơn 4 lần), 512 chiều cân bằng giữa chất lượng và tài nguyên (dung lượng nhỏ hơn 2 lần so với 1024 chiều), và 1024 chiều khi cần tối đa hóa chất lượng. Thực nghiệm ở Chương 4 cho thấy 1024 chiều đạt MRR/NDCG cao nhất, nhưng đề án vẫn chọn 512 chiều làm cấu hình triển khai chính vì mức chênh lệch chất lượng so với 1024 chiều là nhỏ trong khi dung lượng chỉ mục giảm một nửa — đánh đổi hợp lý cho một hệ thống chạy trên tài nguyên hạn chế.

3.8 Reranker nhẹ bằng chứng cất tri thức

3.8.1 Động lực và thiết kế tổng quan

Mục tiêu của hướng reranker là thay BGE-reranker-v2-m3 (568M tham số) bằng các reranker nhỏ hơn nhưng vẫn giữ được chất lượng xếp hạng. Đề án thử ba mức dung lượng mô hình: MiniLM-L12 33M (cross-encoder/ms-marco-MiniLM-L12-v2) cho cấu hình siêu nhẹ, mmarco-mMiniLMv2 117M (cross-encoder/mmarco-mMiniLMv2-L12-H384-v1) cho cấu hình cân bằng, và mmarco sau cắt tỉa còn 35.6M để giảm chi phí triển khai (dung lượng mô hình, tài nguyên bộ nhớ và chi phí suy luận) nhưng vẫn tận dụng được encoder đa ngôn ngữ mạnh.

Một quyết định thiết kế quan trọng là có cần bước thích nghi miền (giai đoạn A) trước khi chưng cất hay không. Giả thuyết là khi chưa thích nghi với miền dữ liệu đích, student chưa hình thành biểu diễn phù hợp cho đầu vào tiếng Việt — do khoảng cách miền giữa MS MARCO tiếng Anh [23] và tài liệu kỹ thuật tiếng Việt — nên việc tiếp nhận tín hiệu xếp hạng từ teacher kém hiệu quả. Vì vậy, đề án thiết kế quy trình hai giai đoạn (giai đoạn A học tương phản trước, giai đoạn B chưng cất sau); so sánh với phương án chưng cất trực tiếp được trình bày ở Chương 4.

- **Giai đoạn A – Học tương phản:** thích nghi miền bằng dữ liệu kỹ thuật tiếng Việt và hàm mất mát BCE (entropy chéo nhị phân).
- **Giai đoạn B – Chưng cất tri thức:** từ checkpoint giai đoạn A, chưng cất tín hiệu xếp hạng của BGE-reranker-v2-m3 để học mức độ liên quan tinh tế hơn.

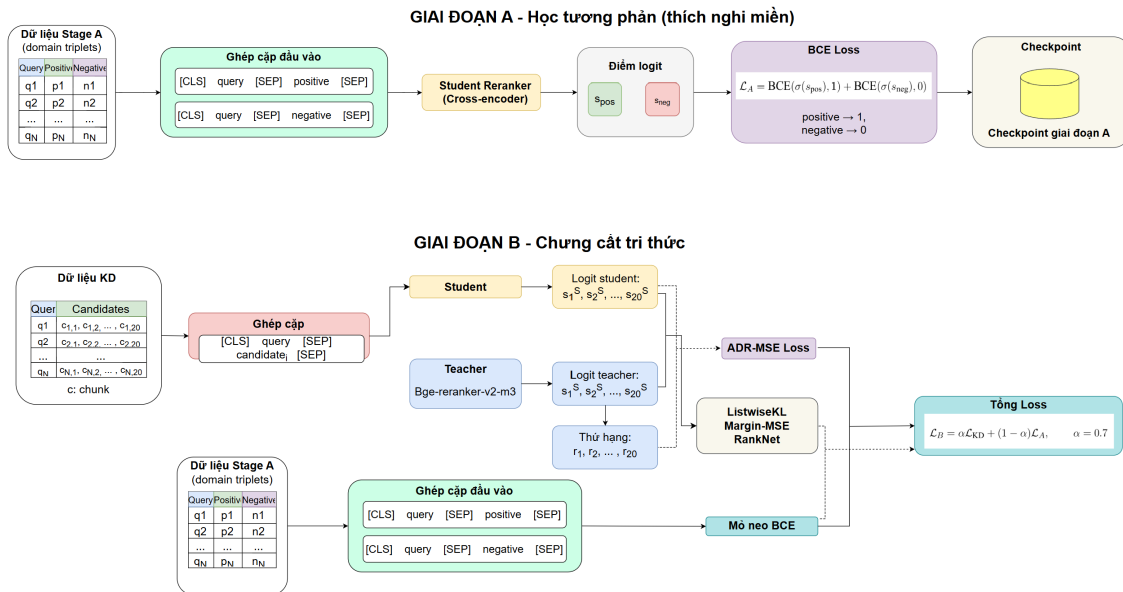


Figure 3.6: Kiến trúc huấn luyện reranker hai giai đoạn

Tín hiệu teacher từ BGE-reranker-v2-m3.

BGE-reranker-v2-m3 là mô hình cross-encoder: với mỗi cặp (q, c_i) gồm câu hỏi và đoạn ứng viên, mô hình trả về một điểm liên quan thực s_i^T (logit), không phải xác suất đã chuẩn hóa trong khoảng $[0, 1]$. Sau khi chấm 20 ứng viên của một câu hỏi, hệ thống sắp xếp các đoạn theo s_i^T giảm dần để tạo thứ hạng teacher. Vì vậy, dữ liệu chung cất lưu hai loại tín hiệu: điểm logit thô của từng ứng viên và vị trí thứ hạng sau khi xếp hạng lại.

Điểm logit thô mang thông tin về độ tự tin tương đối nhưng không được hiệu chỉnh xác suất; khoảng cách logit có thể rất khác nhau giữa các câu hỏi. Do đó các hàm mất mát được thiết kế theo hai hướng. Listwise KL chuyển logit của teacher và student thành phân phối mềm bằng softmax theo từng danh sách ứng viên; Margin-MSE và RankNet dùng chênh lệch logit giữa các cặp ứng viên; còn ADR-MSE bỏ qua thang điểm tuyệt đối của teacher và chỉ chung cất thứ hạng bằng cách ánh xạ thứ hạng về mục tiêu trong khoảng $[-1, +1]$. Như vậy, BGE-reranker-v2-m3 đóng vai trò cung cấp tín hiệu liên quan và thứ hạng, còn từng hàm mất mát quyết định cách chuẩn hóa hoặc khai thác tín hiệu đó trong quá trình chung cất.

3.8.2 Điều chỉnh dữ liệu cho reranker

Reranker không xây dựng lại dữ liệu từ đầu mà kế thừa kết quả trung gian của quy trình dữ liệu embedding ở mục 3.7.1: tập câu hỏi kèm ứng viên đã xếp hạng và nhãn positive do mô hình ngôn ngữ xác định. Điểm khác là reranker cần dữ liệu dạng bộ ba để học xếp hạng tương phản, nên cần thêm bước khai thác negative và chia tập theo câu hỏi để tránh rò rỉ dữ liệu (leakage).

Khai thác negative cho reranker.

Reranker dùng các ứng viên đã xếp hạng từ BGE-reranker-v2-m3 để khai thác negative. Với các đoạn ngoài top-5 đã được giám khảo đánh giá, hệ thống xét các ứng viên ở hạng 6 đến 20; riêng trong top-5, những đoạn được xác nhận là không liên quan vẫn có thể được giữ. Các đoạn liên quan trực tiếp hoặc một phần luôn bị loại. Đoạn cùng tài liệu với positive chỉ được giữ nếu được xác nhận không liên quan, còn đoạn khác tài liệu được giữ nếu vượt ngưỡng chênh lệch điểm. Chênh lệch điểm được dùng để phân loại độ khó: khó trong khoảng 0.1–0.4, trung bình trong khoảng 0.4–0.7, và dễ khi lớn hơn 0.7; chỉ negative khó và trung bình được giữ lại cho tập huấn luyện.

Quy trình thu được 9.917 negative khai thác, trong đó 913 negative khó và 1.676 negative trung bình được sử dụng cho tập reranker.

Table 3.6: Thống kê negative khai thác theo nguồn

Nguồn	Số negatives	Tỉ lệ
Khác tài liệu	9.475	95.5%
Cùng tài liệu và được xác nhận không liên quan	442	4.5%
Tổng	9.917	100%

Chia tập cho reranker.

Các bộ ba (câu hỏi, positive, negative khai thác) được chia theo câu hỏi, không theo bộ ba, để cùng một câu hỏi không xuất hiện ở cả tập huấn luyện và tập kiểm định. Tỉ lệ 90/10 tạo **2.328 bộ ba huấn luyện** và **305 bộ ba kiểm định**, tổng cộng 2.633 bộ ba — toàn bộ đều là negative khai thác từ kết quả xếp hạng thật.

Table 3.7: Tổng quan tập dữ liệu huấn luyện reranker

Tập	Số bộ ba	Số câu hỏi	Tỉ lệ
Huấn luyện	2.328	353	88.4%
Kiểm định	305	50	11.6%
Tổng	2.633	403	100%

3.8.3 Giai đoạn A: Học tương phản

Giai đoạn A tinh chỉnh MiniLM-L12 bằng hàm mất mát entropy chéo nhị phân (BCE) trên các bộ ba trong miền:

$$\mathcal{L}_A = \text{BCE}(\sigma(s_{\text{pos}}), 1) + \text{BCE}(\sigma(s_{\text{neg}}), 0) \quad (3.6)$$

$$\mathcal{L}_A = -\log \sigma(s_{\text{pos}}) - \log (1 - \sigma(s_{\text{neg}})) \quad (3.7)$$

trong đó s_{pos} và s_{neg} là logit của MiniLM cho đoạn positive và đoạn negative.

Một vấn đề thiết kế ở giai đoạn A là có nên bổ sung dữ liệu mMARCO-VI [24] hay không. Động cơ là MiniLM chỉ huấn luyện trên tiếng Anh nên chưa có biểu diễn tốt cho tiếng Việt; một lượng dữ liệu truy hỏi tiếng Việt tổng quát có thể giúp student làm quen với ngôn ngữ trước khi học phân biệt hard negative kỹ thuật. Đồ án đã thử bổ sung mMARCO-VI ở giai đoạn A kết hợp Listwise KL, nhưng không thu được cải thiện đáng kể so với chỉ dùng dữ liệu miền; cấu hình cuối cùng vì vậy không bổ sung mMARCO-VI cho bất kỳ kiến trúc student nào, kể cả MiniLM.

Table 3.8: Siêu tham số của giai đoạn A

Tham số	Giá trị
Tốc độ học	2×10^{-5}
Kích thước batch	32
Số epoch	5
Dừng sớm (patience)	2
Hạt giống ngẫu nhiên	42
Bộ tối ưu	AdamW

3.8.4 Giai đoạn B: Chứng cất tri thức

Giai đoạn B chứng cất BGE-reranker-v2-m3 vào checkpoint giai đoạn A. Với mỗi câu hỏi, teacher và student cùng chấm điểm 20 ứng viên, tạo hai vector logit $\mathbf{s}^T$ và $\mathbf{s}^S$ trên cùng một danh sách ứng viên. Hàm mất mát kết hợp tín hiệu chứng cất và mỏ neo tương phản:

$$\mathcal{L}_B = \alpha \mathcal{L}_{\text{KD}} + (1 - \alpha) \mathcal{L}_A, \quad \alpha = 0.7 \quad (3.8)$$

Với cấu hình này, thành phần chứng cất có trọng số 0.7, còn thành phần mỏ neo tương phản có trọng số $1 - \alpha = 0.3$. Thành phần mỏ neo giúp student duy trì khả năng phân biệt các negative trong miền kỹ thuật đã học ở giai đoạn A, thay vì chỉ tối ưu theo tín hiệu xếp hạng của teacher. Việc lựa chọn α và so sánh với cấu hình chỉ dùng chứng cất ($\alpha = 1.0$) được phân tích ở Chương 4.

Listwise KL Divergence

$$\mathcal{L}_{\text{KL}} = \sum_i P_i^T \log \frac{P_i^T}{P_i^S}, \quad P_i^T = \frac{\exp(s_i^T / \tau)}{\sum_j \exp(s_j^T / \tau)}, \quad P_i^S = \frac{\exp(s_i^S / \tau)}{\sum_j \exp(s_j^S / \tau)} \quad (3.9)$$

Với $\tau = 2.0$, softmax làm mượt logit thô của teacher và tạo phân phối tương đối trong phạm vi 20 ứng viên của từng câu hỏi. Phân phối này không phải xác suất liên quan tuyệt đối, mà chỉ thể hiện mức ưu tiên tương đối giữa các ứng viên trong cùng một danh sách. Hàm mất mát tối ưu để phân phối của student tiến gần phân phối của teacher. Trong thực nghiệm, Listwise KL nhạy với các câu hỏi nhiễu; việc lọc còn 792 câu hỏi có positive rõ ràng giúp quá trình huấn luyện ổn định hơn.

Margin-MSE

$$\mathcal{L}_{\text{MMSE}} = \frac{1}{|P|} \sum_{(i,j) \in P} [(s_i^S - s_j^S) - (s_i^T - s_j^T)]^2 \quad (3.10)$$

Margin-MSE học khoảng cách tương đối giữa các ứng viên. Tuy nhiên, do logit của teacher có biên độ rộng và một tỉ lệ đáng kể các cặp là cặp dễ với margin lớn, ở giai đoạn đầu khi margin của student còn nhỏ, các cặp dễ có margin teacher lớn có thể tạo gradient mạnh hơn nhiều so với các hard negative có margin nhỏ. Điều này khiến hàm mất mát dễ bị chi phối bởi các cặp vốn đã dễ phân biệt. Phân bố margin teacher cụ thể được trình bày ở Chương 4.

RankNet với nhãn mềm của teacher.

$$P_{ij}^T = \sigma(s_i^T - s_j^T), \quad P_{ij}^S = \sigma(s_i^S - s_j^S) \quad (3.11)$$

$$\mathcal{L}_{\text{RN}} = -\frac{1}{n^2 - n} \sum_{i \neq j} [P_{ij}^T \log P_{ij}^S + (1 - P_{ij}^T) \log(1 - P_{ij}^S)] \quad (3.12)$$

Với 20 ứng viên, RankNet tối ưu 380 cặp có thứ tự cho mỗi câu hỏi. Tín hiệu theo từng cặp này phong phú hơn so với chỉ học một positive, nhưng vẫn chịu ảnh hưởng bởi thang điểm của logit teacher: khi chênh lệch logit quá lớn, sigmoid dễ bão hòa và làm giảm độ nhạy giữa các mức liên quan gần nhau.

ADR-MSE (Approx Discounted Rank MSE)

ADR-MSE là một hàm mất mát theo hướng chưng cất dựa trên thứ hạng, được kế thừa từ Rank-DistILLM [10]. Thay vì dùng điểm teacher, hàm mất mát sử dụng vị trí thứ hạng $r_i \in \{0, \dots, n-1\}$ và ánh xạ tuyến tính về $[-1, +1]$:

$$\tilde{r}_i = 1 - \frac{2r_i}{n-1} \quad (3.13)$$

Logit của student được chuẩn hóa theo danh sách ứng viên:

$$\tilde{s}_i = \frac{s_i^S - \mu_s}{\sigma_s + \epsilon} \quad (3.14)$$

$$\mathcal{L}_{\text{ADR}} = \frac{1}{n} \sum_{i=1}^n (\tilde{s}_i - \tilde{r}_i)^2 \quad (3.15)$$

Vị trí thứ hạng là tín hiệu thứ tự, không phụ thuộc vào thang điểm tuyệt đối của teacher. Phép ánh xạ tạo mục tiêu bị chặn và cách đều; ADR-MSE gán trọng số bằng nhau cho các vị trí trong phiên bản cơ sở.

Table 3.9: Ví dụ minh họa cách tính ADR-MSE với năm ứng viên

Ứng viên	Hạng r_i	Mục tiêu $\tilde{r}_i$	Logit s_i	Điểm $\tilde{s}_i$	Thành phần loss
Doc A (positive)	0	+1.00	3.2	+1.455	0.207
Doc B	1	+0.50	1.8	+0.712	0.045
Doc C	2	0.00	0.3	-0.085	0.007
Doc D	3	-0.50	-0.9	-0.722	0.049
Doc E	4	-1.00	-2.1	-1.359	0.129

Với giá trị trung bình logit $\mu_s = 0.460$ và độ lệch chuẩn $\sigma_s = 1.883$, ví dụ trên cho:

$$\mathcal{L}_{\text{ADR}} \approx \frac{0.207 + 0.045 + 0.007 + 0.049 + 0.129}{5} \approx 0.088 \quad (3.16)$$

Table 3.10: Siêu tham số của giai đoạn B

Tham số	Giá trị
Trọng số chung cắt α	0.7
Tốc độ học	1×10^{-5}
Kích thước batch	8 câu hỏi, 20 ứng viên/câu
Nhiệt độ Listwise KL τ	2.0
Số epoch	5
Dừng sớm	Patience 2 theo MRR@10 trên tập kiểm định

3.8.5 Cắt tỉa từ vựng cho mmarco

Bên cạnh MiniLM 33M và mmarco 117M, đồ án thử nén mmarco bằng từ vựng cắt tỉa. Động cơ xuất phát từ cấu trúc tham số của mmarco-mMiniLMv2: model dùng từ vựng XLM-R gồm 250,002 tokens cho 100 ngôn ngữ, nên riêng embedding matrix đã chiếm $250,002 \times 384 \approx 96\text{M}$ tham số, tương đương hơn 80% tổng số tham số (117.6M). Trong khi đó, corpus của đồ án chỉ gồm tiếng Việt và tiếng Anh kỹ thuật, sử dụng một phần nhỏ từ vựng này; phần lớn token của các ngôn ngữ khác không bao giờ xuất hiện.

Ý tưởng là cắt embedding matrix theo các bước sau, giữ nguyên transformer encoder:

1. Thu thập toàn bộ tokens thực sự xuất hiện trong corpus (2,317 đoạn) và queries (991 câu).
2. Giữ lại: các special tokens ([CLS], [SEP], [PAD], [UNK]), toàn bộ tokens xuất hiện trong dữ liệu, cùng 30,000 tokens phổ biến nhất của từ vựng gốc làm dự phòng cho tài liệu mới.

3. Tổng cộng giữ 36,324 trong 250,002 tokens (loại 85.5%).
4. Slice embedding matrix theo các token được giữ, remap token IDs, và rebuild tokenizer với từ vựng Unigram mới.

Cách này giảm tổng số tham số từ 117.6M xuống 35.6M, tức giảm 69.7% dung lượng. Trên corpus mới, UNK rate chỉ 0.225%, cho thấy việc loại 85.5% từ vựng gần như không làm mất token tiếng Việt quan trọng. Vì transformer encoder, phần đảm nhiệm gần như toàn bộ chi phí tính toán được giữ nguyên, độ trễ không giảm tỷ lệ thuận với số tham số: thao tác tra cứu embedding là $O(1)$, nên cắt embedding matrix chỉ giảm bộ nhớ chứ không giảm thời gian suy luận. Lợi ích chính của cắt tia vì vậy là giảm dung lượng model và yêu cầu bộ nhớ (phù hợp triển khai trên thiết bị hạn chế tài nguyên), đồng thời vẫn giữ nguyên năng lực đa ngôn ngữ của encoder mmarco. Đánh giá chất lượng chi tiết được trình bày ở Chương 4 (mục 4.3.5).

3.9 Tinh chỉnh bộ phân loại ý định Sentence-BERT

3.9.1 Kiến trúc và dữ liệu

Bộ phân loại ý định v2 được xây dựng để phân biệt hai lớp: `tra_cuu`, tương ứng với các câu hỏi yêu cầu tra cứu thông tin có sẵn trong tài liệu, và `ting_toan`, tương ứng với các câu hỏi yêu cầu suy luận hoặc tính toán để tạo ra kết quả mới. Backbone của bộ phân loại là mô hình Sentence-BERT tiếng Việt keepitreal/vietnamese-sbert [25], dựa trên PhoBERT-base, với embedding đầu ra 768 chiều.

Thay vì gắn một head tuyến tính lên token [CLS] và tinh chỉnh toàn bộ mô hình bằng cross-entropy, đồ án sử dụng phương pháp SetFit. Cụ thể, encoder được tinh chỉnh bằng contrastive learning trên các cặp câu hỏi, sau đó một head Logistic Regression nhẹ được huấn luyện trên embedding câu. Trước khi đưa vào mô hình, câu hỏi được tách từ bằng Pyvi do các mô hình dựa trên PhoBERT yêu cầu đầu vào đã được tách từ. Sau đó, câu hỏi được encode thành embedding bằng mean pooling và chuẩn hóa L2:

$$\mathbf{e}_q = \text{MeanPool}(\text{SBERT}(\text{segment}(q))) \in \mathbb{R}^{768}, \quad \|\mathbf{e}_q\|_2 = 1 \quad (3.17)$$

$$\hat{y} = \text{LogisticRegression}(\mathbf{e}_q) \in \{\text{tra_cuu}, \text{ting_toan}\} \quad (3.18)$$

Trong giai đoạn contrastive fine-tuning, encoder được huấn luyện bằng CosineSimilarityLoss trên các cặp câu. Các cặp cùng ý định được gán target tương đồng bằng 1, trong khi các cặp khác ý định được gán target bằng 0. Cơ chế này

giúp kéo các câu hỏi cùng ý định lại gần nhau và đẩy các câu hỏi khác ý định ra xa trong không gian embedding.

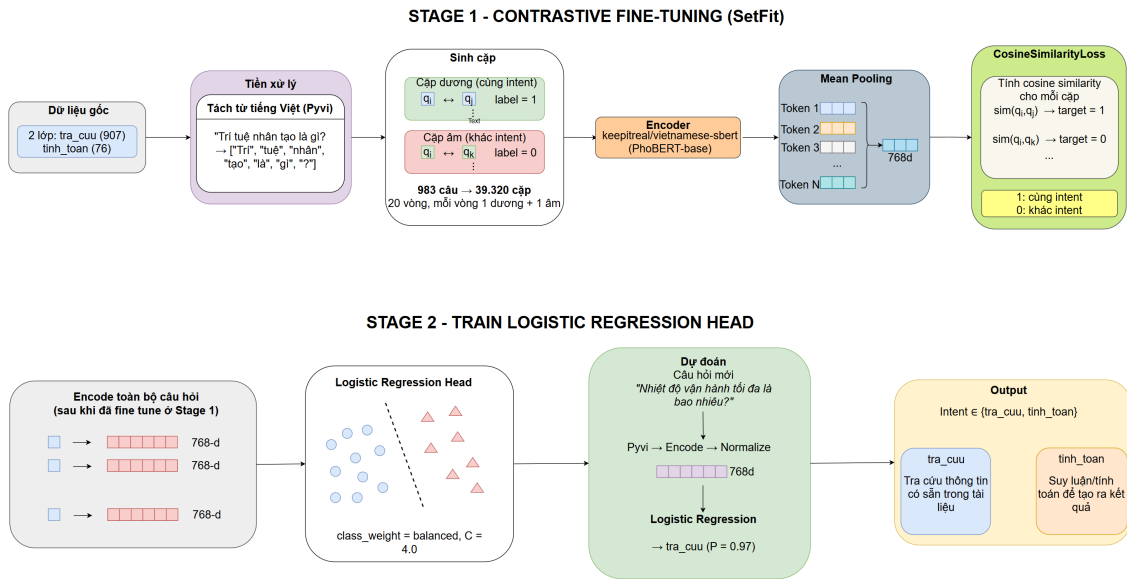


Figure 3.7: Kiến trúc huấn luyện bộ phân loại ý định theo SetFit

Việc lựa chọn SetFit thay cho kiến trúc [CLS] + Linear xuất phát từ quan sát rằng embedding chưa tinh chỉnh không phân biệt tốt hai lớp: mô hình dễ nhầm các câu tra_cuu có chứa số liệu thành tinh_toan, ví dụ câu hỏi “Hằng số điện môi ε_0 có giá trị bao nhiêu?” — thực chất là tra cứu một giá trị có sẵn, không phải yêu cầu tính toán. Ranh giới giữa “thông tin có sẵn” và “kết quả cần tính toán” không được biểu diễn đủ tốt nếu chỉ huấn luyện head phân loại trên embedding đông cứng. Do đó, đề án tinh chỉnh encoder bằng contrastive learning để học biểu diễn phù hợp hơn cho bài toán. So sánh định lượng giữa hai phương án được trình bày ở Chương 4.

Để kiểm tra chất lượng nhãn, đề án phân tích các cặp câu gần giống nhưng khác nhãn, với ngưỡng cosine similarity lớn hơn 0.8. Kết quả chỉ phát hiện một trường hợp mâu thuẫn nhãn thực sự trên toàn bộ tập dữ liệu, cho thấy dữ liệu có độ nhất quán cao và lỗi phân loại chủ yếu đến từ hạn chế biểu diễn của mô hình, không phải do nhiễu nhãn.

Tập dữ liệu ban đầu gồm 991 câu hỏi được gán nhãn thủ công; sau khi loại bỏ trùng lặp còn 983 câu. Do lớp tinh_toan là lớp thiểu số, một tập kiểm thử cố định có thể chứa quá ít mẫu tính toán để phản ánh ổn định chất lượng mô hình. Vì vậy, đề án sử dụng stratified 5-fold cross-validation để đánh giá. Mô hình triển khai cuối cùng được huấn luyện lại trên toàn bộ tập dữ liệu sạch. Ví dụ, câu hỏi “Theo tài liệu Public_021, nhiệt độ vận hành tối đa là bao nhiêu?” thuộc lớp tra_cuu, trong khi câu hỏi yêu cầu tính chi phí nhiên liệu từ quãng đường, mức tiêu hao và

giá xăng thuộc lớp `tin_h_toan`.

3.9.2 Xử lý mất cân bằng lớp

`tra_cuu` chiếm 907/983 câu (khoảng 92.3%), còn `tin_h_toan` chỉ có 76/983 câu (khoảng 7.7%), tỉ lệ xấp xỉ 12:1. Một classifier luôn dự đoán lớp đa số đã đạt accuracy khoảng 92%, do đó accuracy là metric dễ gây hiểu nhầm; F1-macro được dùng để đánh giá bình đẳng hai lớp. Mất cân bằng được xử lý ở hai tầng.

Lấy mẫu tăng cường cặp tương phản.

Khi sinh cặp huấn luyện cho SetFit, các cặp được lấy mẫu cân bằng theo lớp (oversampling) thay vì giữ tỉ lệ tự nhiên 92/8, để lớp thiểu số xuất hiện đủ trong cả cặp dương lẫn cặp âm. Mỗi anchor sinh một cặp dương và một cặp âm, nên tổng số cặp xấp xỉ $2 \times T \times N$, với T là số vòng lặp sinh cặp.

Trọng số lớp ở bộ phân loại.

Bộ phân loại Logistic Regression dùng trọng số lớp cân bằng (balanced): trọng số mỗi lớp tỉ lệ nghịch với tần suất, $w_c = n/(K \cdot n_c)$ với $K = 2$ lớp:

$$w_{\text{tra_cuu}} = \frac{983}{2 \times 907} \approx 0.54, \quad w_{\text{tin_h_toan}} = \frac{983}{2 \times 76} \approx 6.47 \quad (3.19)$$

Lỗi trên lớp `tin_h_toan` vì vậy bị phạt mạnh hơn khoảng 12 lần.

Table 3.11: Siêu tham số tinh chỉnh Sentence-BERT (SetFit) cho bộ phân loại ý định

Tham số	Giá trị
Backbone	keepitreal/vietnamese-sbert (PhoBERT-base)
Contrastive loss	CosineSimilarityLoss
Số vòng lặp sinh cặp	20 (sinh $\approx 2 \cdot 20 \cdot N$ cặp)
Chiến lược lấy mẫu	oversampling (lấy mẫu tăng cường)
Epochs	1
Batch size	16
Learning rate	2×10^{-5}
Warmup ratio	0.1
Mixed precision	FP16
Optimizer	AdamW
Bộ phân loại	Logistic Regression, trọng số lớp cân bằng, $C=4.0$

Đánh giá theo F1-macro (qua stratified 5-fold cross-validation) đảm bảo không tối ưu nhằm vào accuracy tổng thể khi recall của lớp thiểu số còn thấp. Kết quả định lượng (F1-macro của SetFit so với baseline embedding đông cứng, và so sánh

các backbone PhoBERT-base) được trình bày ở Chương 4.

3.9.3 Logic định tuyến trong pipeline v2

Trong v1, query được gửi tới GPT-4o-mini qua API để nhận ý định label. Trong v2, query được tách từ và chạy qua Sentence-BERT local; logic sau classification không đổi: `tra_cuu` dùng direct prompt, còn `tinh_toan` dùng Chain-of-Thought prompt. Regex trích xuất Public ID chạy song song với classifier.

Sentence-BERT local loại bỏ phụ thuộc vào external API và giữ dữ liệu câu hỏi trong hạ tầng nội bộ, phù hợp với yêu cầu bảo mật của môi trường doanh nghiệp. Các số liệu định lượng về ý định accuracy và độ trễ định tuyến được trình bày ở Chương 4 trên cùng bộ 390 câu MCQ.

CHƯƠNG 4. ĐÁNH GIÁ THỰC NGHIỆM

4.1 Thiết lập thực nghiệm

4.1.1 Môi trường và phần cứng

Toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên hai môi trường. Huấn luyện reranker và embedding sử dụng GPU NVIDIA RTX 6000, cấu hình đủ để duy trì batch size lớn cho MNRL và chạy teacher BGE-reranker-v2-m3 song song với student trong giai đoạn B.

Các phép đo xếp hạng và pipeline độ trễ mới trong chương này được thực hiện trên RTX 6000 để so sánh các cấu hình production trong cùng một môi trường. Riêng phép đo đầu cuối MCQ được thực hiện trong môi trường CPU/API trên máy cá nhân, dùng để so sánh ảnh hưởng tổng thể của pipeline v1 và v2 tới độ chính xác và độ trễ.

4.1.2 Tập dữ liệu đánh giá

Đồ án sử dụng hai tập đánh giá chính cho truy hồi và xếp hạng. D1 là tập in-domain gồm 158 queries trên corpus 2,317 đoạn, dùng để kiểm tra chất lượng embedding trên miền huấn luyện. D2 là tập độc lập trên 50 PDF kỹ thuật tiếng Việt chưa dùng khi huấn luyện, gồm 390 câu trên corpus 813 đoạn. Cả embedding và reranker đều được đánh giá trên cùng D1/D2, đảm bảo nhất quán giữa các mục.

Table 4.1: Các tập dữ liệu dùng trong đánh giá

Tập	Quy mô	Mục đích
D1 in-domain	158 câu, 2,317 đoạn	Đánh giá embedding trên tài liệu thuộc miền huấn luyện
D2 độc lập	390 câu, 813 đoạn	Đánh giá zero-shot cho cả embedding và reranker, tập test chung
Dev reranker	305 triplets, 50 câu	Theo dõi và lựa chọn checkpoint trong quá trình huấn luyện

Tập D2 (390 queries, 50 PDF) hoàn toàn tách biệt với tập phát triển v1 (991 câu, 200 PDF), không có tài liệu nào trùng lặp. Đây là điều kiện zero-shot: model chưa từng thấy bất kỳ đoạn nào từ 50 PDF này trong quá trình huấn luyện. Kết quả trên tập này do đó phản ánh khả năng tổng quát hóa của model, không phải học thuộc dữ liệu.

4.1.3 Độ đo đánh giá

Đánh giá reranker sử dụng bốn metrics tiêu chuẩn trong truy hồi thông tin: Hit@1 (tỉ lệ gold đoạn xuất hiện ở rank 1), Recall@5 (tỉ lệ gold đoạn xuất hiện trong top-5), MRR@10 (Mean Reciprocal Rank, đo vị trí trung bình của gold đoạn) và NDCG@10 (Normalized Discounted Cumulative Gain, đo chất lượng toàn bộ thứ tự xếp hạng). Metric chính để so sánh là MRR@10 và Recall@5 vì chúng phản ánh trực tiếp hiệu quả của reranker trong pipeline RAG thực tế, nơi top-5 đoạn được đưa vào LLM.

Đánh giá embedding.

Các metrics gồm Acc@1, Acc@5, MRR@10 và NDCG@10 cho embedding. D1 (in-domain) gồm 158 câu trên corpus 2,317 đoạn dùng làm benchmark phát triển chính. D2 (độc lập) gồm 390 câu trên corpus 813 đoạn của 50 tài liệu mới, đồng thời là tập test chung với reranker.

Đánh giá đầu cuối sử dụng accuracy trên 390 câu hỏi trắc nghiệm (exact match với đáp án A/B/C/D), kết hợp đo độ trễ (ms/câu) và so sánh trực tiếp pipeline v1 với v2.

4.2 Đánh giá embedding

4.2.1 Thiết lập đánh giá

Embedding được đánh giá trên hai bộ test. D1 (in-domain) gồm 158 queries trên corpus 2,317 đoạn của các tài liệu thuộc miền huấn luyện, với câu hỏi tách riêng và không dùng khi train. D2 (độc lập) gồm 390 queries trên 50 tài liệu độc lập (corpus 813 đoạn) — đây là cùng bộ 390 câu dùng cho đánh giá reranker (mục 4.1.2). D1 đánh giá truy hồi in-domain; D2 đánh giá zero-shot trên tài liệu chưa từng thấy.

Table 4.2: Các tập dữ liệu đánh giá embedding

Tập test	Queries	Corpus	Nguồn tài liệu	Cách sinh câu hỏi
D1 (in-domain)	158	2,317 đoạn	Miền huấn luyện (200 docs)	Tách từ tập câu hỏi, không dùng khi train
D2 (độc lập)	390	813 đoạn	Độc lập (50 docs)	Tập test chung với reranker

4.2.2 Kết quả trên hai tập kiểm thử

Table 4.3: Kết quả trên tập D1 (158 câu)

Model	Dim	Acc@1	Acc@5	MRR@10	NDCG@10
BGE-M3 raw	1024	70.89%	92.41%	0.8015	0.8207
Vietnamese_Embedding_v1 raw	1024	65.82%	93.04%	0.7718	0.7753
Vietnamese_Embedding_v2 raw (base)	1024	65.19%	91.77%	0.7694	0.7800
Fine-tuned	256	62.45 ± 1.08%	89.45 ± 0.79%	0.7489 ± 0.0055	0.7646 ± 0.0021
Fine-tuned	512	70.46 ± 0.30%	93.67 ± 0.00%	0.8028 ± 0.0020	0.8149 ± 0.0016
Fine-tuned	1024	73.21 ± 0.60%	94.73 ± 0.30%	0.8220 ± 0.0037	0.8269 ± 0.0031

Table 4.4: Kết quả trên tập D2 (390 câu)

Model	Dim	Acc@1	Acc@5	MRR@10	NDCG@10
BGE-M3 raw	1024	67.95%	91.03%	0.7752	0.8158
Vietnamese_Embedding_v1 raw	1024	70.26%	91.28%	0.7909	0.8338
Vietnamese_Embedding_v2 raw (base)	1024	70.77%	91.54%	0.7951	0.8315
Fine-tuned	256	68.38 ± 0.12%	88.29 ± 0.32%	0.7711 ± 0.0015	0.8099 ± 0.0016
Fine-tuned	512	71.79 ± 0.42%	91.54 ± 0.21%	0.7982 ± 0.0017	0.8339 ± 0.0009
Fine-tuned	1024	71.45 ± 0.12%	93.42 ± 0.44%	0.8059 ± 0.0002	0.8427 ± 0.0004

Về cách báo cáo số liệu. Mọi mô hình trong hai bảng trên được đánh giá ở cùng một thiết lập: độ dài chuỗi tối đa 1024 token, khớp với cấu hình huấn luyện của mô hình tinh chỉnh (Bảng 3.4) và đủ để bao trọn đoạn dài nhất trong corpus (923 token). Cột *Dim* là số chiều vector đầu ra sau khi cắt theo Matryoshka — một tham số độc lập với độ dài chuỗi đầu vào. Mô hình nền là suy luận thuần túy, không có thành phần ngẫu nhiên nên chỉ cần một lần đo; mô hình tinh chỉnh phụ thuộc vào hạt giống ngẫu nhiên (thứ tự trộn batch, và do đó thành phần in-batch negative của MNRL) nên được huấn luyện lại với ba hạt giống ngẫu nhiên {42, 123, 7} và báo cáo dưới dạng trung bình ± độ lệch chuẩn.

So sánh với mô hình nền. Trên D1, mô hình tinh chỉnh ở 1024d đạt $73.21 \pm 0.60\%$ Acc@1, vượt base model (Vietnamese_Embedding_v2 raw, 65.19%) 8.02 điểm và vượt mô hình nền tốt nhất (BGE-M3 raw, 70.89%) 2.32 điểm. Trên D2, khoảng cách hẹp hơn nhiều: $71.45 \pm 0.12\%$ so với 70.77% của base (+0.68 điểm). Chênh lệch giữa hai tập cho thấy phần lớn lợi ích đến từ việc thích nghi với đúng miền huấn luyện; trên tài liệu hoàn toàn mới, cải thiện nhỏ nhưng ổn định (độ lệch chuẩn chỉ 0.12 điểm). Mô hình tinh chỉnh ở 512d vẫn vượt base model trên cả hai tập, xác nhận việc cắt vector qua Matryoshka không làm mất lợi ích của tinh chỉnh.

Một lưu ý về độ nhạy với độ dài chuỗi: các mô hình nền, vốn không được huấn

luyện trên corpus này, cho kết quả cao hơn trên D1 khi chỉ mã hoá 512 token đầu của mỗi đoạn thay vì đọc trọn đoạn (Vietnamese_Embedding_v2 raw đạt 67.72% thay vì 65.19%). Nếu so với cấu hình thuận lợi nhất này của base model, mức cải thiện của mô hình tinh chỉnh là +5.49 điểm — vẫn đáng kể, và là con số thận trọng hơn. Hiện tượng không xuất hiện trên D2, nơi các đoạn phần lớn ngắn hơn 512 token.

Trên D2, cấu hình 512d đạt $\text{Acc}@1$ cao nhất ($71.79 \pm 0.42\%$), nhỉnh hơn 1024d ($71.45 \pm 0.12\%$), trong khi 1024d vẫn dẫn đầu ở $\text{Acc}@5$, $\text{MRR}@10$ và $\text{NDCG}@10$. Vì chênh lệch $\text{Acc}@1$ (0.34 điểm) nhỏ hơn tổng độ lệch chuẩn của hai cấu hình, khác biệt này nằm trong vùng dao động ngẫu nhiên và không nên được diễn giải như ưu thế thực chất của 512d.

4.2.3 Phân tích số chiều biểu diễn MRL

Table 4.5: So sánh 512d và 1024d trên D1 và D2

Test	Dim	Acc@1	MRR@10	NDCG@10
D1	256	$62.45 \pm 1.08\%$	0.7489 ± 0.0055	0.7646 ± 0.0021
D1	512	$70.46 \pm 0.30\%$	0.8028 ± 0.0020	0.8149 ± 0.0016
D1	1024	$73.21 \pm 0.60\%$	0.8220 ± 0.0037	0.8269 ± 0.0031
D2	256	$68.38 \pm 0.12\%$	0.7711 ± 0.0015	0.8099 ± 0.0016
D2	512	$71.79 \pm 0.42\%$	0.7982 ± 0.0017	0.8339 ± 0.0009
D2	1024	$71.45 \pm 0.12\%$	0.8059 ± 0.0002	0.8427 ± 0.0004

Số liệu ở ba số chiều cho thấy một đường cong đánh đổi rõ ràng, không tuyến tính. Từ 1024d xuống 512d, chất lượng giảm nhẹ: -2.75 điểm $\text{Acc}@1$ và -0.0192 $\text{MRR}@10$ trên D1; trên D2, $\text{Acc}@1$ thậm chí nhỉnh hơn (chênh lệch nằm trong dao động ngẫu nhiên) và $\text{MRR}@10$ chỉ giảm 0.0077. Nhưng từ 512d xuống 256d, chất lượng sụt mạnh: -8.01 điểm $\text{Acc}@1$ trên D1 và -3.41 điểm trên D2. Nói cách khác, nén một nửa số chiều ($1024 \rightarrow 512$) gần như miễn phí, còn nén tiếp một nửa nữa ($512 \rightarrow 256$) thì rất tốn kém.

Vì vậy, đồ án chọn 512d làm cấu hình triển khai chính (mục 3.7.5). So với 1024d, cấu hình này giảm một nửa dung lượng chỉ mục, trong khi $\text{MRR}@10$ chỉ giảm khoảng 2,3% tương đối trên D1 và 1,0% trên D2. Đây là đánh đổi phù hợp cho hệ thống chạy trên tài nguyên hạn chế.

Ngược lại, 256d không phải lựa chọn triển khai hợp lý cho bài toán này: mức mất mát chất lượng quá lớn so với phần dung lượng tiết kiệm thêm. Vai trò của 256d trong Matryoshka-Loss chủ yếu là một ràng buộc chính quy hoá, buộc mô hình dồn thông tin quan trọng về các chiều đầu, chứ không phải một chế độ triển khai thực tế.

4.2.4 Thử nghiệm mở rộng coverage bằng query tổng hợp

Phần lớn corpus không có câu hỏi gán sẵn: trong 2,317 đoạn, chỉ 634 câu hỏi huấn luyện phủ một phần corpus, còn 1,412 đoạn chưa từng xuất hiện trong bất kỳ cặp (câu hỏi, positive) nào. Đồ án thử nghiệm mở rộng độ phủ bằng cách sinh thêm câu hỏi cho các đoạn còn trống, dùng một mô hình ngôn ngữ để tạo câu hỏi tự nhiên cho từng đoạn, sau đó lọc qua ba lớp kiểm tra độc lập: loại các đoạn không đủ chất lượng (quá ngắn, hoặc phần lớn là số/ký hiệu) trước khi sinh; loại câu hỏi sao chép nguyên văn cụm từ dài từ đoạn gốc; và kiểm tra bằng truy hồi BGE-M3, chỉ giữ câu hỏi nếu đoạn gốc thực sự nằm trong top-10 kết quả truy hồi bằng chính câu hỏi đó. Sau lọc, 1,966 cặp câu hỏi–positive tổng hợp được thêm vào 1,180 cặp gốc (with_gen), so với chỉ dùng 1,180 cặp gốc (baseline).

Table 4.6: So sánh baseline và with_gen qua 3 seed (Acc@1, %)

Test	Dim	baseline	with_gen	Δ	p	Kết luận
D1	256	62.45 $\pm$ 1.08	65.61 $\pm$ 1.08	+3.16	0.043	with_gen tốt hơn
D1	512	70.46 $\pm$ 0.30	70.04 $\pm$ 0.79	−0.42	0.541	không ý nghĩa
D1	1024	73.21 $\pm$ 0.60	71.52 $\pm$ 0.52	−1.69	0.041	with_gen kém hơn
D2	256	68.38 $\pm$ 0.12	69.15 $\pm$ 1.03	+0.77	0.401	không ý nghĩa
D2	512	71.79 $\pm$ 0.42	69.66 $\pm$ 0.85	−2.13	0.052	sát ngưỡng
D2	1024	71.45 $\pm$ 0.12	69.74 $\pm$ 0.91	−1.71	0.115	không ý nghĩa

Ghi chú: mean $\pm$ sample std qua 3 seed (42, 123, 7); p-value từ Welch’s t-test hai mẫu độc lập.

Trên dim 1024 (cấu hình đo chất lượng cao nhất), thêm câu hỏi tổng hợp làm giảm Acc@1 trên D1 có ý nghĩa thống kê (Welch’s t-test, $p = 0.041$, $n = 3$ seed); trên D2, xu hướng giảm cùng chiều với biên độ tương đương (−1.71 điểm) nhưng chưa đạt ngưỡng quy ước ($p = 0.115$), một phần do phương sai cao hơn của cấu hình with_gen trên tập này. Ở dim 256, chiều hướng đảo ngược: with_gen nhỉnh hơn trên cả D1 ($p = 0.043$) và D2 (không ý nghĩa). Ở dim 512 — cấu hình triển khai chính — cả hai kết quả đều không đạt ý nghĩa thống kê.

Cần lưu ý hai giới hạn khi diễn giải bảng trên. Thứ nhất, với $n = 3$ seed mỗi cấu hình, ước lượng phương sai còn nhiều bất định; đây là xu hướng nhất quán hơn là kiểm định chặt chẽ. Thứ hai, sáu phép so sánh được thực hiện trên cùng một

cặp tập test, nên cần hiệu chỉnh cho multiple comparisons: với ngưỡng Bonferroni ($\alpha = 0.05/6 \approx 0.0083$), không phép so sánh nào trong bảng đạt ý nghĩa thống kê, kể cả hai trường hợp $p = 0.041$ và $p = 0.043$. Vì vậy, phát hiện đảo chiều ở dim 256 không nên được nhấn mạnh như một hiệu ứng đã xác nhận chắc chắn, đặc biệt vì cấu hình triển khai chính của đồ án là 512 chiều, nơi cả hai phương án không khác biệt rõ rệt.

Kết hợp với các kết quả không cải thiện khác đã thử trong quá trình phát triển v2, đồ án chọn giữ nguyên cấu hình đơn giản nhất: chỉ huấn luyện trên cặp (câu hỏi, positive) thật, không mở rộng bằng câu hỏi tổng hợp. Với mô hình nền đã được huấn luyện quy mô lớn và dữ liệu miền tương đối nhỏ, việc thêm dữ liệu tổng hợp có nguy cơ pha loãng tín hiệu từ các cặp thật hơn là bổ sung thông tin hữu ích.

4.3 Đánh giá reranker

4.3.1 Kết quả chính trên tập kiểm thử reranker

Tập test reranker là bộ mở rộng 390 câu zero-shot (mục 4.1.2). Candidate pool được cố định là top-20 đoạn được truy xuất từ toàn bộ corpus 813 đoạn bằng embedding đã được tinh chỉnh. Việc cố định pool dense là một lựa chọn có chủ đích: khi mọi reranker cùng xếp hạng lại trên một pool đầu vào giống nhau, chênh lệch metric giữa chúng phản ánh đúng năng lực xếp hạng của từng mô hình, không lẫn với dao động của bước hợp nhất (fusion) ở tầng truy hồi. Pipeline triển khai vẫn dùng truy hồi lai (hybrid) như mô tả ở Chương 3. Khi chưa rerank, riêng embedding đạt $\text{MRR}@10 = 0.7961$ trên candidate pool top-20 của bộ 390 câu, đây là điểm tham chiếu để đánh giá đóng góp của từng reranker.

Các model trong Bảng 4.7 bao phủ hai nhóm so sánh.

BGE-reranker-v2-m3 là teacher và là mốc chất lượng mạnh nhất của v1.

ViRanker [26] là cross-encoder reranker tiếng Việt công khai. Baseline mmarco base là mô hình đa ngôn ngữ đã học xếp hạng passage từ mMARCO, dùng để tách đóng góp của pretrained model khỏi đóng góp của domain distillation. Bảng 4.7 so sánh teacher, các student sau distillation và các baseline này trên cùng candidate pool.

Table 4.7: Kết quả reranker trên bộ 390 câu hỏi

Model	Params	Hit@1	Recall@5	MRR@10	NDCG@10
BGE-reranker-v2-m3 teacher	568M	0.8256	0.9615	0.8862	0.9068
mmarco-mMiniLMv2 (H384) + ADR-MSE	117M	0.8325 ± 0.0029	0.9615 ± 0.0000	0.8865 ± 0.0008	0.9073 ± 0.0006
Pruned H384 + ADR-MSE	35.6M	0.8291 ± 0.0015	0.9615 ± 0.0026	0.8847 ± 0.0006	0.9051 ± 0.0006
ViRanker SOTA VI	568M	0.7872	0.9564	0.8593	0.8844
mmarco-mMiniLMv2 (H384) base	117M	0.7615	0.9410	0.8442	0.8732
Pruned H384 base	35.6M	0.7564	0.9385	0.8423	0.8710

Ghi chú: Metrics của H384 và Pruned H384 báo cáo trung bình $\pm$ độ lệch chuẩn trên 3 hạt giống {42, 123, 7}

Student mmarco-mMiniLMv2 (H384) 117M sau distillation đạt $\text{MRR@10} = 0.8862$, ngang bằng teacher BGE-reranker-v2-m3 trên tập đánh giá này, trong khi số tham số nhỏ hơn khoảng 4.8 lần. Từ vệt cắt tia tạo ra phiên bản pruned 35.6M gần như không làm suy giảm chất lượng: pruned đạt $\text{MRR@10} = 0.8844$, bằng 99.8% so với mmarco 117M và so với teacher.

So với ViRanker, pruned 35.6M cao hơn 0.0251 điểm MRR@10 dù có số tham số nhỏ hơn khoảng 16 lần. Kết quả này cho thấy dữ liệu đúng miền và quá trình distillation phù hợp có thể quan trọng hơn việc chỉ tăng kích thước mô hình hoặc sử dụng dữ liệu tổng quát quy mô lớn.

4.3.2 Nghiên cứu loại bỏ thành phần

Table 4.8: So sánh cấu hình huấn luyện trên MiniLM, H384 và pruned H384

Cấu hình	Hit@1	Recall@5	MRR@10	NDCG@10
<i>MiniLM 33M (English-only), Listwise KL</i>				
Không giai đoạn A, $\alpha = 1.0$	0.6821	0.9179	0.7855	0.8222
Có giai đoạn A, $\alpha = 1.0$	0.6872	0.9205	0.7860	0.8216
Có giai đoạn A, $\alpha = 0.7$	0.6949	0.9231	0.7920	0.8272
<i>H384 / pruned H384 (đã ngôn ngữ), ADR-MSE, $\alpha = 0.7$</i>				
H384, có giai đoạn A	0.8256	0.9615	0.8837	0.9050
H384, chỉ giai đoạn B	0.8308	0.9615	0.8862	0.9069
Pruned H384, chỉ giai đoạn B	0.8282	0.9590	0.8844	0.9051

Ghi chú: các cấu hình H384 và pruned H384 dùng cho triển khai được huấn luyện không qua giai đoạn A — base model được đưa trực tiếp vào giai đoạn B. Dòng “H384, có giai đoạn A” là thí nghiệm đối chứng ở mục 4.3.3.

Bảng 4.8 cho thấy ba nhóm hiện tượng.

Thứ nhất, khi cố định $\alpha = 1.0$ để cô lập đúng đóng góp của giai đoạn A, mức cải

thiện gần như bằng không: 0.7855 (không giai đoạn A) so với 0.7860 (có giai đoạn A), tức $+0.0005$ MRR@10 — tương đương khoảng 0.2 câu hỏi trên tổng số 390. Giai đoạn A, hiểu như một bước huấn luyện tuần tự đặt trước chung cất, không đóng góp đáng kể.

Thứ hai, toàn bộ lợi ích thực sự đến từ nhánh contrastive L_{CL} chạy song song với chung cất: giữ nguyên giai đoạn A và chỉ hạ α từ 1.0 xuống 0.7 cho $+0.0060$ MRR@10 (0.7860 $\rightarrow$ 0.7920). Điều đáng chú ý là hai cấu hình này dùng cùng một tập dữ liệu và cùng một hàm mất mát (BCE pointwise trên dữ liệu miền); khác biệt duy nhất là cách đưa tín hiệu vào quá trình huấn luyện — tuần tự hay đồng thời. Mục 4.3.3 phân tích hiện tượng này chi tiết trên kiến trúc H384.

Thứ ba, khoảng cách giữa hai backbone lớn hơn hẳn mọi hiệu ứng siêu tham số: MiniLM dù được huấn luyện đầy đủ cũng chỉ đạt $\text{MRR@10} = 0.7920$, thấp hơn cả baseline không rerank (0.7961, Bảng 4.15), trong khi H384 với cùng pipeline đạt 0.8862 — chênh lệch $+0.094$. Nguyên nhân chính là ms-marco-MiniLM-L12-v2 sử dụng backbone và tokenizer tiếng Anh, không được tiền huấn luyện phù hợp với tiếng Việt; cơ chế loại bỏ dấu của tokenizer còn làm mất thông tin thanh điệu trước khi dữ liệu được đưa vào mô hình. Vì vậy, hạn chế của MiniLM không thể được khắc phục hoàn toàn chỉ bằng thay đổi dữ liệu, hàm mất mát hoặc siêu tham số huấn luyện.

Table 4.9: So sánh các hàm mất mát chung cất trên MiniLM 33M (giai đoạn A no-mMARCO, giai đoạn B $\alpha = 0.7$)

Loss	Teacher signal	Hit@1	Recall@5	MRR@10	NDCG@10
Listwise KL	Score distribution	69.49%	92.31%	0.7920	0.8272
RankNet	Pairwise probabilities	67.18%	91.79%	0.7747	0.8122
ADR-MSE	Rank positions	66.67%	91.54%	0.7734	0.8108
Margin-MSE	Score margins	62.05%	88.97%	0.7360	0.7811

Trên MiniLM 33M, Listwise KL đạt kết quả cao nhất trong nhóm loss được so sánh, với $\text{MRR@10} = 0.7920$ và $\text{NDCG@10} = 0.8272$. RankNet đứng thứ hai ($\text{MRR@10} = 0.7747$), nhỉnh hơn ADR-MSE rất nhỏ (0.7734). Margin-MSE thấp nhất vì phụ thuộc trực tiếp vào biên độ logit của teacher; khi teacher margin có thang điểm rộng, loss này dễ bị chi phối bởi các cặp dễ thay vì các hard negative kỹ thuật.

Tuy nhiên, thứ hạng này không cố định theo kiến trúc. Bảng 4.10 lặp lại đúng phép so sánh bốn loss này trên kiến trúc H384 (mmarco-mMiniLMv2), kiến trúc dùng cho các model triển khai chính.

Table 4.10: So sánh các hàm mất mát chung cất trên H384 (giai đoạn A, giai đoạn B $\alpha = 0.7$)

Loss	Teacher signal	Hit@1	Recall@5	MRR@10	NDCG@10
ADR-MSE	Rank positions	83.08%	96.15%	0.8862	0.9069
RankNet	Pairwise probabilities	82.31%	96.15%	0.8842	0.9050
Margin-MSE	Score margins	82.56%	96.41%	0.8832	0.9039
Listwise KL	Score distribution	82.05%	96.67%	0.8815	0.9024

Trên H384, ADR-MSE dẫn đầu cả bốn metric, dù biên độ chênh lệch giữa bốn loss rất hẹp (0.8815–0.8862 MRR@10, chênh lệch dưới 0.005). Kết hợp hai bảng, kết luận rút ra là: lựa chọn hàm mất mát ảnh hưởng rõ rệt hơn trên kiến trúc yếu (MiniLM 33M, English-only, cần thích nghi miền mạnh), nơi Listwise KL vượt trội các loss còn lại; trên kiến trúc mạnh sẵn có encoder đa ngôn ngữ tốt (H384), cả bốn loss hội tụ về chất lượng gần tương đương, và ADR-MSE là lựa chọn ổn định nhất trong nhóm này – đây là căn cứ cho việc chọn ADR-MSE cho các model triển khai chính (mmarco 117M và pruned 35.6M), vốn đều dựa trên kiến trúc H384.

4.3.3 Vai trò của giai đoạn A và hiện tượng quên thảm khốc

Kết quả trên MiniLM ở mục trước gợi ý rằng giai đoạn A không đóng góp, nhưng MiniLM là một kiến trúc yếu (thua cả baseline không rerank), nên không thể ngoại suy kết luận sang kiến trúc dùng cho triển khai. Đồ án vì vậy lặp lại thí nghiệm trực tiếp trên H384, giữ nguyên mọi siêu tham số khác (ADR-MSE, $\alpha = 0.7$, 5 epoch, learning rate 10^{-5} , seed 42) và chỉ thay đổi việc có hay không giai đoạn A.

Table 4.11: Vai trò của giai đoạn A trên H384 (ADR-MSE, $\alpha = 0.7$)

Cấu hình	Hit@1	Recall@5	MRR@10	NDCG@10
H384 base (zero-shot)	0.7615	0.9410	0.8442	0.8732
H384 chỉ giai đoạn A	0.8051	0.9462	0.8646	0.8847
H384 giai đoạn A + B	0.8256	0.9615	0.8837	0.9050
H384 chỉ giai đoạn B	0.8308	0.9615	0.8862	0.9069
Δ (có A – không A)	−0.0051	0.0000	−0.0025	−0.0018

Bảng 4.11 cho phép một suy luận phản chứng theo ba bước.

Bước 1 — giai đoạn A thực sự học được. So với base zero-shot, cấu hình chỉ có giai đoạn A nâng MRR@10 từ 0.8442 lên 0.8646 (+0.0204, tương đương khoảng 8 câu hỏi). Điều này cho thấy tập dữ liệu của giai đoạn A chứa tín hiệu học có giá trị và mô hình có khả năng khai thác tín hiệu đó.

Bước 2 — tín hiệu này không còn sau khi huấn luyện tuần tự. Nếu kiến thức học được ở giai đoạn A vẫn được bảo toàn sau giai đoạn B, thì cấu hình A+B phải đạt kết quả không thấp hơn cấu hình chỉ có B, vì nó khởi đầu từ một mô hình đã học thêm thông tin. Tuy nhiên, kết quả thực nghiệm lại ngược lại: A+B chỉ đạt 0.8837, thấp hơn cấu hình chỉ-B (0.8862). Điều này cho thấy phần cải thiện +0.0204 từ giai đoạn A hầu như đã bị quá trình chung cất ghi đè, phù hợp với hiện tượng *catastrophic forgetting*, khi quá trình tối ưu ở giai đoạn B làm mất các tham số đã được điều chỉnh ở giai đoạn A.

Bước 3 — tối ưu đồng thời giúp bảo toàn tín hiệu đó. Phương pháp đề xuất không thay đổi dữ liệu hay hàm mất mát của giai đoạn A: nhánh L_{CL} vẫn sử dụng cùng tập dữ liệu và cùng BCE pointwise. Điểm khác biệt là L_{CL} được tối ưu đồng thời với L_{KD} thông qua hàm mất mát tổng hợp $L = \alpha L_{KD} + (1 - \alpha)L_{CL}$, thay vì tối ưu tuần tự như A rồi mới đến B. Nếu giả thuyết về hiện tượng quên thảm khốc là đúng, thì việc duy trì ràng buộc từ L_{CL} trong suốt quá trình chung cất sẽ giúp bảo toàn tín hiệu mà giai đoạn A đã học, thay vì để nó bị ghi đè.

Kết luận. Thích nghi miền vẫn cần thiết, nhưng cách đưa tín hiệu vào mới là yếu tố quyết định: huấn luyện tuần tự khiến kiến thức bị chung cất ghi đè, còn huấn luyện đồng thời thì giữ được. Kết quả này nhất quán trên cả hai backbone — MiniLM (+0.0005) và H384 (−0.0025), cả hai đều nằm trong vùng nhiễu thống kê. Vì vậy các mô hình triển khai chính (mmarco 117M và pruned 35.6M) loại bỏ hoàn toàn giai đoạn A và chỉ dựa vào $\alpha < 1$ để thực hiện thích nghi miền.

4.3.4 Khảo sát trọng số α giữa chung cất và neo contrastive

Siêu tham số α trong $L = \alpha L_{KD} + (1 - \alpha)L_{CL}$ điều khiển tỉ lệ giữa tín hiệu xếp hạng từ teacher và tín hiệu phân biệt từ dữ liệu miền. Đồ án khảo sát bốn giá trị trên pruned H384 — kiến trúc dùng cho triển khai cuối và cũng là môi trường sạch nhất để đo α , vì cấu hình này không qua giai đoạn A, nên nhánh L_{CL} là nguồn tín hiệu miền duy nhất.

Table 4.12: Khảo sát α trên pruned H384 (ADR-MSE, không giai đoạn A)

α	w_{KD}	w_{CL}	Hit@1	Recall@5	MRR@10	NDCG@10
0.3	0.3	0.7	0.8128	0.9692	0.8780	0.8997
0.5	0.5	0.5	0.8205	0.9641	0.8815	0.9028
0.7	0.7	0.3	0.8282	0.9590	0.8844	0.9051
1.0	1.0	0.0	0.8205	0.9564	0.8796	0.9015

Hai phát hiện rút ra từ Bảng 4.12.

Phát hiện 1 — nhánh L_{CL} là nguyên nhân khiến student vượt teacher. Đây là điểm quan trọng nhất, vì nó giải thích một quan sát thoạt nhìn nghịch lý ở Bảng 4.7: student vượt teacher ở Hit@1 (0.8308 so với 0.8256). Nếu chỉ chung cất thuần túy, student giỏi nhất cũng chỉ bằng teacher. Khảo sát α cho thấy cơ chế thật:

- Teacher BGE: Hit@1 = 0.8256
- Student $\alpha = 1.0$ (chung cất thuần, không có tín hiệu miền): Hit@1 = 0.8205 thấp hơn teacher
- Student $\alpha = 0.7$ (chung cất + neo contrastive): Hit@1 = 0.8282 vượt teacher

Khi loại bỏ nhánh L_{CL} , student không vượt được teacher đúng như kỳ vọng lý thuyết, vì bắt chước teacher thì tốt nhất cũng chỉ bằng teacher. Chỉ khi bổ sung tín hiệu giám sát từ dữ liệu miền, thứ mà teacher không có, student mới vượt lên. Nói cách khác, student = năng lực xếp hạng của teacher + thích nghi miền, và chính về thứ hai tạo ra phần vượt trội.

Phát hiện 2 — tồn tại đánh đổi giữa recall và precision ở đỉnh. Recall@5 giảm đơn điệu khi α tăng (0.9692 $\rightarrow$ 0.9641 $\rightarrow$ 0.9590 $\rightarrow$ 0.9564), ngược chiều hoàn toàn với Hit@1, MRR@10 và NDCG@10. Điều này phản ánh bản chất hai tín hiệu: nhánh L_{CL} (BCE nhị phân “liên quan / không liên quan”) giúp mô hình không đánh rơi đoạn đúng khỏi top-5, tức tối ưu recall; còn nhánh L_{KD} (chung cất thứ hạng chi tiết từ teacher) giúp đẩy đúng đoạn tốt nhất lên hạng đầu, tức tối ưu precision ở đỉnh danh sách. Đề án chọn $\alpha = 0.7$ vì nó tối ưu MRR@10 và NDCG@10 — các độ đo phù hợp với ngữ cảnh RAG, nơi chất lượng của vài đoạn xếp hạng đầu quan trọng hơn việc bao phủ rộng.

Cần nói rõ về ý nghĩa thống kê: với $n = 390$, chênh lệch giữa các giá trị α liên kề (khoảng 0.003–0.008 MRR@10) tương đương một đến ba câu hỏi, nằm trong vùng nhiễu của một phép đo đơn lẻ. Điều thực sự thuyết phục không phải một cặp so sánh riêng lẻ, mà là tính đơn điệu nhất quán của cả đường cong: Hit@1, MRR@10 và NDCG@10 cùng đạt đỉnh tại $\alpha = 0.7$, trong khi Recall@5 giảm đều theo chiều ngược lại. Nhiều ngẫu nhiên khó tạo ra một cấu trúc nhất quán trên cả bốn độ đo như vậy.

4.3.5 Cắt tỉa từ vựng cho mmarco

Table 4.13: Nén Mmarco bằng cắt tỉa từ vựng

Thành phần	Trước cắt tỉa	Sau cắt tỉa	Giảm
Từ vựng	250.002 token	36,329 tokens	85.5%
Total params	117.6M	35.6M	69.7%
Transformer encoder	Giữ nguyên	Giữ nguyên	—

Pruning chỉ cắt embedding matrix và giữ nguyên transformer encoder. Tokens được giữ gồm special tokens, toàn bộ tokens xuất hiện trong corpus và queries, cùng 30,000 tokens phổ biến nhất để an toàn cho tài liệu mới. Trên corpus mới, UNK rate chỉ 0.225%, cho thấy cắt tỉa không làm mất token tiếng Việt quan trọng.

Table 4.14: Chất lượng pruned reranker sau tinh chỉnh

Model	Params	Hit@1	Recall@5	MRR@10	NDCG@10
BGE-reranker-v2-m3 teacher	568M	0.8256	0.9615	0.8862	0.9068
mmarco 117M đã tinh chỉnh	117M	0.8308	0.9615	0.8862	0.9069
Pruned 35.6M đã tinh chỉnh	35.6M	0.8282	0.9590	0.8844	0.9051

Pruned 35.6M giữ 99.8% MRR@10 của mmarco 117M (0.8844/0.8862) nhưng giảm memory dung lượng 3.3 lần. Độ trễ không giảm tương ứng vì transformer layers vẫn giữ nguyên; lợi ích chính của cắt tỉa là giảm tham số, dung lượng lưu trữ và yêu cầu bộ nhớ, không phải tốc độ.

4.3.6 Phân tích sự phụ thuộc của reranker vào độ mạnh của bộ truy hồi

Table 4.15: So sánh các reranker trên bộ truy hồi mạnh

Reranker	Params	Hit@1	Recall@5	MRR@10	NDCG@10	Delta MRR	Rerank ms
Không reranker	–	0.7077	0.9154	0.7943	0.8312	baseline	–
mmarco 117M base	117M	0.7615	0.9410	0.8442	0.8732	+0.0499	50.8
Pruned base	35.6M	0.7564	0.9385	0.8423	0.8710	+0.0480	51.0
ViRanker SOTA VI	568M	0.7872	0.9564	0.8593	0.8844	+0.0650	385.7
Pruned 35.6M đã tinh chỉnh	35.6M	0.8282	0.9590	0.8844	0.9051	+0.0901	51.0
mmarco 117M đã tinh chỉnh	117M	0.8308	0.9615	0.8862	0.9069	+0.0919	50.8
BGE-reranker-v2-m3 teacher	568M	0.8256	0.9615	0.8862	0.9068	+0.0919	385.5

Ghi chú: Rerank ms đo riêng bước xếp hạng lại top-20 candidate (max_len 512), không gồm thời gian truy hồi.

Với bộ truy hồi mạnh, mọi reranker từ 35.6M trở lên đều cải thiện rõ rệt so với không rerank. mmarco 117M đã tinh chỉnh đạt MRR@10 ngang bằng BGE-reranker-v2-m3 trên pipeline này (0.8862/0.8862) nhưng thời gian rerank chỉ 50.8ms, nhanh hơn BGE-reranker-v2-m3 khoảng 7.6 lần. Pruned 35.6M vượt ViRanker 568M cả về MRR (0.8844 so 0.8593) lẫn thời gian rerank (51.0ms so

385.7ms, nhanh khoảng 7.6 lần), xác nhận lợi ích của distillation trên dữ liệu đúng miền. Đáng chú ý, thời gian rerank của pruned 35.6M xấp xỉ mmarco 117M dù giảm 69.7% tham số: vì cắt tỉa chỉ cắt embedding matrix mà giữ nguyên encoder, lợi ích là tiết kiệm bộ nhớ chứ không phải tốc độ.

4.3.7 Khảo sát các kỹ thuật nén

Kết quả ở mục 4.3.5 đặt ra một câu hỏi: vì sao giảm 69.7% tham số mà độ trễ không đổi (50.8 $\rightarrow$ 51.0 ms)? Câu trả lời nằm ở cấu trúc chi phí tính toán của một lượt suy luận cross-encoder:

$$\text{Cost} \approx \underbrace{L}_{\text{số tầng}} \times \left[\underbrace{O(s^2 \cdot d)}_{\text{attention}} + \underbrace{O(s \cdot d^2)}_{\text{FFN}} \right] \quad (4.1)$$

với L là số tầng transformer, s là độ dài chuỗi và d là chiều ẩn. Với H384: $L = 12$, $s = 512$, $d = 384$.

Cắt tỉa từ vựng không tác động vào bất kỳ biến nào trong công thức (4.1). Bảng embedding là một phép tra cứu (lookup): nó chiếm bộ nhớ nhưng không phát sinh phép nhân ma trận. Toàn bộ FLOPs nằm ở 12 tầng transformer, vốn được giữ nguyên. Từ nhận định này, đề án khảo sát hai kỹ thuật nén khác, tấn công trực tiếp vào các thành phần của công thức (4.1).

a, Cắt tỉa tầng (giảm L)

Kỹ thuật này giữ lại 6 trong 12 tầng của pruned H384 (chọn các tầng chẵn 0, 2, 4, 6, 8, 10 theo công thức khởi tạo của DistilBERT), sau đó huấn luyện lại giai đoạn B với cùng cấu hình (ADR-MSE, $\alpha = 0.7$).

Table 4.16: Cắt tỉa tầng: L12 so với L6 trên pruned H384

Cấu hình	Tầng	Params	Dung lượng	Rerank (ms)	Hit@1	MRR @10
Pruned L12	12	35.6M	135.8 MB	50.4	0.8282	0.8844
Pruned L6	6	24.9M	95.2 MB	30.2	0.5769	0.7161
<i>Không rerank</i>	—	—	—	—	0.7077	0.7943

Chất lượng sụp đổ hoàn toàn: MRR@10 giảm từ 0.8844 xuống 0.7161, thấp hơn cả baseline không rerank (0.7943). Nguyên nhân nằm ở cơ chế residual stream của transformer: tầng $i + 1$ được huấn luyện để nhận đầu vào đã qua xử lý của tầng i . Khi loại bỏ tầng i , tầng $i + 1$ nhận một phân phối đầu vào hoàn toàn xa lạ, và 5 epoch tinh chỉnh không đủ để khôi phục. Để cắt tỉa tầng thành công, cần chung cắt theo tầng (layer-wise distillation — ép khớp hidden states và attention maps ở

từng tầng ngay trong quá trình huấn luyện, như TinyBERT hoặc DistilBERT thực hiện), chứ không thể cắt rồi tinh chỉnh dần.

Một quan sát phụ: độ trễ chỉ giảm 1.67 lần ($50.4 \rightarrow 30.2$ ms) chứ không phải 2 lần như kỳ vọng khi bỏ một nửa số tầng. Phần chênh lệch là chi phí cố định (tokenize, tra cứu embedding, khởi tạo kernel, truyền dữ liệu giữa CPU và GPU) không phụ thuộc vào FLOPs. Với các mô hình đã nhỏ, chi phí cố định này chiếm tỉ trọng đáng kể.

b, Lượng tử hoá INT8

Kỹ thuật thứ hai không thay đổi kiến trúc mà giảm độ chính xác biểu diễn số của trọng số, từ FP32 xuống INT8. Đồ án dùng lượng tử hoá động (dynamic quantization) qua ONNX Runtime, không yêu cầu dữ liệu hiệu chuẩn.

Table 4.17: Lượng tử hoá INT8 trên pruned 35.6M

Backend	Dung lượng	Hit@1	Recall@5	MRR@10	NDCG@10
PyTorch FP32 (GPU)	135.8 MB	0.8282	0.9590	0.8844	0.9051
ONNX FP32 (CPU)	136.0 MB	0.8308	0.9590	0.8856	0.9061
ONNX INT8 (CPU)	34.8 MB	0.8231	0.9641	0.8834	0.9041

Lượng tử hoá INT8 giảm dung lượng **74%** ($136.0 \rightarrow 34.8$ MB) trong khi chất lượng gần như không đổi: MRR@10 chênh -0.0022 so với ONNX FP32, tương đương dưới một câu hỏi trên 390. Đáng chú ý, Recall@5 của bản INT8 (0.9641) thậm chí cao hơn cả hai bản còn lại — dấu hiệu cho thấy chênh lệch là nhiễu số học chứ không phải suy giảm hệ thống; nếu INT8 thực sự làm hỏng mô hình thì mọi độ đo phải cùng giảm.

c, Tổng hợp

Table 4.18: So sánh ba kỹ thuật nén

Kỹ thuật	Tác động vào	Dung lượng	Độ trễ	Chất lượng
Cắt tia từ vựng	Bảng embedding	-69.7%	không đổi	-0.2%
Cắt tia tầng (L6)	Số tầng L	-30%	-40%	sụp đổ
Lượng tử hoá INT8	Độ chính xác số	-74%	-27%	-0.1%

Kết quả thực nghiệm cho thấy không tồn tại một kỹ thuật nén duy nhất có thể đồng thời tối ưu mọi tiêu chí. Mỗi phương pháp chỉ tác động lên một thành phần nhất định của mô hình: cắt tia từ vựng làm giảm kích thước bảng embedding nhưng hầu như không cải thiện độ trễ, trong khi cắt tia tầng có thể giảm chi phí suy luận

nhưng gây suy giảm đáng kể chất lượng. Vì vậy, hệ thống cuối kết hợp cắt tia từ vựng và lượng tử hoá INT8. Hai kỹ thuật này bổ sung cho nhau: cắt tia từ vựng giảm kích thước bảng tra cứu, còn lượng tử hoá giảm số bit dùng để biểu diễn tham số. Mô hình sau nén có dung lượng 34.8 MB và có thể triển khai hoàn toàn trên CPU.

4.3.8 Đánh giá triển khai trên CPU

Toàn bộ các phép đo độ trễ ở các mục trước được thực hiện trên GPU. Tuy nhiên, mục tiêu triển khai của đề án là một hệ thống chạy được trên tài nguyên hạn chế, không phụ thuộc GPU. Mục này đo lại độ trễ của teacher và student trên cùng một CPU (16 luồng), rerank top-20 ứng viên với $\text{max_len} = 512$, độ dài trung vị 371 token mỗi cặp.

Table 4.19: Độ trễ trên CPU (16 luồng, rerank top-20)

Model	Params	Dung lượng	Trung vị (ms)	p95 (ms)	MRR@10	Tăng tốc
BGE teacher (PyTorch)	568M	2,165.8 MB	11,209	11,385	0.8862	1.0×
Student FP32 (ONNX)	35.6M	136.0 MB	1,472	2,374	0.8856	7.6×
Student INT8 (ONNX)	35.6M	34.8 MB	1,074	1,209	0.8834	10.4×

Trên cùng phần cứng CPU, teacher BGE 568M cần 11.2 giây cho mỗi truy vấn — một mức độ trễ không thể chấp nhận trong triển khai thực tế, xác nhận rằng mô hình này bắt buộc phải chạy trên GPU. Student sau lượng tử hoá INT8 chỉ cần 1.07 giây, tức nhanh hơn 10.4 lần, đồng thời nhẹ hơn 62 lần (34.8 MB so với 2.17 GB), trong khi MRR@10 chỉ kém 0.0028 — tương đương khoảng một câu hỏi trên 390.

Ngoài nhanh hơn, bản INT8 còn ổn định hơn: khoảng cách giữa p95 và trung vị thu hẹp rõ rệt so với FP32 (1,209 so với 1,074 ms, thay vì 2,374 so với 1,472 ms), nên xét theo p95 — chỉ số quyết định trải nghiệm ở trường hợp xấu — INT8 nhanh hơn FP32 gần 2 lần.

Kết luận. Kết hợp cắt tia từ vựng và lượng tử hoá INT8 cho ra một reranker 34.8 MB đạt chất lượng ngang teacher 568M ($\text{MRR@10} = 0.8834$ so với 0.8862), chạy được hoàn toàn trên CPU với độ trễ khoảng một giây — loại bỏ hoàn toàn yêu cầu GPU khỏi tầng xếp hạng lại của hệ thống.

4.4 Đánh giá bộ phân loại ý định

Bộ phân loại ý định v2 (mô tả ở mục 3.9) tinh chỉnh một Sentence-BERT tiếng Việt (keepitreal/vietnamese-sbert, dựa trên PhoBERT-base) theo phương pháp SetFit — contrastive learning trên encoder rồi đặt một head Logistic Regression trên embedding. Dữ liệu gồm 983 câu sạch với tỉ lệ lớp xấp xỉ 12:1, nên metric chính là F1-macro và việc đánh giá dùng stratified 5-fold cross-validation; một tập test tách riêng 80/20 được dùng bổ sung để phân tích cơ cấu lỗi theo từng lớp.

Bảng 4.20 so sánh ba cấu hình theo F1-macro 5-fold. Tinh chỉnh encoder bằng contrastive learning nâng F1-macro từ 0.859 (embedding đông cứng, chỉ train head) lên 0.947, cải thiện khoảng 0.09 điểm. Hai backbone Sentence-BERT tiếng Việt dựa trên PhoBERT-base cho kết quả tương đương; BKAI Vietnamese bi-encoder có mean ngang nhưng độ lệch chuẩn nhỏ hơn (ổn định hơn).

Table 4.20: So sánh cấu hình ý định classifier theo F1-macro

Cấu hình	F1-macro (5-fold CV)
Embedding đông cứng + Logistic Regression	0.859
SetFit tinh chỉnh (keepitreal/vietnamese-sbert)	0.947 ± 0.028
SetFit tinh chỉnh (bkai-foundation-models/vietnamese-bi-encoder)	0.950 ± 0.017

Độ lệch chuẩn (± 0.02 – 0.03) chủ yếu đến từ việc lớp `tinh_toan` chỉ có 76 mẫu, bị chia mỏng qua các fold (khoảng 15 mẫu mỗi tập test); mỗi câu phân loại sai làm F1-macro của fold đó thay đổi vài phần trăm. Vì vậy con số 5-fold (0.947) đáng tin hơn kết quả single-split (lên tới 0.981), vốn là đầu lạc quan.

Cơ cấu cải thiện thể hiện rõ khi phân tích theo lớp trên tập test tách riêng. Ở cấu hình đông cứng, lớp `tinh_toan` có precision thấp (0.60–0.64): model nhầm nhiều câu `tra_cuu` có chứa số liệu thành `tinh_toan` (ví dụ “hằng số điện môi có giá trị bao nhiêu?” — thực chất là tra một giá trị có sẵn). Sau tinh chỉnh contrastive, precision lớp `tinh_toan` đạt 1.00 và không còn false positive nào trên tập test — ranh giới có sẵn so với phải tính đã được học đúng. Bảng 4.21 trình bày chất lượng nội tại của classifier v2; Bảng 4.22 mới so sánh router v1 và v2 trên cùng tập 390 câu MCQ đầu cuối, tránh trộn kết quả test tách riêng/CV với kết quả pipeline.

Table 4.21: Đánh giá nội tại bộ phân loại ý định v2 (SetFit)

Metric	Kết quả	Protocol
Accuracy	99.49%	Test tách riêng 80/20 trên tập ý định sạch
F1-macro	0.947 ± 0.028	Stratified 5-fold CV trên 983 câu
F1 lớp tra_cuu	0.997	Test tách riêng
F1 lớp tinh_toan	0.966	Test tách riêng
Độ trễ/câu	$\approx 11\text{ms}$ đơn, $< 1\text{ms}$ batch	Đo suy luận local trên GPU T4
Chi phí/1M queries	\$0	Chạy local, không gọi API

Table 4.22: So sánh bộ định tuyến v1 (GPT) và v2 (Sentence-BERT/SetFit)

Metric	GPT-4o-mini router (v1)	Sentence-BERT/SetFit router (v2)
Độ chính xác ý định	94.62% (369/390)	97.69% (381/390)
Số câu sai ý định	21	9
Gold tra_cuu $\rightarrow$ routed tinh_toan	21	6
Gold tinh_toan $\rightarrow$ routed tra_cuu	0	3
Phân bố routed ý định	322 tra_cuu, 68 tinh_toan	340 tra_cuu, 50 tinh_toan
Độ trễ định tuyến trung bình	743.0ms/câu (API)	103.5ms/câu (local)
Chi phí/1M queries	Có chi phí API, phụ thuộc biểu giá và token đầu vào	\$0

Đồ án cũng khảo sát việc bổ sung dữ liệu bằng paraphrase do LLM sinh (786 câu) nhưng loại bỏ hướng này. Kiểm tra cho thấy paraphrase bị trôi nhãn: LLM thường bỏ số liệu cụ thể và biến câu yêu cầu tính toán thành câu hỏi công thức hoặc khái niệm (mang sắc thái tra_cuu), khiến 17/49 câu tinh_toan bị đổi ý nghĩa trong khi vẫn giữ nhãn cũ. Đưa dữ liệu này vào huấn luyện sẽ tiêm nhãn sai vào đúng lớp thiếu số đang yếu. Đây là biểu hiện của distribution gap khi dùng synthetic data, và là lý do model cuối chỉ dùng dữ liệu gán nhãn tay.

So với mục tiêu thiết kế (accuracy $\geq 97\%$, độ trễ thấp), F1-macro 0.947 và accuracy trên test tách riêng 99.49% đáp ứng ngưỡng chất lượng nội tại. Khi đưa vào pipeline MCQ 390 câu, router cục bộ dựa trên Sentence-BERT/SetFit đạt ý định accuracy 97.69%, cao hơn GPT router v1 94.62%. Về độ trễ, classifier local đạt khoảng 11ms/câu khi xử lý đơn lẻ trên GPU T4 và dưới 1ms/câu khi batch; trong phép đo đầu cuối trên máy cá nhân, định tuyến v2 trung bình 103.5ms/câu,

nhanh hơn GPT router v1 743.0ms/câu khoảng 7.2 lần, đồng thời loại bỏ chi phí API và cho phép chạy offline.

4.5 Đánh giá đầu cuối trên câu hỏi trắc nghiệm

Để đánh giá tác động đầu cuối của toàn bộ pipeline v2, đồ án chạy lại bài toán MCQ trên 390 câu hỏi, so sánh trực tiếp pipeline v1 (baseline) với pipeline v2. Pipeline v1 dùng embedding 1024 chiều, GPT router và reranker BGE-reranker-v2-m3; pipeline v2 dùng embedding 512 chiều đã tinh chỉnh (MNRL, resample positive theo epoch), router cục bộ dựa trên Sentence-BERT/SetFit (mục 4.4) và reranker pruned 35.6M. Phép đo thực hiện trong môi trường CPU/API trên máy cá nhân (mục 4.1.1), nên độ trễ phản ánh thiết lập triển khai thực tế chứ không so trực tiếp với độ trễ xếp hạng đo trên GPU ở các bảng trước.

Cần nói rõ về nguồn gốc của bộ 390 câu này. Vì 50 tài liệu của D2 không có bộ câu hỏi–đáp án tham chiếu sẵn (khác với 991 câu gốc của cuộc thi Viettel AI Race trên 200 tài liệu train), toàn bộ câu hỏi và đáp án gold của D2 do tác giả tự biên soạn: 343 câu ban đầu (dùng chung cho đánh giá embedding và reranker ở các mục trước) được sinh bằng GPT theo từng lô nhỏ (khoảng 10 đoạn/lô), sau đó tác giả kiểm tra thủ công tính hợp lý của câu hỏi và độ chính xác của đáp án trước khi chấp nhận từng lô. Khi chuẩn bị đánh giá MCQ đầu cuối, tác giả phát hiện 343 câu này không có câu hỏi `trinh_toan` nào, nên bổ sung thêm 47 câu `trinh_toan` theo cùng quy trình (GPT sinh, tác giả kiểm tra thủ công), tạo thành bộ 390 câu dùng trong mục này. Quy trình kiểm tra thủ công giúp đảm bảo độ chính xác của nhãn ở mức chấp nhận được, nhưng người kiểm tra không độc lập với người đánh giá hệ thống, nên không loại trừ hoàn toàn khả năng thiên lệch trong cách diễn đạt câu hỏi (ví dụ xu hướng vô thức chấp nhận các câu “nhìn hợp lý”, tức có xu hướng khớp từ vựng thuận lợi cho retrieval). Ngoài ra, cần lưu ý rằng 343/390 câu này đã được dùng để so sánh và lựa chọn cấu hình embedding/reranker ở các mục 4.3.1–4.3.6 và mục 4.2.1–4.2.3 trước khi được dùng lại ở đây để đánh giá đầu cuối; do đó bộ 390 câu đóng cả vai trò tập phát triển (dev, dùng để chọn cấu hình) lẫn tập đánh giá cuối, nên các con số tuyệt đối trong mục này có thể lạc quan hơn một chút so với khi đánh giá trên một tập hoàn toàn độc lập chưa từng dùng trong quá trình lựa chọn mô hình. Mục 4.5.3 trình bày một phép kiểm chứng bổ sung trên một mẫu độc lập để định lượng chính xác mức độ lạc quan này.

Table 4.23: Đánh giá đầu cuối trên 390 câu trắc nghiệm: pipeline v1 so với v2

Pipeline	Accuracy	Độ trễ E2E /câu	Intent acc.
v1 baseline (1024d, GPT router, BGE-reranker-v2-m3)	97.44% (380/390)	$\approx 25,898\text{ms}$	94.62%
v2 (512d, Sentence-BERT/SetFit router, pruned 35.6M PyTorch)	97.44% (380/390)	$\approx 3,632\text{ms}$	97.69%
v2 + reranker ONNX INT8	97.18% (379/390)	$\approx 2,714\text{ms}$	97.69%

Về độ chính xác, hai pipeline đạt cùng số câu đúng (380/390). Nếu tính toàn bộ end-to-end gồm cả bước sinh đáp án bằng LLM, v2 giảm thời gian trung bình mỗi câu khoảng 7.1 lần. Tuy nhiên, bước sinh đáp án phụ thuộc API và dao động mạnh theo tải hệ thống/câu hỏi, đặc biệt với câu tính_toan; do đó đồ án báo cáo thêm độ trễ core pipeline chỉ gồm định tuyến, truy hồi và rerank ở Bảng 4.24. Trên phần core này, tức đúng các thành phần được tối ưu trong đồ án, v2 nhanh hơn v1 khoảng 11.2 lần và loại bỏ chi phí gọi API cho cả định tuyến lẫn rerank. Đáng chú ý, router cục bộ dựa trên Sentence-BERT/SetFit của v2 phân loại ý định tốt hơn GPT router của v1 (97.69% so với 94.62%, chi tiết ở mục 4.4).

Table 4.24: Độ trễ trung bình theo thành phần (ms/câu)

Module	v1 baseline	v2	v2 (ONNX INT8)	Speedup v1→INT8
Routing	743.0	103.5	83.4	8.9×
Truy hồi	21.9	32.1	25.8	0.8×
Rerank	23,166.6	2,002.0	1,105.7	21.0×
Core pipeline (không LLM)	23,931.5	2,137.6	1,214.9	19.7×
Sinh câu trả lời (LLM)	1,966.2	1,494.6	1,498.6	1.3×
Tổng	25,897.8	3,632.1	2,713.6	9.5×

Rerank đóng góp phần tăng tốc lớn nhất: v2 PyTorch nhanh hơn v1 khoảng 11.6 lần ở riêng khối rerank, còn cấu hình ONNX INT8 nhanh hơn v1 khoảng 21.0 lần. Tiếp theo là định tuyến, khoảng 7.2 lần khi chuyển từ GPT API sang router cục bộ dựa trên Sentence-BERT/SetFit. Nếu chỉ xét ba thành phần hệ thống RAG được tối ưu (routing + retrieval + rerank), v2 PyTorch giảm độ trễ từ 23,931.5ms xuống 2,137.6ms mỗi câu, tương đương 11.2 lần; với ONNX INT8, độ trễ core còn 1,214.9ms, tương đương 19.7 lần so với v1. Bước sinh câu trả lời bằng LLM không

thuộc phần tối ưu chính và có độ dao động lớn, nên kéo speedup end-to-end xuống còn 7.1 lần với v2 PyTorch và 9.5 lần với ONNX INT8. Về truy hồi, v2 có recall hơi thấp hơn v1 (gold-hit 96.41% so với 96.92%, nhiều hơn hai câu miss), một đánh đổi nhỏ khi dùng embedding 512 chiều nhưng không làm thay đổi accuracy cuối cùng.

Table 4.25: Dung lượng lưu trữ của embedding cache và index trong pipeline đầu cuối

Thành phần	v1 baseline 1024d	v2 512d	Tỉ lệ v1/v2
Model embedding	2,187.00 MiB	2,181.91 MiB	1.00×
Chunk embedding cache	3.18 MiB	1.59 MiB	2.00×
Query embedding cache	1.58 MiB	0.82 MiB	1.93×
Chroma DB	10.68 MiB	9.45 MiB	1.13×
Cache + Chroma	15.44 MiB	11.85 MiB	1.30×
Tổng model + runtime storage	2,202.44 MiB	2,193.77 MiB	1.004×

Bảng 4.25 cho thấy lợi ích bộ nhớ chính của v2 nằm ở phần vector runtime: đoạn cache giảm từ ma trận (813, 1024) xuống (813, 512) float32, còn query cache giảm từ 390 vector 1024 chiều xuống 390 vector 512 chiều. Vì vậy phần raw embedding cache giảm gần đúng 2 lần. Chroma DB chỉ giảm 1.13 lần do ngoài vector còn chứa metadata và cấu trúc chỉ mục. Nếu tính cả thư mục model embedding, tổng dung lượng chỉ giảm nhẹ vì hai encoder đều có kích thước khoảng 2.18GB; tuy nhiên trong triển khai với corpus lớn hơn, phần cache/index sẽ tăng tuyến tính theo số đoạn và lợi thế 512d sẽ rõ hơn.

Nhìn chung, pipeline v2 giữ được độ chính xác của v1, nhanh hơn khoảng 7.1 lần nếu tính end-to-end và khoảng 11.2 lần trên phần core pipeline không gồm LLM, đồng thời giảm lưu trữ runtime cho vector/index và chạy hoàn toàn cục bộ, đúng với mục tiêu hiệu quả của đề án.

4.5.1 Triển khai reranker ONNX INT8 trong pipeline đầu cuối

Mục 4.3.8 đã đo reranker INT8 một cách cô lập trên CPU. Mục này đo tác động của nó khi lắp vào pipeline đầu cuối, thay cho bản PyTorch, giữ nguyên mọi thành phần khác (cùng embedding 512d, cùng router, cùng LLM sinh đáp án).

Về độ trễ, chỉ khối rerank thay đổi thực chất: từ 2,002.0 ms xuống 1,105.7 ms mỗi câu, tương đương $1.81\times$. Chênh lệch ở routing (103.5 so với 83.4 ms) và truy hồi (32.1 so với 25.8 ms) là dao động runtime giữa hai lần chạy chứ không phản ánh thay đổi thuật toán — hai cấu hình dùng chung router và chung bộ truy hồi. Khối core (routing + truy hồi + rerank), tức đúng phần được tối ưu trong đề án,

giảm từ 2,137.6 ms xuống 1,214.9 ms ($1.76\times$); so với v1 baseline, mức tăng tốc đạt $19.7\times$. Tổng thời gian đầu cuối chỉ nhanh hơn $1.34\times$ vì bước sinh đáp án bằng LLM (khoảng 1,500 ms, không đổi) đã chiếm hơn một nửa thời gian còn lại — một dấu hiệu cho thấy tầng truy hồi–xếp hạng đã được tối ưu tới mức không còn là nút thắt của pipeline.

Về chất lượng, ONNX INT8 đạt 379/390 câu đúng so với 380/390 của bản PyTorch, tức kém đúng một câu. Cần thận trọng khi diễn giải con số này: chênh lệch một câu trên 390 tương đương 0.26 điểm phần trăm, nằm hoàn toàn trong dao động thống kê của cỡ mẫu này và không đủ để kết luận rằng lượng tử hoá làm giảm chất lượng. Ba dấu hiệu ủng hộ nhận định đó. Thứ nhất, tỉ lệ gold nằm trong top-5 sau rerank của bản INT8 lại cao hơn (377/390 so với 376/390) — nếu lượng tử hoá thực sự làm suy giảm khả năng xếp hạng, chỉ số này phải giảm chứ không tăng. Thứ hai, độ chính xác định tuyến ý định không đổi (381/390) vì hai cấu hình dùng chung router. Thứ ba, và quan trọng nhất, khi soi câu bị mất (câu số 254): gold đoạn không nằm trong top-5 ở cả hai cấu hình — nghĩa là ở câu này LLM trả lời mà không có ngữ cảnh đúng, và bản PyTorch trả lời đúng là do may mắn đoán trúng chứ không phải nhờ reranker tốt hơn. Hai câu còn lại đối dự đoán (182 và 341) đều sai ở cả hai cấu hình.

Nói cách khác, khác biệt một câu giữa hai cấu hình không quy được về chất lượng của reranker. Kết luận thực tiễn: với mức tăng tốc $1.81\times$ ở khối rerank, giảm dung lượng từ 136.0 MB xuống 34.8 MB (mục 4.3.8) và chất lượng không suy giảm một cách đo được, ONNX INT8 là cấu hình được khuyến nghị cho triển khai trên CPU.

Về đuôi độ trễ. Thời gian tổng có dao động lớn giữa các câu (trung vị 1,906 ms nhưng p95 tới 8,555 ms). Phân rã theo thành phần cho thấy toàn bộ phân dao động này đến từ bước sinh đáp án bằng LLM, trong khi khối rerank rất ổn định. Nói cách khác, sau khi tối ưu, tầng truy hồi–xếp hạng không còn là nguồn gây chậm của hệ thống.

4.5.2 Phân rã nguồn lỗi đầu cuối

Bảng 4.23 cho thấy v2 đạt cùng số câu đúng với v1 (380/390). Để xác định lỗi còn lại đến từ thành phần nào trong pipeline, đề án phân rã từng câu trả lời sai theo hai trục độc lập: (1) bộ phân loại ý định định tuyến đúng hay sai, và (2) đáp án cuối đúng hay sai. Kết quả tạo thành ma trận 2×2 ở Bảng 4.26.

Table 4.26: Ma trận định tuyến $\times$ đáp án cuối trên 390 câu trắc nghiệm

Trường hợp	v1 (GPT router)		v2 (Sentence-BERT/SetFit router)	
	Đáp án đúng	Đáp án sai	Đáp án đúng	Đáp án sai
Định tuyến đúng	361	8	372	9
Định tuyến sai	19	2	8	1

Ô đáng chú ý nhất là định tuyến sai **và** đáp án sai — tức lỗi định tuyến thực sự làm hỏng đáp án cuối: v2 chỉ có 1 câu, v1 có 2 câu. Như vậy, mặc dù v2 có chín câu định tuyến sai, gần như toàn bộ là vô hại với độ chính xác: sáu câu tra_cuu bị định tuyến nhầm sang tinh_toan chỉ phát sinh thêm một lượt suy luận (lỗi chi phí/độ trễ, không sai đáp án), còn trong ba câu tinh_toan bị nhầm sang tra_cuu, chỉ duy nhất một câu thực sự mất bước suy luận từng bước và bị lật đáp án (ví dụ câu hỏi “*kích thước large_string gấp bao nhiêu lần kích thước buffer*” — một phép chia bị nhận nhầm là tra cứu do cách phát biểu đơn giản, đúng dạng lỗi biên đã mô tả ở mục 4.4 liên quan đến câu tính toán phát biểu giống tra cứu). Hệ quả là việc thay GPT router bằng router cục bộ dựa trên Sentence-BERT/SetFit không làm giảm độ chính xác, thậm chí còn giảm một câu lỗi do định tuyến.

Như vậy, việc thay router không làm giảm độ chính xác đầu cuối; các lỗi còn lại chủ yếu phản ánh đánh đổi recall ở tầng truy hồi khi giảm chiều biểu diễn xuống 512 và giới hạn của bước sinh đáp án — mục 4.5 đã ghi nhận v2 có tỉ lệ gold-hit thấp hơn v1 một chút — một lựa chọn có chủ đích để đổi lấy dung lượng vector index và chi phí tìm kiếm tương đồng nhỏ hơn hai lần. Các hướng cải thiện tầng truy hồi được thảo luận ở mục 4.8.

4.5.3 Kiểm chứng đối chiếu trên mẫu độc lập

Như đã nêu ở mục 4.5, bộ 390 câu MCQ vừa đóng vai trò tập phát triển (dùng để lựa chọn cấu hình embedding và reranker) vừa đóng vai trò tập đánh giá cuối, nên các chỉ số tuyệt đối có thể bị lạc quan hơn thực tế do hiện tượng lựa chọn cấu hình trên chính tập dùng để báo cáo (một dạng winner’s curse trong model selection). Để định lượng mức độ ảnh hưởng này, đồ án xây dựng thêm một mẫu đối chiếu độc lập gồm 71 câu hỏi trắc nghiệm, lấy từ 16 tài liệu trong tập D2 (không trùng với 12 tài liệu đã đóng góp phần lớn bộ 390 câu), theo cùng quy trình sinh câu hỏi bằng GPT và kiểm tra thủ công như mục 4.5. Điểm khác biệt then chốt: 71 câu này chưa từng được dùng ở bất kỳ bước lựa chọn cấu hình nào trước đó — toàn bộ được sinh và kiểm tra sau khi cấu hình cuối cùng của pipeline v1 và v2 đã được chốt. Tỉ lệ ý định trong mẫu là 61 câu tra_cuu và 10 câu tinh_toan ($\approx 14\%$), gần với tỉ lệ 12% của tập huấn luyện gốc (mục 3.9.2).

Table 4.27: So sánh v1 và v2 trên mẫu đối chiếu độc lập (71 câu)

Chỉ số	v1 baseline	v2
Accuracy	92.96% (66/71)	94.37% (67/71)
Định tuyến đúng	95.77% (68/71)	98.59% (70/71)
Gold có trong top-5	95.77% (68/71)	94.37% (67/71)

Ba quan sát chính. Thứ nhất, so với kết quả khoảng 97% trên bộ 390 câu ở Bảng 4.23, độ chính xác trên mẫu đối chiếu độc lập giảm xuống 92.96% với v1 và 94.37% với v2, đúng như dự đoán về hiện tượng lạc quan hóa khi tập 390 câu vừa được dùng để chọn cấu hình vừa được dùng để báo cáo. Thứ hai, và quan trọng hơn, **kết luận cốt lõi không những được xác nhận mà còn thể hiện mạnh hơn**: trên mẫu đối chiếu, v2 cao hơn v1 một câu (67/71 so với 66/71), củng cố luận điểm rằng pipeline v2 giữ được chất lượng ít nhất tương đương v1 trong khi giảm độ trễ đáng kể (xem Bảng 4.29). Thứ ba, mức cải thiện của router cục bộ Sentence-BERT/SetFit so với GPT router tái lập với độ lớn tương tự: +3.07 điểm phần trăm trên 390 câu (97.69% – 94.62%) so với +2.82 điểm phần trăm trên mẫu đối chiếu (98.59% – 95.77%), cho thấy đây là một cải thiện thật, không phải hiện tượng ngẫu nhiên do lựa chọn cấu hình.

Table 4.28: Ma trận định tuyến $\times$ đáp án cuối trên mẫu đối chiếu (71 câu)

Trường hợp	v1 (GPT router)		v2 (Sentence-BERT/SetFit router)	
	Đáp án đúng	Đáp án sai	Đáp án đúng	Đáp án sai
Định tuyến đúng	63	5	66	4
Định tuyến sai	3	0	1	0

Ma trận ở Bảng 4.28 tái khẳng định phát hiện đã nêu ở mục 4.5.2: ô định tuyến sai và đáp án sai bằng 0 ở cả hai pipeline trên mẫu đối chiếu này. Ba câu v1 định tuyến sai (đều tra_cuu bị nhận nhầm thành tinh_toan) và một câu v2 định tuyến sai (một câu tinh_toan bị nhận nhầm thành tra_cuu) đều không làm lật đáp án cuối. Bốn lỗi đáp án còn lại của v2 đều xảy ra khi định tuyến đã đúng, cho thấy sai khác chủ yếu nằm ở truy hồi/sinh đáp án thay vì bộ định tuyến.

Table 4.29: Độ trễ trung bình theo thành phần trên mẫu đối chiếu (ms/câu)

Module	v1 baseline	v2	Speedup
Routing	752.7	100.0	7.5 $\times$
Truy hồi	6.2	19.6	0.3 $\times$
Rerank	13,956.2	1,671.3	8.4 $\times$
Sinh câu trả lời (LLM)	2,486.9	1,512.3	1.6 $\times$
Tổng	17,202.0	3,303.3	5.2$\times$

Về độ trễ, mức tăng tốc trên mẫu đối chiếu ($5.2\times$ tổng, $8.4\times$ riêng rerank) thấp hơn so với trên 390 câu ($7.1\times$ tổng, $11.6\times$ riêng rerank). Vì hai lần đo được thực hiện ở hai thời điểm khác nhau trên cùng máy cá nhân (mục 4.1.1), chênh lệch này nhiều khả năng phản ánh dao động tải hệ thống hoặc điều kiện đo (ví dụ trạng thái cache, tải CPU nền) giữa hai lần chạy hơn là một thay đổi thực chất trong hành vi của pipeline; với $n = 71$, ước lượng độ trễ cũng kém ổn định hơn so với $n = 390$. Dù vậy, mức tăng tốc vẫn ở mức đáng kể (hơn 5 lần cho tổng độ trễ, hơn 8 lần riêng bước rerank), nên kết luận về lợi ích tốc độ của v2 vẫn đứng vững, chỉ nên hiệu bội số chính xác ($7.1\times$ hay $5.2\times$) là phụ thuộc điều kiện đo chứ không phải một hằng số cố định.

Một phát hiện đáng chú ý khác liên quan đến chỉ số gold-hit của v2: rà soát chi tiết cho thấy 4 trong số 71 câu (gold chunk theo nhãn của bộ tạo câu không nằm trong top-5 kết quả truy hồi) vẫn được trả lời đúng, nhờ ngữ cảnh chứa các đoạn khác trình bày cùng nội dung hoặc chủ đề liên quan (ví dụ một định nghĩa tương đương ở đoạn khác, hoặc một công thức/máy móc liên quan đủ để suy ra đáp án). Do đó, trong 4 lỗi đầu cuối của v2 trên mẫu đối chiếu, không lỗi nào xuất phát từ việc mất gold khỏi top-5; toàn bộ là lỗi sinh đáp án dù ngữ cảnh có chứa thông tin liên quan. Điều này cho thấy chỉ số gold-hit dựa trên nhãn gold đơn nhất (mỗi câu hỏi chỉ gán đúng một đoạn nguồn) có thể đánh giá thấp hơn khả năng truy hồi thực tế của hệ thống, vì một nội dung kỹ thuật có thể được trình bày ở nhiều đoạn khác nhau trong corpus — cùng hiện tượng false negative đã thảo luận ở mục 2.4.4 cho dữ liệu huấn luyện embedding, nay quan sát được ở phía đánh giá. Đây là giới hạn chung của các độ đo dạng $\text{Hit}@k/\text{gold-hit}$ khi giả định mỗi câu hỏi chỉ có một đáp án nguồn duy nhất, không phải hạn chế riêng của pipeline v2.

Tóm lại, phép kiểm chứng đối chiếu xác nhận: (i) các chỉ số tuyệt đối trên bộ 390 câu có phần lạc quan hơn thực tế khoảng 3–5 điểm phần trăm do hiệu ứng lựa chọn cấu hình, đúng như dự đoán; (ii) kết luận so sánh giữa v1 và v2 — accuracy tương đương hoặc nhỉnh hơn nhẹ cho v2, tốc độ nhanh hơn đáng kể, router cục bộ chính xác hơn GPT router — vẫn đứng vững và tái lập nhất quán trên dữ liệu độc lập; và (iii) một phần chỉ số gold-hit có thể bị đánh giá thấp hơn thực tế do giới hạn của nhãn gold đơn nhất.

4.6 Phân tích token ngoài từ vựng sau cắt tỉa

Một rủi ro tiềm ẩn của cắt tỉa từ vựng là token trong tập đánh giá có thể không nằm trong từ vựng rút gọn và bị thay bằng [UNK], làm giảm chất lượng mã hoá. Đồ án đo UNK rate trực tiếp bằng tokenizer pruned trên tập D2 — hoàn toàn ngoài miền, không dùng khi xây dựng từ vựng.

Kết quả: toàn bộ 390 câu hỏi D2 có UNK rate = 0.000%. Trên 813 đoạn corpus D2, chỉ 49 trong số 196.046 token bị UNK (0.025%), phân bố trên 33 đoạn — tương đương dưới 2 token mỗi đoạn bị ảnh hưởng. Ở mức này reranker đọc được trên 99.97% nội dung tài liệu ngoài miền, và chiến lược giữ lại 30.000 token phổ biến nhất cùng toàn bộ token trong corpus miền đã đủ phủ từ vựng tiếng Việt kỹ thuật trong thực tế.

4.7 Khuyến nghị cấu hình triển khai

Từ kết quả embedding và reranker, cấu hình được khuyến nghị cho pipeline v2 là đã tinh chỉnh embedding 512d để truy xuất top-20, sau đó dùng mmarco-mMiniLMv2 117M đã tinh chỉnh để rerank. Cấu hình này đạt gần chất lượng teacher BGE-reranker-v2-m3 nhưng có độ trễ thấp hơn nhiều trong phép đo xếp hạng pipeline.

Table 4.30: Các cấu hình pipeline khuyến nghị

Ưu tiên	Cấu hình	MRR @10	Độ trễ	Nhận xét
Tốc độ tối đa	Embedding tinh chỉnh 512d, không reranker	0.7943	6.8ms (encode+truy hồi)	Phù hợp khi tài nguyên rất hạn chế
Cân bằng khuyến nghị	Embedding tinh chỉnh 512d + mmarco 117M đã tinh chỉnh	0.8862	50.8ms (rerank)	Ngang MRR của BGE-reranker-v2-m3 với thời gian rerank thấp hơn khoảng 7.6 lần
Bộ nhớ thấp	Embedding tinh chỉnh 512d + pruned mmarco 35.6M	0.8844	51.0ms (rerank)	Giảm tham số 69.7%, chất lượng gần mmarco 117M
Chất lượng tối đa	Embedding tinh chỉnh 512d + BGE-reranker-v2-m3 teacher	0.8862	385.5ms (rerank)	Chất lượng cao nhất, thời gian rerank lớn

Kết quả này cho thấy lựa chọn tối ưu không phải luôn là model nhỏ nhất. Khi bộ truy hồi đã mạnh, reranker 33M không đủ capacity để khai thác candidate list tốt hơn, trong khi mmarco 117M và pruned 35.6M vẫn cải thiện đáng kể. Vì vậy, pipeline production nên dùng 512d embedding làm bộ truy hồi chính và chọn giữa mmarco 117M hoặc pruned 35.6M tùy ràng buộc bộ nhớ.

4.8 Phân tích lỗi tổng thể

Tổng hợp phân tích từ các thành phần cho thấy ba nguồn gốc lỗi chính theo thứ tự tác động giảm dần.

Nguồn 1 – Giới hạn trần của truy hồi.

Một phần lỗi xuất hiện trước khi reranker nhìn thấy dữ liệu: gold đoạn không nằm trong top-20 candidates hoặc chỉ xuất hiện ở vị trí thấp. Embedding tinh chỉnh 512d cải thiện Acc@1 trên D1 (in-domain) từ 65.19% lên $70.46 \pm 0.30\%$ và trên D2 độc lập từ 70.77% lên $71.79 \pm 0.42\%$, giúp giảm nhóm lỗi truy hồi nhưng không loại bỏ hoàn toàn. Các trường hợp khó nhất gồm acronym match, bảng số liệu liên kề và đoạn có nội dung gần giống nhau.

Nguồn 2 – Sai lệch nhỏ trong xếp hạng.

Nhiều lỗi không phải sai hoàn toàn mà là gold nằm gần ngưỡng chọn ngữ cảnh, ví dụ rank 6–10. Tăng top-K từ 5 lên 6 hoặc 7 có thể khôi phục thêm một phần các query này, nhưng phải đánh đổi với ngữ cảnh window lớn hơn và chi phí generation cao hơn. Đây là trade-off cần được quyết định theo yêu cầu triển khai thực tế.

Nguồn 3 – Nhầm lẫn đặc thù miền.

Cross-domain noise và same-domain confusion phản ánh giới hạn của embedding và BM25 với tài liệu kỹ thuật đặc thù. Các hướng cải thiện tiếp theo gồm bổ sung dữ liệu domain-specific cho embedding tinh chỉnh, áp dụng query expansion để tăng recall trước khi rerank, hoặc cải thiện chunking strategy cho bảng biểu và đoạn định nghĩa ngắn.

Nguồn 4 – Giới hạn của các kỹ thuật nén đã khảo sát.

Mục 4.3.7 cho thấy ba kỹ thuật nén tác động vào ba thành phần khác nhau của chi phí tính toán, và không kỹ thuật nào cải thiện được đồng thời cả ba mặt (dung lượng, độ trễ, chất lượng) ở mức tối đa. Các hướng nén giảm được độ trễ mà đồ án chưa khảo sát đầy đủ gồm: (i) chưng cất theo tầng đúng cách (layer-wise distillation kiểu TinyBERT, ép khớp hidden states và attention maps ngay trong quá trình huấn luyện) thay vì cất tĩa tầng rồi tinh chỉnh ngắn — cách tiếp cận sau đã được chứng minh là phá huỷ mô hình; (ii) lượng tử hoá tĩnh (nén cả activation, có thể đạt $2\text{--}3\times$ thay vì $1.37\times$ của lượng tử hoá động, nhưng cần tập dữ liệu hiệu chuẩn); và (iii) PreTTR [27] — tiền tính biểu diễn token của tài liệu tại thời điểm lập chỉ mục, chỉ ghép với biểu diễn truy vấn ở các tầng trên. Hướng thứ ba đặc biệt phù hợp với hệ thống của đồ án vì tập đoạn văn là cố định và biết trước, song đòi hỏi huấn luyện lại mô hình với ràng buộc kiến trúc tách (các tầng dưới không được thấy truy vấn) và phát sinh chi phí lưu trữ đáng kể (khoảng 0.35 MB mỗi đoạn ở FP16). Ngoài ra, ở mức hệ thống, hai đòn bẩy rẻ hơn có thể áp dụng ngay: cascade (rerank top-10 thay vì top-20, giảm khoảng một nửa chi phí nếu recall@10 không kém recall@20) và length bucketing (sắp xếp ứng viên theo độ dài trước khi chia batch, giảm lãng phí do đệm động).

Nhìn tổng thể, pipeline v2 đạt mục tiêu chính: đã tinh chỉnh embedding 512d (MNRL, resample positive theo epoch) đạt $\text{MRR@10} = 0.7954$ trên D2 độc lập (390 câu), mmarco 117M đã tinh chỉnh nâng pipeline reranker lên $\text{MRR@10} = 0.8862$ với thời gian rerank 50.8ms, còn pruned 35.6M đạt $\text{MRR@10} = 0.8844$ với dung lượng nhỏ hơn. Sau khi kết hợp cắt tỉa từ vựng với lượng tử hoá INT8, mô hình triển khai cuối chỉ còn **34.8 MB** và chạy được hoàn toàn trên CPU (1.07 giây mỗi truy vấn, nhanh hơn teacher 10.4 lần trên cùng phần cứng), qua đó loại bỏ yêu cầu GPU khỏi tầng xếp hạng lại. Các giới hạn còn lại tập trung ở truy hồi ceiling, dữ liệu domain-specific và chunking strategy.

CHƯƠNG 5. KẾT LUẬN

5.1 Tổng kết

Đề án giải quyết bài toán hỏi đáp trên kho tài liệu kỹ thuật tiếng Việt bằng kiến trúc Retrieval-Augmented Generation, với trọng tâm là tối ưu hai thành phần quyết định chất lượng truy hồi: mô hình embedding và reranker. Xuất phát từ một hệ thống nền tảng (v1), đề án đo lường và xác định ba điểm nghẽn cụ thể về độ trễ, chi phí và chất lượng, sau đó đề xuất hệ thống v2 với nguyên tắc mỗi thay đổi kỹ thuật phải giải quyết một vấn đề đã được đo lường thay vì tối ưu dựa trên giả định.

Về embedding, đề án tinh chỉnh mô hình Vietnamese_Embedding_v2 (dựa trên BGE-M3) bằng hàm mất mát MultipleNegativesRankingLoss trên các cặp câu hỏi–positive của miền, kết hợp Matryoshka Representation Learning. Mô hình sau tinh chỉnh nâng Acc@1 trên tập in-domain từ 65.19% lên $73.21 \pm 0.60\%$ (trung bình trên ba seed), đồng thời vượt cả BGE-M3 gốc trên tập tài liệu độc lập, cho thấy việc thích nghi miền có hiệu quả nhất quán chứ không chỉ trên dữ liệu quen thuộc. Nhờ MRL, biểu diễn 512 chiều giữ được phần lớn chất lượng của 1024 chiều, giúp giảm một nửa dung lượng vector index mà không hy sinh đáng kể độ chính xác.

Về reranker, đề án chưng cất teacher BGE-reranker-v2-m3 (568M tham số) thành các student nhẹ thông qua quy trình hai giai đoạn (contrastive learning rồi chưng cất tri thức) với hàm mất mát ADR-MSE theo hướng rank-based. Student mmarco 117M đạt $MRR@10 = 0.8862$, bằng đúng teacher 568M, còn phiên bản cắt tỉa từ vừng 35.6M đạt $MRR@10 = 0.8844$ với dung lượng nhỏ hơn nhiều. Đáng chú ý, reranker 35.6M vượt ViRanker 568M (0.8593, lớn hơn khoảng 16 lần) trên cùng tập đánh giá. Sau lượng tử hoá INT8, mô hình chỉ còn 34.8 MB và chạy trên CPU nhanh hơn teacher khoảng 10 lần, loại bỏ hoàn toàn yêu cầu GPU khỏi tầng xếp hạng lại.

Về routing, đề án thay GPT router bằng một bộ phân loại ý định cục bộ dựa trên Sentence-BERT, loại bỏ phụ thuộc vào API ngoài, giảm độ trễ và chi phí trên mỗi truy vấn đồng thời giữ dữ liệu trong hạ tầng nội bộ.

5.2 Đóng góp chính

Các đóng góp chính của đề án gồm:

- **Quy trình xây dựng dữ liệu huấn luyện dùng chung cho embedding và reranker.** Quy trình dùng mô hình ngôn ngữ làm giám khảo xác định positive, lọc khả năng trả lời độc lập, rồi khai thác hard negative từ kết quả truy hồi cho tầng xếp hạng. Cùng một nguồn dữ liệu được tái sử dụng cho cả hai tầng với

tiêu chí lọc riêng.

- **Tinh chỉnh embedding theo miền kết hợp MRL.** Tinh chỉnh bằng MultipleNegativesRankingLoss trên các cặp câu hỏi–positive của miền, kết hợp MRL để một mô hình duy nhất cho biểu diễn dùng được ở ba số chiều; nhờ đó embedding 512 chiều giữ được phần lớn chất lượng của 1024 chiều với nửa dung lượng index.
- **Nén reranker bằng chung cất tri thức kết hợp cắt tỉa từ vựng.** Chung cất teacher BGE-reranker-v2-m3 qua hai giai đoạn rồi cất tỉa từ vựng, rồi lượng tử hoá INT8, tạo ra reranker 34.8 MB đạt chất lượng cao hơn các baseline tiếng Việt lớn hơn nhiều lần và chạy được hoàn toàn trên CPU.
- **Hệ thống RAG tối ưu đầu cuối.** Tích hợp embedding tinh chỉnh 512 chiều, reranker chung cất và bộ phân loại ý định cục bộ thay GPT router, giảm độ trễ, dung lượng và chi phí suy luận so với hệ thống v1 trong khi vẫn giữ chất lượng truy hồi và xếp hạng.

5.3 Hạn chế

Bên cạnh các kết quả đạt được, đồ án còn một số hạn chế.

Thứ nhất, mức cải thiện của tinh chỉnh embedding rõ rệt trên tập in-domain nhưng khiêm tốn trên tập tài liệu độc lập (khoảng một điểm phần trăm Acc@1), do base model đã được huấn luyện contrastive ở quy mô lớn và lượng dữ liệu miền còn nhỏ. Bi-encoder cũng có trần kiến trúc khi phải nén toàn bộ ngữ nghĩa vào một vector đơn, khiến việc phân biệt các hard negative khác nhau ở mức chi tiết factual (số liệu, thực thể) trở nên khó khăn; phần này được bù đắp bởi reranker nhưng chưa được giải quyết triệt để ở tầng embedding.

Thứ hai, cả dữ liệu huấn luyện lẫn dữ liệu đánh giá đều có một phần được sinh tự động qua mô hình ngôn ngữ và xác nhận thủ công bởi chính tác giả. Với 50 tài liệu của tập D2 — vốn không có bộ câu hỏi–đáp án tham chiếu sẵn — toàn bộ câu hỏi và đáp án gold dùng cho đánh giá reranker, embedding và MCQ đầu cuối (mục 4.1.2, 4.5) do tác giả tự biên soạn với sự hỗ trợ của GPT, sau đó tự kiểm tra thủ công. Người kiểm tra không độc lập với người đánh giá hệ thống, nên không loại trừ hoàn toàn khả năng thiên lệch trong cách diễn đạt câu hỏi. Thêm vào đó, bộ 390 câu MCQ vừa được dùng để lựa chọn cấu hình embedding/reranker vừa được dùng lại để báo cáo kết quả đầu cuối, nên các chỉ số tuyệt đối có thể lạc quan hơn thực tế do hiệu ứng lựa chọn cấu hình trên chính tập báo cáo. Để định lượng ảnh hưởng này, một mẫu đối chiếu độc lập gồm 71 câu (chưa từng dùng ở bất kỳ bước lựa chọn cấu hình nào) đã được đánh giá riêng ở mục 4.5.3: kết quả cho thấy độ

chính xác giảm khoảng 4–5 điểm phần trăm so với bộ 390 câu, xác nhận mức lạc quan như dự đoán, nhưng kết luận so sánh giữa v1 và v2 — độ chính xác tương đương, tốc độ nhanh hơn đáng kể, router cục bộ chính xác hơn GPT router — vẫn tái lập nhất quán trên dữ liệu độc lập này.

Thứ ba, cắt tía từ vung giảm dung lượng mô hình nhưng bản thân nó không giảm đáng kể độ trễ của Transformer encoder, do số tầng và kích thước ẩn được giữ nguyên; phần tăng tốc chủ yếu đến từ chưng cất (student nhỏ hơn teacher) và lượng tử hoá INT8. Lượng tử hoá được dùng ở đây là dạng động (chỉ nén trọng số), chưa khai thác hết mức tăng tốc mà lượng tử hoá tĩnh có thể mang lại.

Thứ tư, hệ thống được đánh giá trên một miền tài liệu kỹ thuật cụ thể. Khả năng tổng quát hóa sang các miền khác hoặc các loại tài liệu khác cần được kiểm chứng thêm.

5.4 Hướng phát triển

Từ các hạn chế trên, đề án đề xuất ba hướng phát triển tiếp theo.

Thứ nhất, về dữ liệu và chất lượng truy hồi, mẫu đối chiếu độc lập ở mục 4.5.3 là một bước đầu để kiểm chứng độ tin cậy của đánh giá; hướng xa hơn là xây dựng một bộ kiểm thử được một người chú thích độc lập (không tham gia phát triển hệ thống) xác minh hoàn toàn, nhằm loại bỏ khả năng thiên lệch do người đánh giá đồng thời là người biên soạn câu hỏi. Đồng thời, hướng khả dĩ để nâng trăn chất lượng ở tầng truy hồi là mở rộng số lượng câu hỏi thật của miền, thay vì tăng độ phức tạp của hàm mất mát.

Thứ hai, về mô hình, lượng tử hoá tĩnh (nén cả activation, cần tập dữ liệu hiệu chuẩn) có thể đạt mức tăng tốc cao hơn lượng tử hoá động đang dùng; ngoài ra có thể khảo sát cắt tía tầng kèm chưng cất, cũng như thử nghiệm các kiến trúc reranker khác để tìm điểm cân bằng tốt hơn giữa chất lượng và tốc độ.

Thứ ba, về khả năng tổng quát hóa và triển khai thực tế, việc đánh giá hệ thống trên nhiều miền tài liệu khác nhau và thử nghiệm trong môi trường vận hành thực tế sẽ giúp kiểm chứng tính ứng dụng của các giải pháp đề xuất ngoài phạm vi miền tài liệu kỹ thuật hiện tại.

TÀI LIỆU THAM KHẢO

- [1] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2005.11401, 2020.
- [2] Beijing Academy of Artificial Intelligence, *BAAI/bge-reranker-v2-m3*, <https://huggingface.co/BAAI/bge-reranker-v2-m3>, 2024.
- [3] OpenAI, *GPT-4o mini: Advancing cost-efficient intelligence*, <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>, 2024.
- [4] AITeamVN, *Vietnamese_embedding_v2*, https://huggingface.co/AITeamVN/Vietnamese_Embedding_v2, 2025.
- [5] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: BM25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009. DOI: 10.1561/15000000019
- [6] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, “Reciprocal rank fusion outperforms condorcet and individual rank learning methods,” in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2009, pp. 758–759. DOI: 10.1145/1571941.1572114
- [7] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *NeurIPS Deep Learning Workshop*, arXiv:1503.02531, 2015.
- [8] S. Hofstätter, S. Althammer, M. Schröder, M. Sertkan, and A. Hanbury, “Improving efficient neural ranking models with cross-architecture knowledge distillation,” *arXiv preprint arXiv:2010.02666*, 2020.
- [9] C. Burges et al., “Learning to rank using gradient descent,” in *Proceedings of the 22nd International Conference on Machine Learning*, 2005, pp. 89–96. DOI: 10.1145/1102351.1102363
- [10] F. Schlatt et al., “Rank-distillm: Closing the effectiveness gap between cross-encoders and LLMs for passage re-ranking,” in *Advances in Information Retrieval (ECIR 2025)*, arXiv:2405.07920, 2025.
- [11] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:1706.03762, 2017.
- [12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, arXiv:1810.04805, 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423

- [13] D. Q. Nguyen and A. T. Nguyen, “PhoBERT: Pre-trained language models for vietnamese,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, arXiv:2003.00744, 2020, pp. 1037–1042. DOI: 10.18653/v1/2020.findings-emnlp.92
- [14] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using siamese BERT-networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, arXiv:1908.10084, 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410
- [15] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” *arXiv preprint arXiv:2402.03216*, 2024.
- [16] M. Henderson et al., “Efficient natural language response suggestion for smart reply,” *arXiv preprint arXiv:1705.00652*, 2017.
- [17] A. V. Solatorio, “GISTEmbed: Guided in-sample selection of training negatives for text embedding fine-tuning,” *arXiv preprint arXiv:2402.16829*, 2024.
- [18] A. Kusupati et al., “Matryoshka representation learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2205.13147, 2022.
- [19] J. Wei et al., “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:2201.11903, 2022.
- [20] L. Tunstall et al., “Efficient few-shot learning without prompts,” *arXiv preprint arXiv:2209.11055*, 2022.
- [21] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems (TOIS)*, vol. 20, no. 4, pp. 422–446, 2002. DOI: 10.1145/582415.582418
- [22] Datalab, *Marker: Convert documents to markdown, json, chunks, and html*, <https://github.com/datalab-to/marker>, 2024.
- [23] T. Nguyen et al., “MS MARCO: A human generated MACHine reading COMprehension dataset,” in *Proceedings of the Workshop on Cognitive Computation: Integrating Neural and Symbolic Approaches 2016 Co-located with the 30th Annual Conference on Neural Information Processing Systems*, arXiv:1611.09268, 2016.
- [24] L. Bonifacio et al., “mMARCO: A multilingual version of the MS MARCO passage ranking dataset,” *arXiv preprint arXiv:2108.13897*, 2021.

- [25] keepitreal, *Vietnamese-sbert*, <https://huggingface.co/keepitreal/vietnamese-sbert>.
- [26] P.-N. Dang, K.-L. Nguyen, and T.-H. Pham, *ViRanker: A BGE-M3 and block-wise parallel transformer cross-encoder for vietnamese reranking*, <https://huggingface.co/namdp-ptit/ViRanker>, 2025.
- [27] S. MacAvaney, F. M. Nardini, R. Perego, N. Tonellotto, N. Goharian, and O. Frieder, “Efficient document re-ranking for transformers by precomputing term representations,” in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, arXiv:2004.14255, 2020, pp. 49–58. DOI: 10.1145/3397271.3401093

PHỤ LỤC A. MÔI TRƯỜNG VÀ CẤU HÌNH HUẤN LUYỆN

A.1 Môi trường phần cứng và phần mềm

Toàn bộ quá trình huấn luyện và đánh giá được thực hiện trên nền tảng Kaggle Notebook. Việc huấn luyện embedding và reranker dùng GPU NVIDIA RTX 6000; việc đánh giá độ trễ bộ phân loại ý định dùng GPU NVIDIA T4; còn phép đo đầu cuối trên bài toán trắc nghiệm được thực hiện trong môi trường CPU và API trên máy cá nhân để phản ánh thiết lập triển khai thực tế.

Các thư viện chính được sử dụng gồm:

Table A.1: Các thư viện chính sử dụng trong đồ án

Thư viện	Vai trò
PyTorch	Khung học sâu nền tảng
sentence-transformers	Huấn luyện embedding (MNRL, GIST, MatryoshkaLoss) và SetFit
transformers (Hugging Face)	Tải và tinh chỉnh reranker (cross-encoder)
scikit-learn	Bộ phân loại Logistic Regression, đánh giá phân loại
Pyvi (ViTokenizer)	Tách từ tiếng Việt
NumPy	Tính toán số học

A.2 Siêu tham số huấn luyện embedding

Mô hình embedding được tinh chỉnh từ AITeamVN/Vietnamese_Embedding_v2 (dựa trên BGE-M3, 1024 chiều) theo lộ trình hai giai đoạn, mỗi giai đoạn bọc trong MatryoshkaLoss với ba kích thước {256, 512, 1024}.

Table A.2: Siêu tham số huấn luyện embedding

Tham số	Giai đoạn 1 (GIST)	Giai đoạn 2 (MNRL)
Số epoch	2	1
Tốc độ học	2×10^{-6}	5×10^{-7}
Kích thước batch	64	32
Số bước khởi động	5	5
Kích thước MRL	{256, 512, 1024}	
Độ dài chuỗi tối đa	512 token	
Nhiệt độ GIST	0.01	
Scale MNRL	20 ($\tau = 0.05$)	
Mô hình dẫn hướng GIST	BGE-M3	

A.3 Siêu tham số huấn luyện reranker

Reranker được huấn luyện theo hai giai đoạn: giai đoạn A (học tương phản, thích nghi miền) và giai đoạn B (chưng cất tri thức từ teacher BGE-reranker-v2-m3). Bộ tối ưu dùng AdamW, dừng sớm theo $MRR@10$ trên tập kiểm định.

Table A.3: Siêu tham số huấn luyện reranker

Tham số	Giai đoạn A	Giai đoạn B
Số epoch	5	5
Tốc độ học	2×10^{-5}	1×10^{-5}
Kích thước batch	32	16
Dừng sớm (patience)	2	2
Hạt giống ngẫu nhiên	42	42
Độ dài chuỗi tối đa	512 token	512 token
Trọng số chưng cất α	–	0.7
Nhiệt độ Listwise KL τ	–	2.0
Số ứng viên mỗi câu hỏi	–	20
Bộ tối ưu	AdamW	

A.4 Siêu tham số huấn luyện bộ phân loại ý định

Bộ phân loại ý định dùng SetFit với encoder keepitreal/vietnamese-sbert (dựa trên PhoBERT-base, 768 chiều), tính chỉnh tương phản bằng CosineSimilarityLoss rồi huấn luyện bộ phân loại Logistic Regression trên embedding câu đã chuẩn hóa.

Table A.4: Siêu tham số huấn luyện bộ phân loại ý định

Tham số	Giá trị
Encoder nền	keepitreal/vietnamese-sbert
Số epoch (SetFit)	1
Kích thước batch	16
Tốc độ học	2×10^{-5}
Số vòng lặp sinh cặp	20
Tỉ lệ khởi động	0.1
Hàm mất mát tương phản	CosineSimilarityLoss
Bộ phân loại	Logistic Regression
Trọng số lớp	cân bằng (balanced)
Hệ số điều chuẩn C	4.0
Tách từ	Pyvi (ViTokenizer)
Hạt giống ngẫu nhiên	42

PHỤ LỤC B. VÍ DỤ DỮ LIỆU HUẤN LUYỆN

Phụ lục này minh họa cấu trúc dữ liệu huấn luyện đã mô tả ở Chương 3, gồm ví dụ bộ ba cho embedding và ví dụ hai lớp ý định. Các đoạn văn dài được rút gọn để minh họa; dấu [...] biểu thị phần đã lược bớt.

B.1 Ví dụ bộ ba cho embedding

Mỗi câu hỏi được ghép với một positive (đoạn chứa câu trả lời) và các negative. Ví dụ dưới đây lấy từ miền tài liệu về vật liệu chống cháy.

Table B.1: Ví dụ một bộ ba huấn luyện embedding

Thành phần	Nội dung
Câu hỏi	Chiều dày màng khô của lớp phủ chống cháy được xác định bằng cách nào?
Positive	“Độ dày màng khô phải được xác định trực tiếp trên các mẫu thử nghiệm, sau khi lớp phủ được để khô hoàn toàn theo quy định của khách hàng. Phòng thử nghiệm phải đo chiều dày bằng dụng cụ sử dụng nguyên tắc cảm ứng điện từ hoặc nguyên tắc dòng điện xoáy với đường kính tiếp xúc đầu dò ít nhất là 2,5 mm. [...]”
Negative (khai thác)	“Đối với vật liệu bọc phủ dạng phản ứng, độ dày trung bình của lớp sơn lót phải được đo trước và trừ đi từ tổng chiều dày [...]”

Negative khai thác là một đoạn thật trong corpus, gần positive trong không gian truy hồi nhưng không trực tiếp cung cấp thông tin cần thiết để trả lời câu hỏi.

B.2 Ví dụ hai lớp ý định

Câu hỏi được phân thành hai lớp: tra_cuu (tra cứu thông tin có sẵn trong tài liệu) và tinh_toan (cần suy luận hoặc tính toán để tạo ra kết quả).

Table B.2: Ví dụ câu hỏi hai lớp ý định

Lớp	Ví dụ câu hỏi
tra_cuu	Chiều dày màng khô của lớp phủ chống cháy được xác định bằng cách nào?
tra_cuu	Trong mô hình nhà thông minh, IoT chủ yếu đóng vai trò gì?
tinh_toan	Trong tài liệu Public_002, đối với đoạn liệt kê các học phần từ số 47 đến 81, nếu các học phần này được trải đều trên 7 trang, trung bình mỗi trang mô tả bao nhiêu học phần?
tinh_toan	Tính chi phí nhiên liệu cho quãng đường 120 km, mức tiêu thụ 7 L/100 km, giá xăng 24.000 đồng/L?

Điểm khác biệt giữa hai lớp: câu tra_cuu có thể trả lời bằng cách trích thông tin

trực tiếp từ tài liệu, trong khi câu tính_toan yêu cầu thực hiện phép tính trên các số liệu lấy được (ví dụ đếm số học phần rồi chia cho số trang, hoặc tính tích quãng đường với mức tiêu thụ và đơn giá).

PHỤ LỤC C. CÁC PROMPT SỬ DỤNG VỚI MÔ HÌNH NGÔN NGỮ

Phụ lục này tập hợp các prompt dùng với mô hình ngôn ngữ trong hai vai trò: (1) vận hành hệ thống hỏi đáp (định tuyến ý định và sinh câu trả lời), và (2) xây dựng dữ liệu huấn luyện (xác định positive, sinh query-positive bổ sung). Các trường trong dấu ngoặc nhọn như `\{question\}` là chỗ điền nội dung động khi chạy.

C.1 Prompt định tuyến ý định

Prompt phân loại câu hỏi thành một trong hai lớp `tra_cuu` hoặc `tinh_toan`.

Phân loại intent của câu hỏi sau. Trả lời chỉ một từ:

- "`tra_cuu`" nếu câu hỏi yêu cầu tra cứu, tìm kiếm thông tin,
→ định nghĩa, giải thích
- "`tinh_toan`" nếu câu hỏi yêu cầu tính toán, suy luận số liệu,
→ so sánh theo phép tính

Câu hỏi: `{question}`

Intent:

C.2 Prompt sinh câu trả lời theo ý định

Với câu `tra_cuu`, hệ thống dùng một lượt gọi để chọn đáp án trực tiếp:

Dựa vào thông tin sau đây, trả lời câu hỏi trắc nghiệm.

CONTEXT:

`{context}`

QUESTION:

`{question}`

OPTIONS:

`{options_text}`

Chỉ trả lời đúng 1 dòng, chỉ ghi chữ cái đáp án (A/B/C/D).

Nếu có nhiều đáp án đúng thì ghi tất cả, ví dụ: AB hoặc ACD.

Không giải thích.

ANSWER:

Với câu `tinh_toan`, hệ thống dùng hai lượt gọi. Lượt một sinh phân tích từng bước:

Dựa vào context sau, phân tích câu hỏi từng bước.

CONTEXT:

{context}

QUESTION:

{question}

OPTIONS:

{options_text}

Phân tích từng bước, giữ nguyên số liệu quan trọng.

Chưa cần kết luận đáp án cuối cùng:

Lượt hai chọn đáp án cuối từ phần phân tích:

Dựa vào phân tích sau, chọn đáp án đúng cho câu hỏi trắc nghiệm.

PHÂN TÍCH:

{reasoning}

QUESTION:

{question}

OPTIONS:

{options_text}

Chỉ trả lời đúng 1 dòng, chỉ ghi chữ cái đáp án (A/B/C/D).

Nếu có nhiều đáp án đúng thì ghi tất cả, ví dụ: AB hoặc ACD.

Không giải thích.

ANSWER:

C.3 Prompt giám khảo xác định positive

Prompt yêu cầu mô hình phân loại từng đoạn ứng viên thành gold, partial hoặc irrelevant, trả về kết quả dạng JSON.

Judge relevance cho RAG tiếng Việt.

Phân loại từng chunk:

- gold: đủ thông tin trả lời trực tiếp
- partial: liên quan nhưng không đủ
- irrelevant: không liên quan

JSON only:

{"gold_indices": [...], "partial_indices": [...],

```
"irrelevant_indices": [...], "confidence": "high|medium|low"}
```

Phần prompt người dùng kèm câu hỏi và danh sách đoạn:

```
QUERY: {question}
```

```
INTENT: {intent}
```

CÁC ĐOẠN VĂN:

```
[CHUNK_0] (bge_score={score})
```

```
{chunk_text}
```

```
...
```

Phân loại từng chunk.

C.4 Prompt sinh query-positive bổ sung

Prompt này được dùng để tạo thêm câu hỏi cho các đoạn chưa xuất hiện trong tập huấn luyện hoặc đánh giá. System prompt đặt vai trò và yêu cầu trả về JSON hợp lệ:

Bạn tạo dữ liệu query-positive cho hệ thống truy hồi tài liệu.

Nhiệm vụ: đọc một đoạn văn pháp lý/kỹ thuật và viết các câu hỏi

→ mà người dùng thật có thể gõ để tìm đoạn đó.

Luôn trả về JSON hợp lệ, không giải thích thêm.

User prompt nhận nội dung đoạn và số câu hỏi cần sinh:

Bạn là người dùng đang tra cứu tài liệu pháp lý/kỹ thuật. Đọc

→ đoạn văn sau và đặt câu hỏi mà đoạn này trả lời được.

ĐOẠN VĂN:

```
{chunk_text}
```

YÊU CẦU:

- Đặt {n_questions} câu hỏi KHÁC NHAU mà đoạn văn trên trả lời được.

- Viết như người dùng thật gõ tìm kiếm: ngắn gọn, tự nhiên.
- KHÔNG "xin vui lòng", KHÔNG trang trọng.

- QUAN TRỌNG: KHÔNG sao chép nguyên cụm từ dài trong đoạn văn.
- Diễn đạt lại bằng từ ngữ của bạn, dùng từ đồng nghĩa. Câu hỏi và đoạn văn KHÔNG được trùng lặp nhiều từ khóa.

- Câu hỏi phải trả lời được CHỈ bằng đoạn văn này, không cần thông tin ngoài.
- thông tin ngoài.

- Nếu đoạn văn có số liệu/công thức/điều kiện tính toán, đặt ít
↳ nhất 1 câu hỏi dạng tính toán/áp dụng.
- Nếu đoạn văn quá ngắn/không có nội dung đáng hỏi (mục lục,
↳ tiêu đề, bảng trống), trả về danh sách rỗng.

Trả về JSON:

```
{"questions": ["câu hỏi 1", "câu hỏi 2"], "chunk_quality":  
↳ "good" | "low"}
```